



# 自然语言处理

WELCOME!

杨海钦

2025-2026-1 学期

基于 Julia, Junjie 等课件更新

# 面试问题

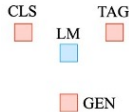
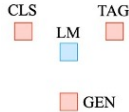
- **自监督预训练**的核心动机是什么？相比于传统的监督学习，它解决了工业场景中的哪些痛点问题？
- 工业界常见的**预训练模型架构**有哪几种？代表模型是什么？它们的架构逻辑及适配场景是什么？
  - 什么是top-k采样？什么是top-p采样？它们是为了解决什么问题？
- 请说明BERT的**掩码语言建模**(MLM)，T5的去噪

# 大纲

- 预训练(自监督学习)的动机
- 预训练的架构和训练目标
  - 编码器架构
  - 编码器-解码器架构
  - 解码器架构
- 附录
  - Transformer
  - 开放式文本生成

# NLP的主要范式

## Major Paradigm in NLP

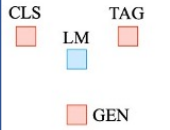
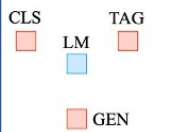
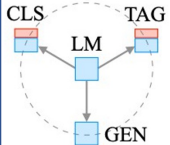
| 范式 Paradigm                                       | 工程化 Engineering                                                        | 任务关系 Task Relation                                                                  |
|---------------------------------------------------|------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| a. Fully Supervised Learning (Non-Neural Network) | <b>Features</b><br>(e.g., word identity, POS, sentence length)         |  |
| b. Fully Supervised Learning (Neural Network)     | <b>Architecture</b> (e.g., convolutional, recurrent, self-attentional) |  |

Pre 2017



# NLP的主要范式

## Major Paradigm in NLP

| 范式 Paradigm                                       | 工程化 Engineering                                                             | 任务关系 Task Relation                                                                  |          |
|---------------------------------------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------|----------|
| a. Fully Supervised Learning (Non-Neural Network) | <b>Features</b><br>(e.g., word identity, POS, sentence length)              |  | Pre 2017 |
| b. Fully Supervised Learning (Neural Network)     | <b>Architecture</b> (e.g., convolutional, recurrent, self-attentional)      |  |          |
| c. Pre-train, Fine-tune                           | <b>Objective</b> (e.g., masked language modeling, next sentence prediction) |  | 2017~19  |
|                                                   |                                                                             |                                                                                     | 2017~    |
|                                                   |                                                                             |                                                                                     | 2022~    |
|                                                   |                                                                             |                                                                                     | 2025~    |

Pengfei Liu, et al.: Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in NLP. ACM Comput. Surv. (2023)

Pengcheng Jiang, et al. Adaptation of Agentic AI. <https://arxiv.org/abs/2512.16301>

# NLP的主要范式

## Major Paradigm in NLP

| 范式 Paradigm                                       | 工程化 Engineering                                                             | 任务关系 Task Relation |          |
|---------------------------------------------------|-----------------------------------------------------------------------------|--------------------|----------|
| a. Fully Supervised Learning (Non-Neural Network) | <b>Features</b><br>(e.g., word identity, POS, sentence length)              |                    | Pre 2017 |
| b. Fully Supervised Learning (Neural Network)     | <b>Architecture</b> (e.g., convolutional, recurrent, self-attentional)      |                    |          |
| c. Pre-train, Fine-tune                           | <b>Objective</b> (e.g., masked language modeling, next sentence prediction) |                    | 2017~19  |
| d. Pre-train, Prompt, Predict                     | <b>Prompt</b> (e.g., cloze, prefix)                                         |                    | 2019~    |
| e. Pre-train, Alignment, (Fine-tune), Predict     |                                                                             |                    | 2022~    |
| f. Agentic AI                                     |                                                                             |                    | 2025~    |

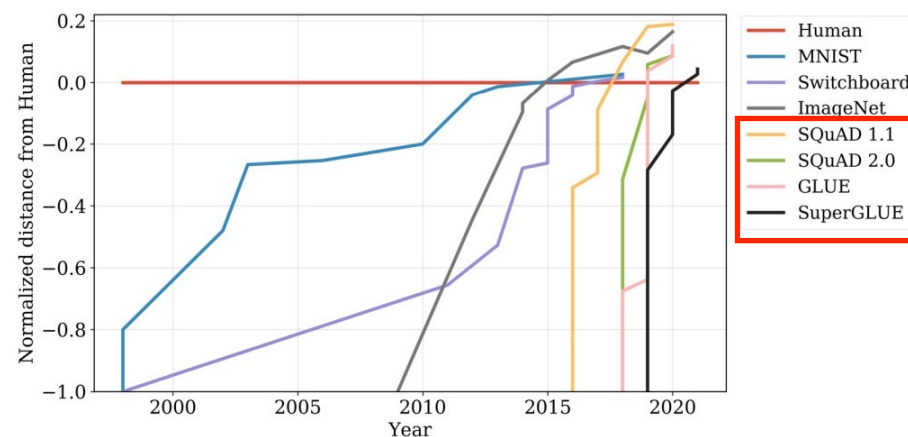
Pengfei Liu, et al.: Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in NLP. ACM Comput. Surv. (2023)

Pengcheng Jiang, et al. Adaptation of Agentic AI. <https://arxiv.org/abs/2512.16301>

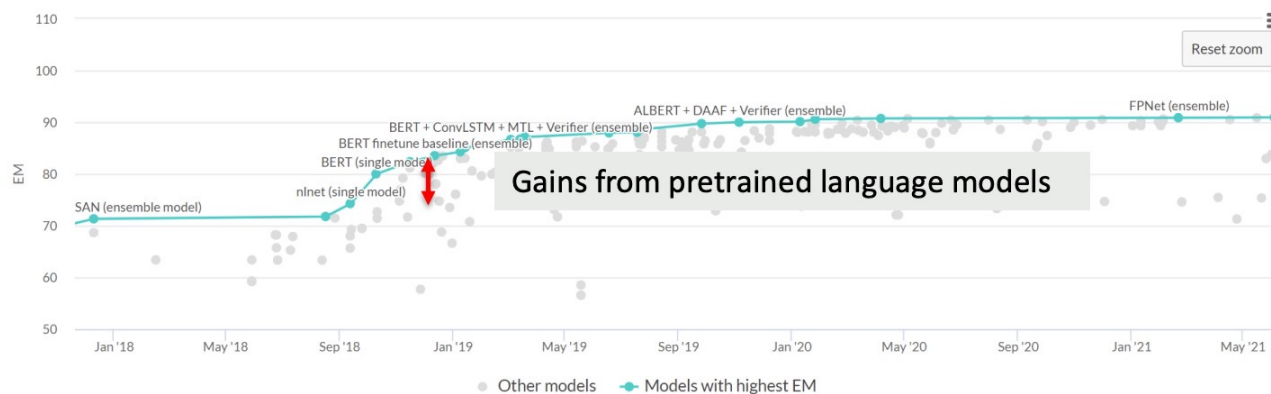
# 预训练的革命

## The Pre-training Revolution

- 预训练对NLP系统的工作效果产生了重大而切实的影响



NLP任务



# 全监督学习方法的问题



**Food Review:** "I recently had the pleasure of dining at Fusion Bites, and the experience was nothing short of spectacular. The menu boasts an exciting blend of global flavors, and each dish is a masterpiece in its own right."

- 假设给定带有情感标签的10万条食物评论的数据集，我们如何训练一个模型并对未见过的食物评论进行情感分析？
- 我们可以直接训练一个随机初始化的模型，输入是食物评论文本，输出为“积极”或“消极”的情绪标签

# 全监督学习方法的问题



**Food Review:** "I recently had the pleasure of dining at Fusion Bites, and the experience was nothing short of spectacular. The menu boasts an exciting blend of global flavors, and each dish is a masterpiece in its own right."



**Movie Review:** "The narrative unfolds with a steady pace, showcasing a blend of various elements. While the performances are competent, and the cinematography captures the essence of the story, the overall impact falls somewhere in the middle."

- 如果需求是**电影评论**分类，我们是否可以用**食物评论**训练出来的系统来预测情绪呢？

由于**分布偏移问题**，可能不能很好地泛化！

# 迁移学习：计算机视觉的历史教训

## Transfer Learning

- 不同于每次遇到新任务或新领域时，都需要随机初始化神经网络进行训练
  - 我们可以将从一个任务/领域学到的表征重用于另一个任务/领域吗？

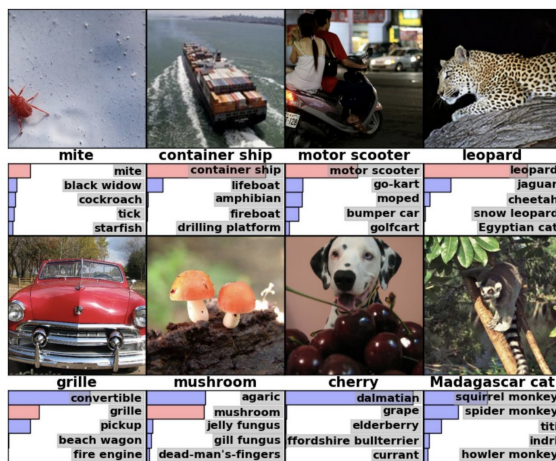
# 迁移学习: 计算机视觉的历史教训 Transfer Learning

- 思路: 在大规模数据集上训练一个(非常)深的神经网络, 并重用该网络中学到的表征去适应新的任务

## ImageNet Challenge

IMAGENET

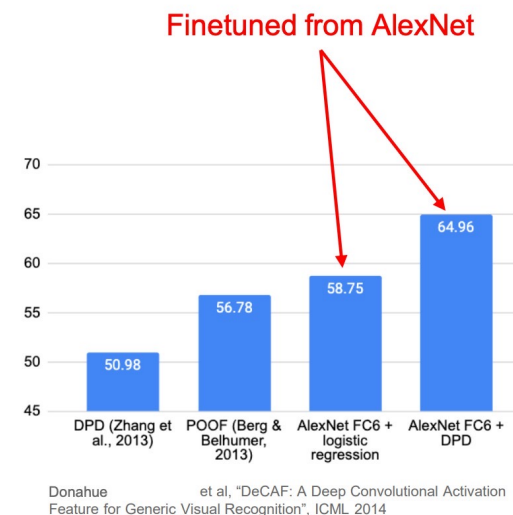
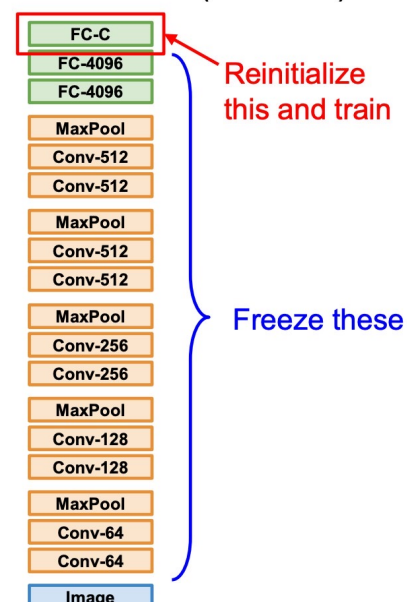
- 1,000 object classes (categories).
- Images:
  - 1.2 M train
  - 100k test.



1. Train on Imagenet



2. Small Dataset (C classes)



# 为什么NLP花了这么长时间？

- 自2014年以来，在计算机视觉领域，下载预训练(来自 Image Net)的深度神经网络模型并根据手头的问题对其进行“微调”，而不是从头开始，这已经成为一种常见的范式
- 直到2017年末，NLP才出现这种情况
- 通常在NLP任务中使用预训练的词向量，如word2vec, GloVe, 虽然这些有助于提高性能，但大多数情况只是稍微提升



# 为什么NLP花了这么长时间？

2017年之后发生了什么呢？

- 我们可以将这个延迟归结为两个因素

## 1. 缺乏大规模通用数据集

- 不清楚什么是最能代表NLP任务领域的合适的NLP任务(分类、QA、NLI、句法分析、语言建模？)获得如此大规模的高质量标签也是一个挑战

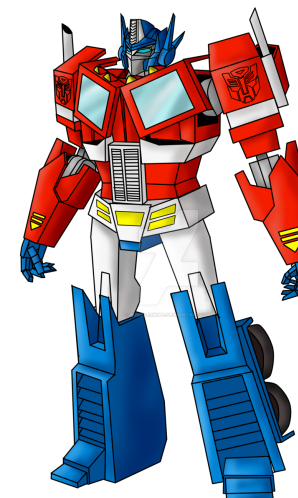
## 2. 用于NLP的神经网络模型通常非常浅层

- 2017年之前，NLP中使用的主要模型是循环神经网络，例如LSTM
- 这些模型通常有1-2个隐藏层，将它们扩展到更大的层不容易，因为这些模型是出了名的难以训练

自监督学习

Self-supervised Learning

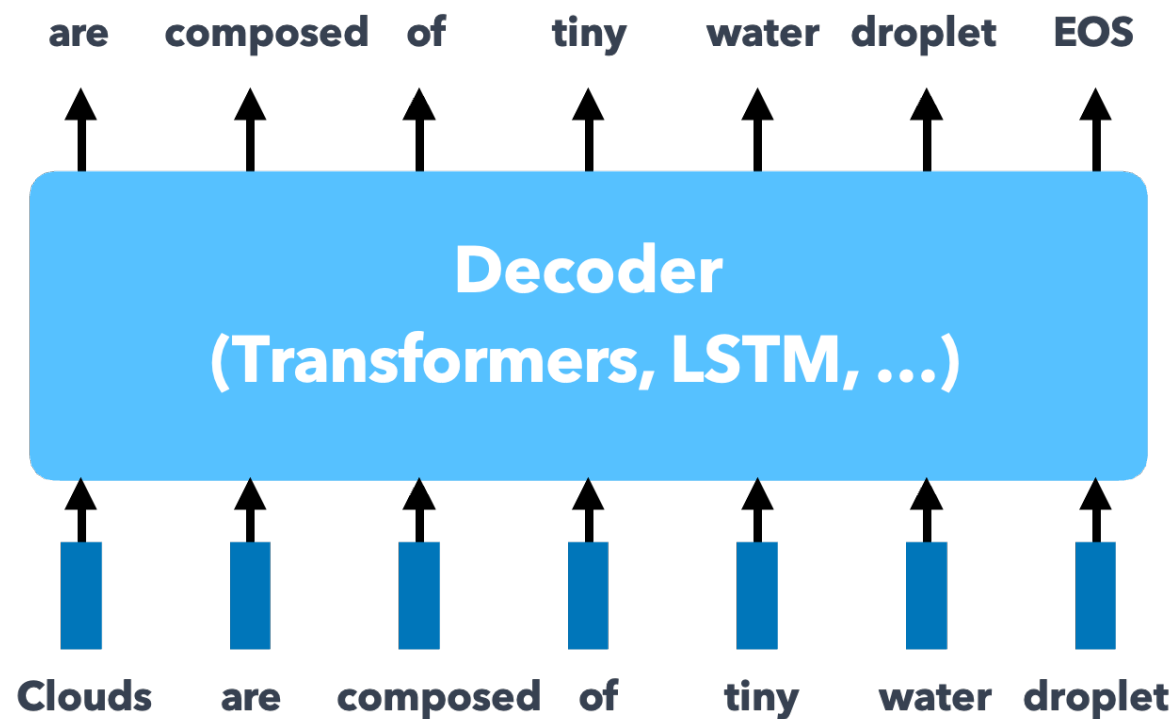
Transformer



# 自监督学习预训练：学习基本模式、结构和语义知识

- 通过语言建模进行预训练[Dai and Le, 2015]
  - $P_{\theta}(w_t|w_{1:t-1})$ : 给定前文对下一个词的概率分布建模
  - 大量(英文)语料是这个模式
    - 书籍、网页
  - 用大量原始文本数据对神经网络进行语言建模的**自监督训练**
  - 学习的网络参数可以在其他任务重用

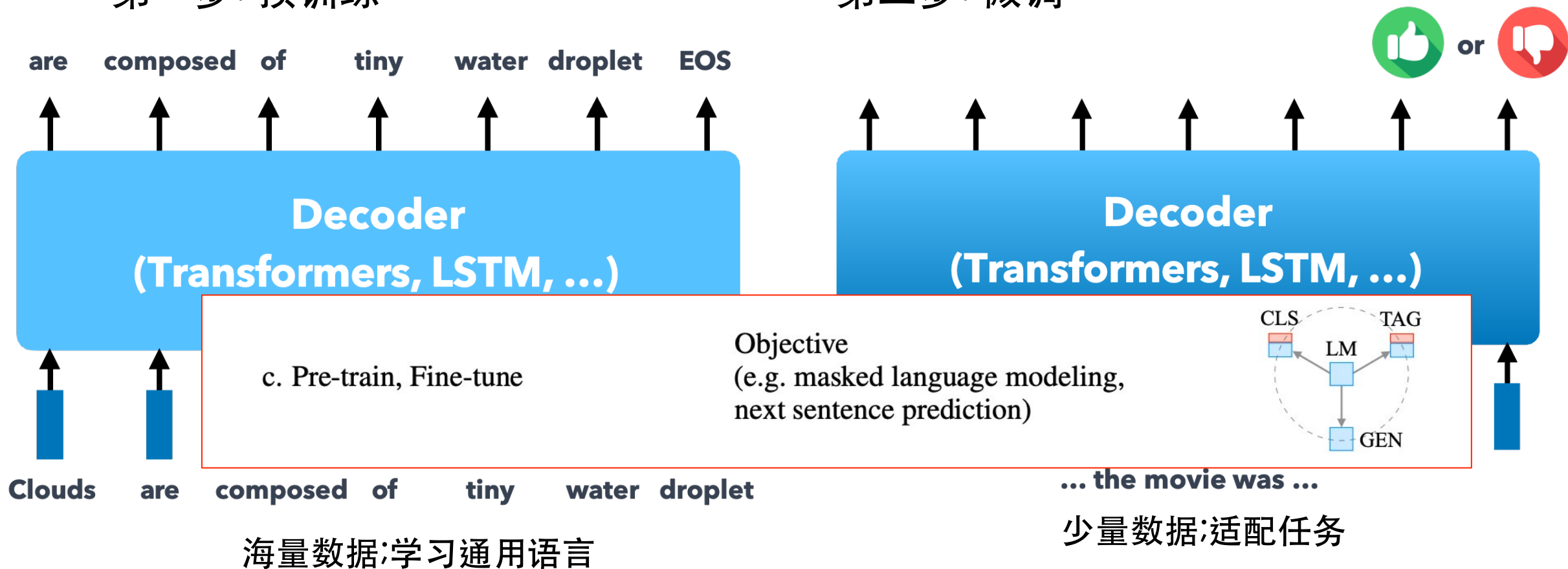
为什么这个过程称为自监督？



# 针对特定任务的监督微调 Supervised Fine-tuning

- 第一步: 预训练

- 第二步: 微调



# 为什么这种方法有效？

# 原始文本中蕴含大量信息

- 动词 • I went to Hawaii for snorkeling, hiking, and whale watching .
- 介词 • I walked across the street, checking for traffic over my shoulders.
- 常识 • I use knife and fork to eat steak.
- 时间 • Ruth Bader Ginsburg was born in 1933 .
- 地点 • University of Washington is located at seattle , Washington.
- 数学 • I was thinking about the sequence that goes 1, 1, 2, 3, 5, 8, 13, 21, 34 .
- 化学 • Sugar is composed of carbon, hydrogen, and oxygen .

# 为什么先预训练后微调会有效？

- 过程

- 通过逼近预训练损失  $\mathcal{L}_{\text{pretrain}}(\theta)$  获得参数  $\hat{\theta}$
- 接着，从参数  $\hat{\theta}$  开始，逼近微调损失  $\mathcal{L}_{\text{finetune}}(\theta)$

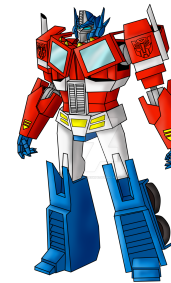
- 在微调过程中，随机梯度下降(相对)接近  $\hat{\theta}$

- 所以，微调的局部极小值可能在  $\hat{\theta}$  附近，会泛化得很好！
- 或者，也许微调损失的梯度在  $\hat{\theta}$  附近会传播得很好！

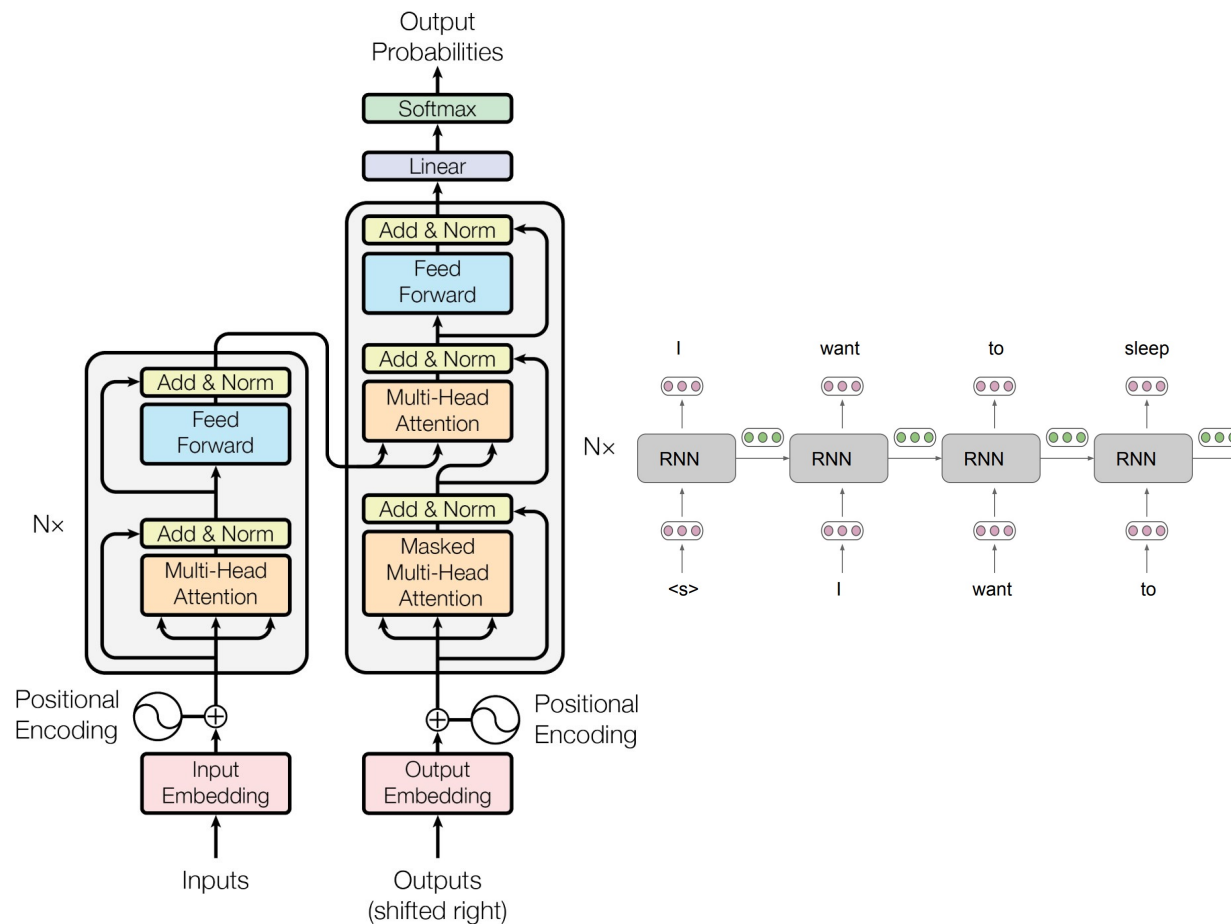
# 预训练-微调范式的优点

- 从丰富的原始文本**获取丰富的底层信息**
- **减少**对难以获得或昂贵的特定任务**标签数据的依赖**
- 为更具备泛化能力的NLP应用**初始化模型参数**
- 通过提供可重用的模型检查点**节省培训成本**
- **提供**语言上下文的**鲁棒表示**

# NLP中解决浅层网络问题的方案： Transformer



- Transformer旨在避免循环神经网络的两个问题：
  - 训练的高度并行化：不需要等前一个时间步的计算结果即可完成下一个时间步的计算
  - 避免梯度消失的训练复杂性：不像RNN(有固定状态需要反复更新)，Transformer有动态记忆，可以避免梯度消失





# 大纲

- 预训练(自监督学习)的动机
- 预训练的架构和训练目标
  - 编码器架构
  - 编码器-解码器架构
  - 解码器架构
- 附录
  - Transformer
  - 开放式文本生成

# 预训练模式/架构

## Pre-training Paradigms/Architectures

Encoder  
/编码器

- e.g., BERT, RoBERTa, DeBERTa, ...
- 自编码器模型/Autoencoder models
- 掩码语言建模/Masked language modeling

Encoder Decoder  
/编码器 /解码器

- e.g., T5, BART, ...
- 序列到序列模型/seq2seq model

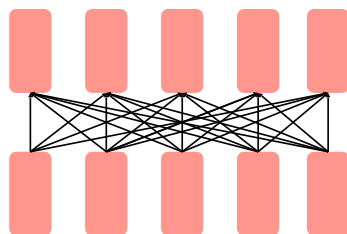
Decoder  
/解码器

- e.g., GPT, GPT2, GPT3, ...
- 自回归模型/Autoregressive model
- 从左到右语言建模/Left-to-right language modelling

# 三种预训练模式/架构

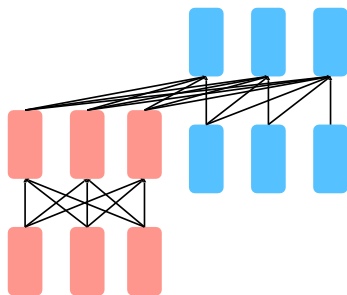
## 3 Pre-training Paradigms/Architectures

Encoder  
/编码器



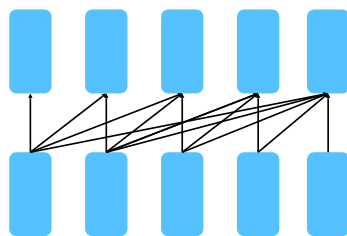
- **双向的/Bidirectional**: 可以基于未来上下文/context为条件生成

Encoder — Decoder  
/编码器 — /解码器



- 将两个不同长度的**序列**映射在一起

Decoder  
/解码器

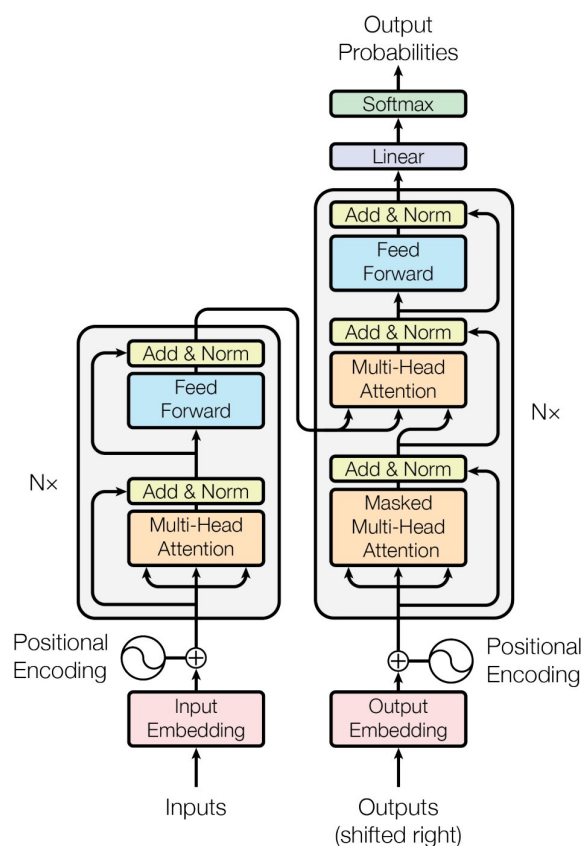


- **语言建模/Language modelling**: 仅能根据过去的上下文/context为条件生成

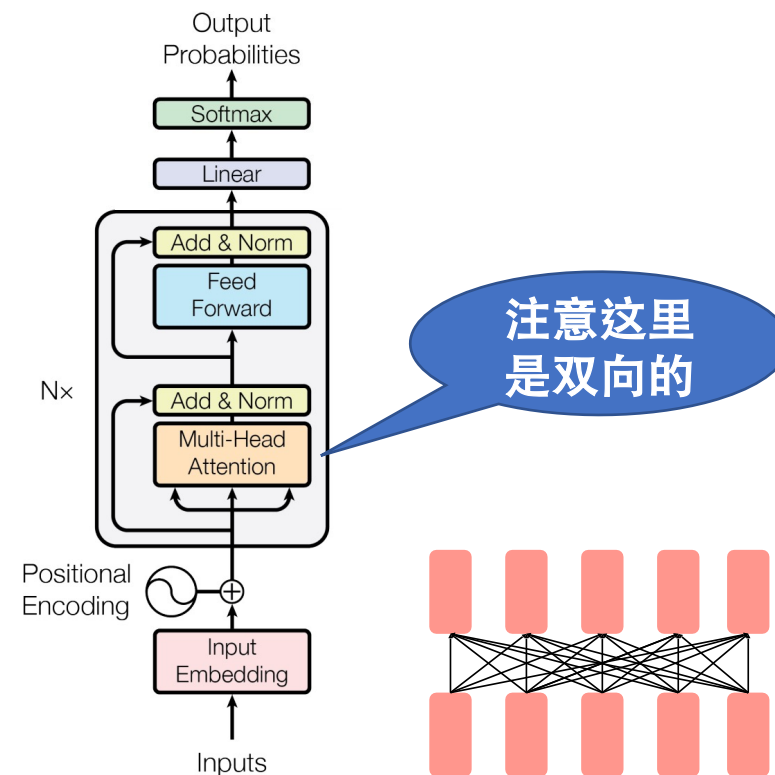
# 编码器：架构

## Encoder: Architecture

### Full-Transformer Architecture (Encoder-Decoder)

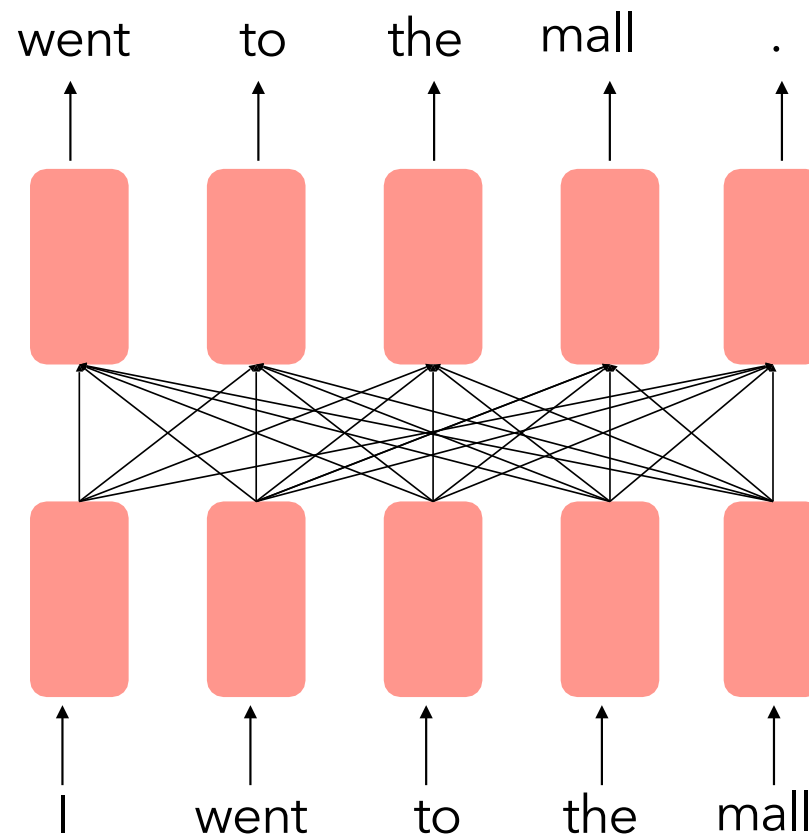


### Encoder-Only Transformer Architecture



# 编码器架构: 训练目标

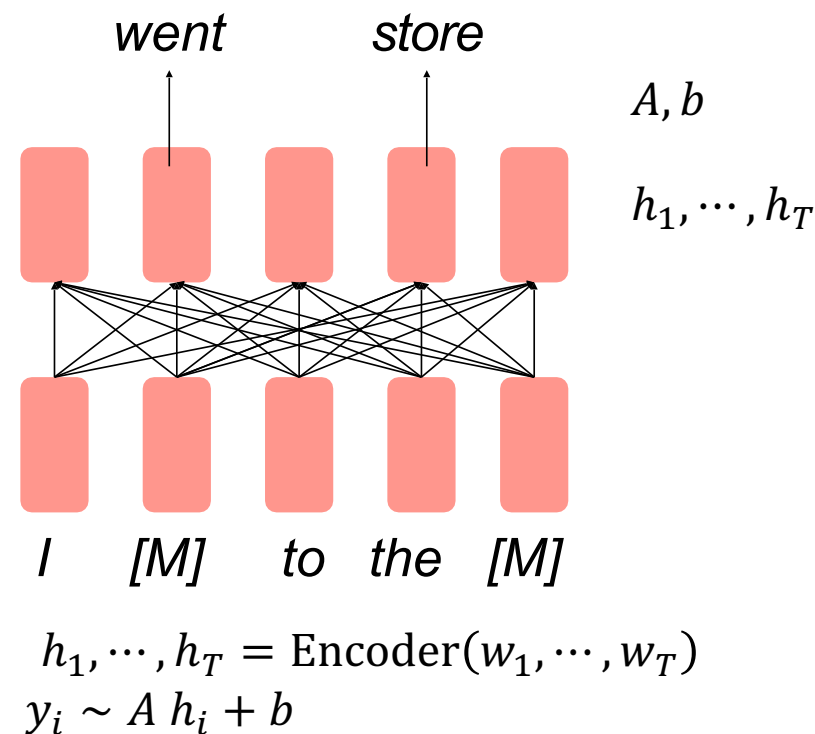
- **回顾:** 用于预训练的语言建模
- 语言模型预训练对编码器架构来说是有问题的
- 为什么?
  - 编码器架构需要获得双向上下文
  - 模型可以通过在预测时只看下一个词元(token)而不实际学习任何语言知识进行作弊!



# 编码器架构: 训练目标

[Devlin et al., 2018]

- 如何对**双向**上下文中的信息进行编码？
- 总体思路: **文本重构/text reconstruction** !
  - Your time is [MASK], so don't [MASK] it living someone else's life. Don't be trapped by [MASK], which is [MASK] with the results of other [MASK]'s thinking. — [MASK] Jobs

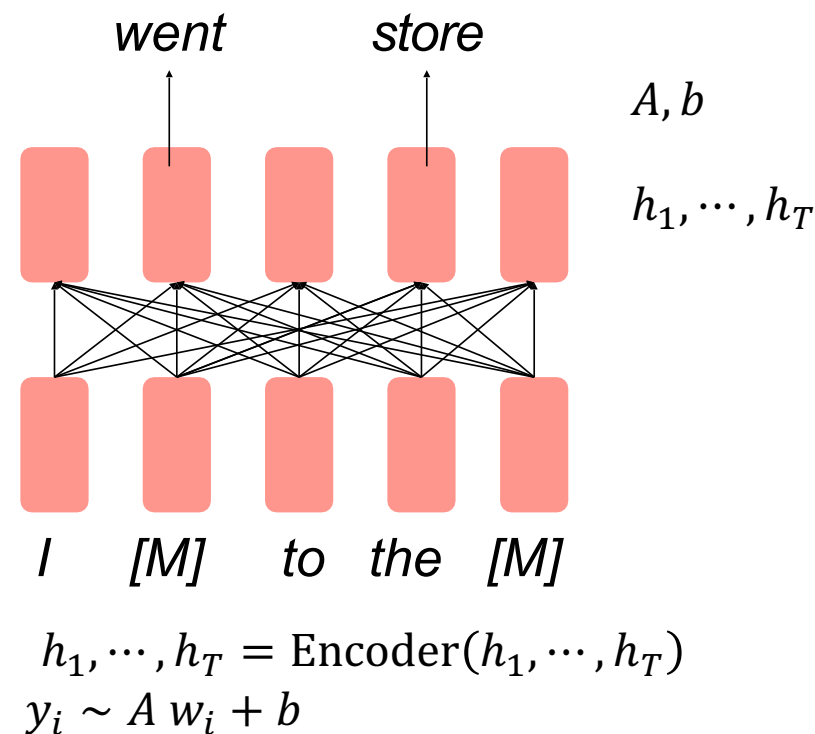


- 只添加来自掩蔽词元的损失值
- 如果 $\tilde{x}$ 是 $x$ 的掩码版本，我们学习的是 $p_\theta(x|\tilde{x})$ . 称为**掩码语言模型(Masked Language Model, MLM)**

# 编码器架构: 训练目标

[Devlin et al., 2018]

- 如何对**双向**上下文中的信息进行编码?
- 总体思路: **文本重构/text reconstruction** !
  - Your time is **limited**, so don't [MASK] it living someone else's life. Don't be trapped by [MASK], which is [MASK] with the results of other [MASK]'s thinking. — [MASK] Jobs

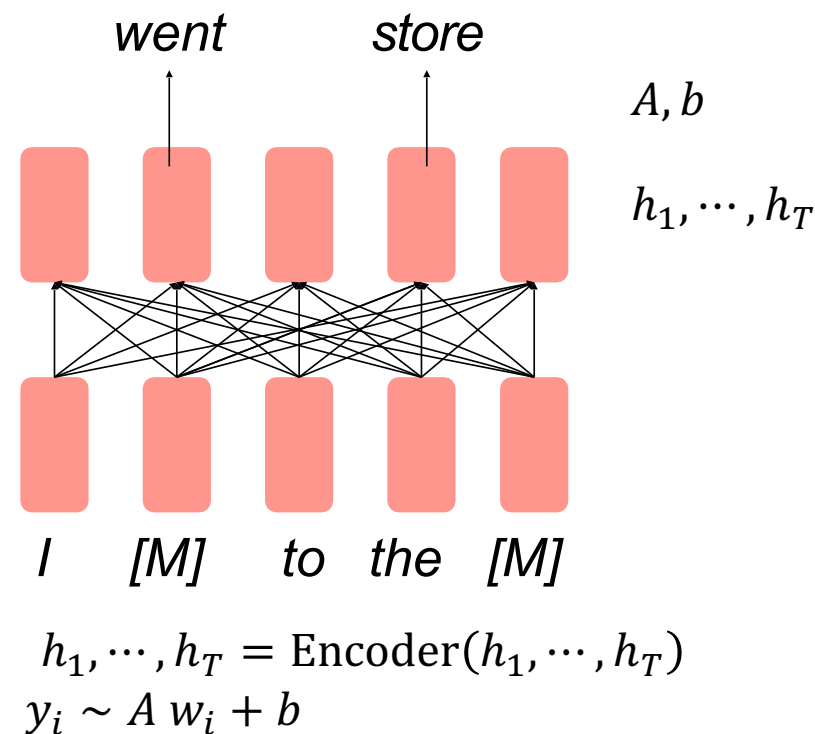


- 只添加来自掩蔽词元的损失值
- 如果 $\tilde{x}$ 是 $x$ 的掩码版本，我们学习的是 $p_\theta(x|\tilde{x})$ . 称为**掩码语言模型(Masked Language Model, MLM)**

# 编码器架构: 训练目标

[Devlin et al., 2018]

- 如何对**双向**上下文中的信息进行编码？
- 总体思路: **文本重构/text reconstruction** !
  - Your time is **limited**, so don't **waste** it living someone else's life. Don't be trapped by **dogma**, which is [MASK] with the results of other [MASK]'s thinking. — [MASK] Jobs



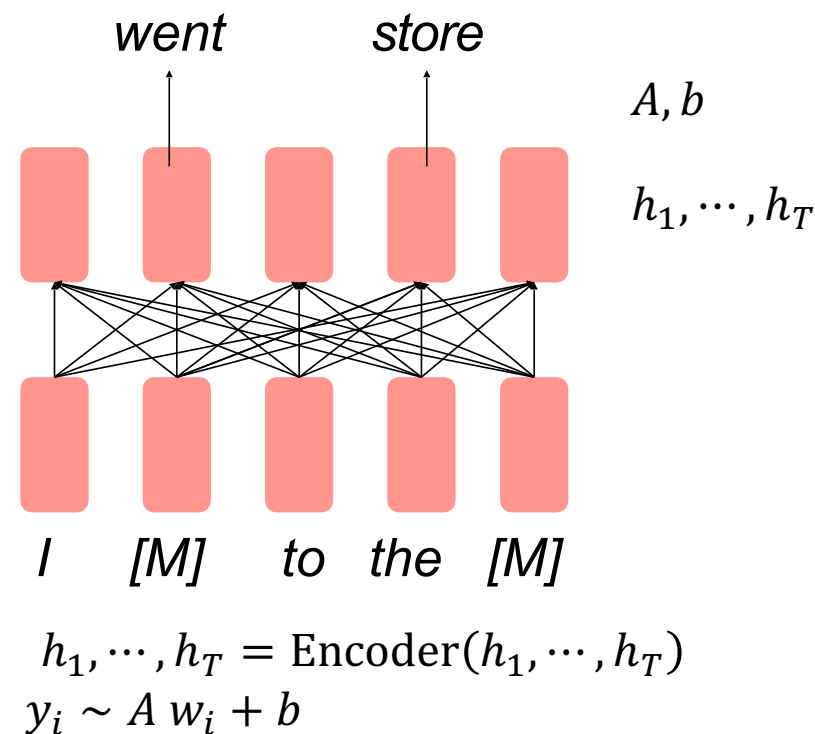
- 只添加来自掩蔽词元的损失值
- 如果 $\tilde{x}$ 是 $x$ 的掩码版本，我们学习的是 $p_\theta(x|\tilde{x})$ . 称为**掩码语言模型(Masked Language Model, MLM)**



# 编码器架构: 训练目标

[Devlin et al., 2018]

- 如何对**双向**上下文中的信息进行编码？
- 总体思路: **文本重构/text reconstruction** !
  - Your time is **limited**, so don't **waste** it living someone else's life. Don't be trapped by **dogma**, which is **living** with the results of other [MASK]'s thinking. — [MASK] Jobs

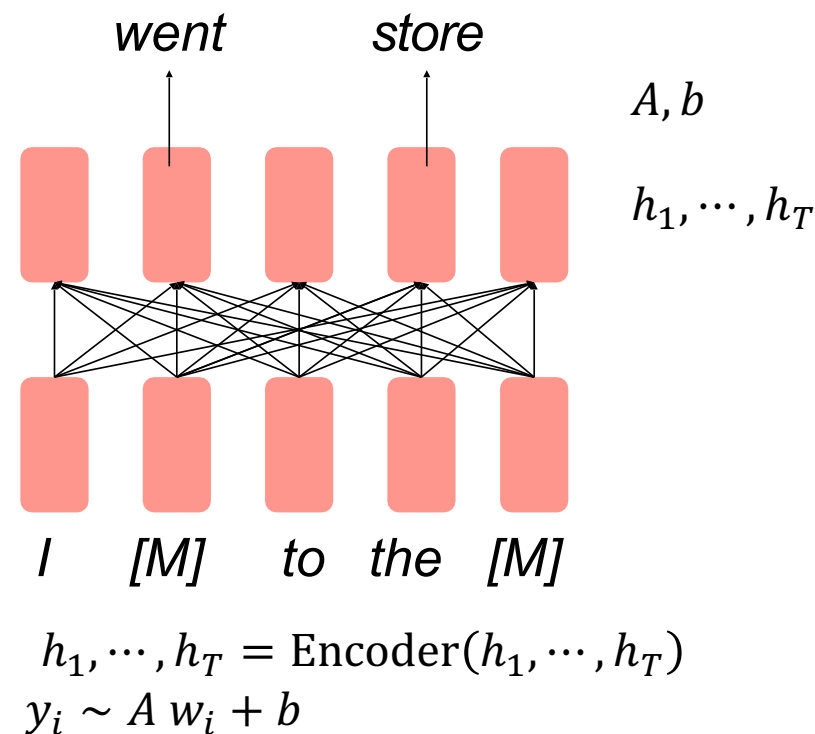


- 只添加来自掩蔽词元的损失值
- 如果 $\tilde{x}$ 是 $x$ 的掩码版本，我们学习的是 $p_\theta(x|\tilde{x})$ . 称为**掩码语言模型(Masked Language Model, MLM)**

# 编码器架构: 训练目标

[Devlin et al., 2018]

- 如何对**双向**上下文中的信息进行编码？
- 总体思路: **文本重构/text reconstruction** !
  - Your time is **limited**, so don't **waste** it living someone else's life. Don't be trapped by **dogma**, which is **living** with the results of other **people's** thinking. — [MASK] Jobs

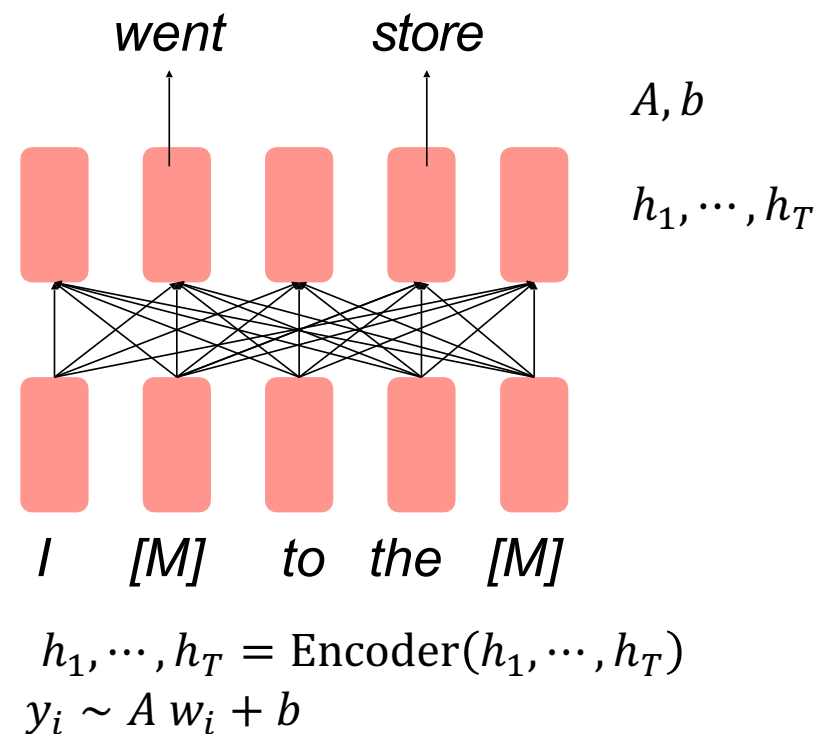


- 只添加来自掩蔽词元的损失值
- 如果 $\tilde{x}$ 是 $x$ 的掩码版本，我们学习的是 $p_\theta(x|\tilde{x})$ . 称为**掩码语言模型(Masked Language Model, MLM)**

# 编码器架构: 训练目标

[Devlin et al., 2018]

- 如何对**双向**上下文中的信息进行编码？
- 总体思路: **文本重构/text reconstruction** !
  - Your time is **limited**, so don't **waste** it living someone else's life. Don't be trapped by **dogma**, which is **living** with the results of other **people's** thinking. — **Steve** Jobs

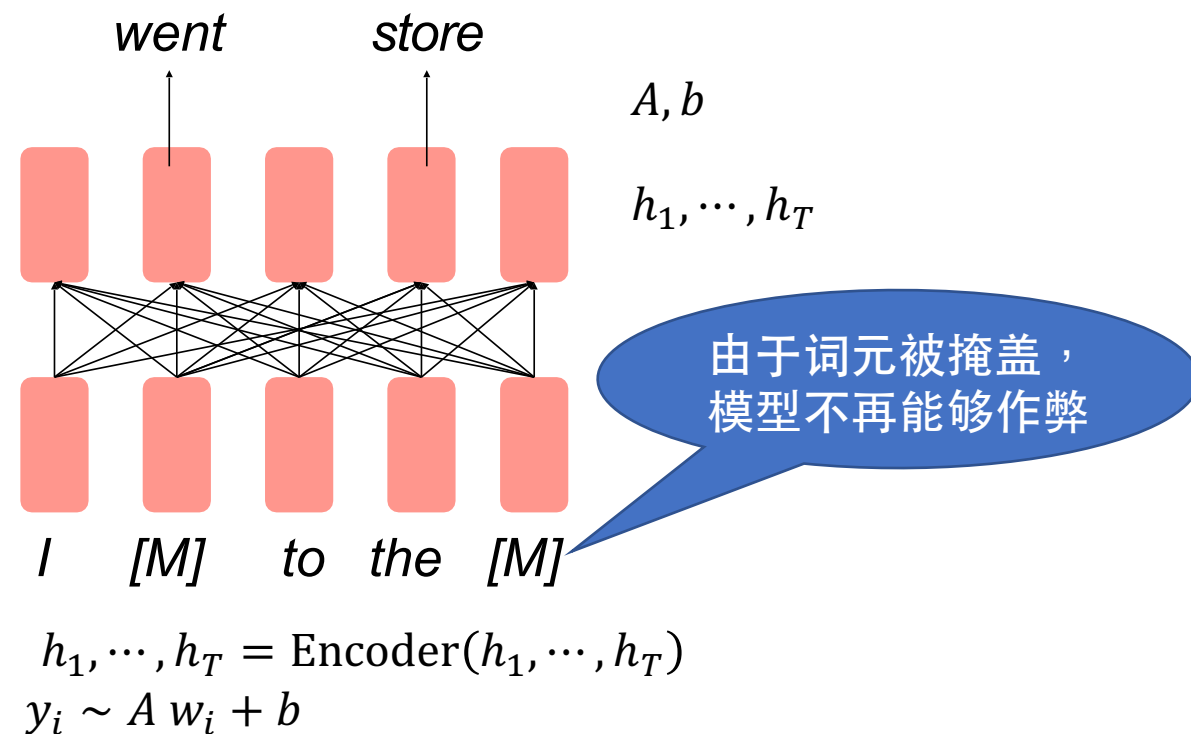


- 只添加来自掩蔽词元的损失值
- 如果 $\tilde{x}$ 是 $x$ 的掩码版本，我们学习的是 $p_\theta(x|\tilde{x})$ . 称为**掩码语言模型(Masked Language Model, MLM)**

# 编码器架构: 训练目标

[Devlin et al., 2018]

- 如何对**双向**上下文中的信息进行编码？
- 总体思路: **文本重构/text reconstruction** !
  - Your time is **limited**, so don't **waste** it living someone else's life. Don't be trapped by **dogma**, which is **living** with the results of other **people's** thinking. — **Steve** Jobs



- 只添加来自掩蔽词元的损失值
- 如果 $\tilde{x}$ 是 $x$ 的掩码版本，我们学习的是 $p_{\theta}(x|\tilde{x})$ . 称为**掩码语言模型(Masked Language Model, MLM)**

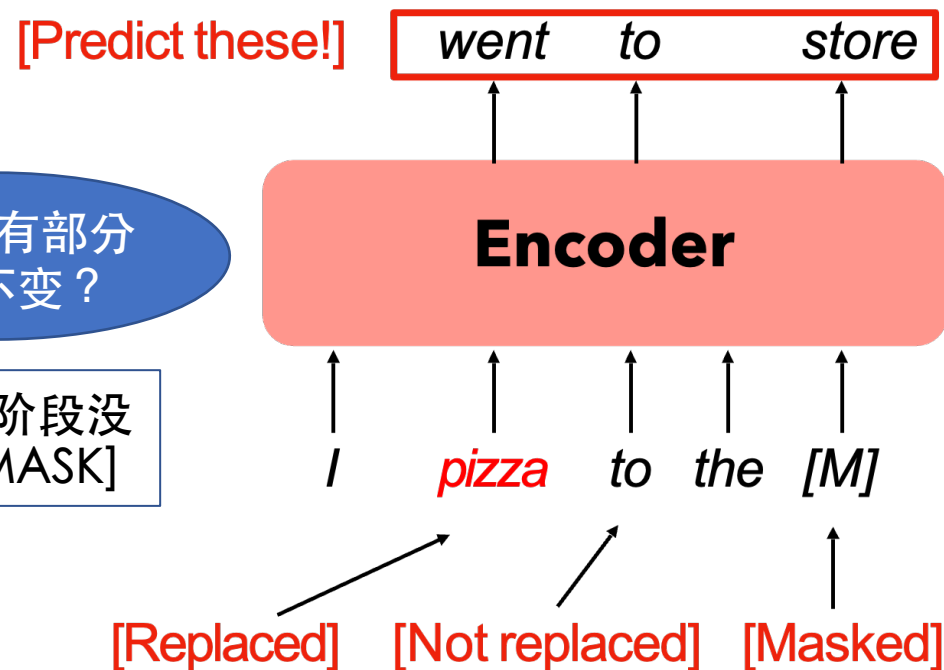
# 编码器架构: BERT

Bidirectional Encoder  
Representations from Transformers [Devlin et al., 2018]

- 2个预训练目标:
  - **掩码语言模型**: 随机预测15%的词元/token
    - 对每个词元:
      - 80%的情况: 用[MASK]替换
      - 10%的情况: 用随机的词元替换
      - 10%的情况: 不变, 但仍做预测
  - Next Sentence Prediction (NSP)
    - 50%的情况: 两个相邻的句子的顺序是正确的
  - **这实际上会损害模型的学习!**

为什么有部分保留不变?

微调阶段没有[MASK]



# 编码器架构: BERT

Bidirectional Encoder

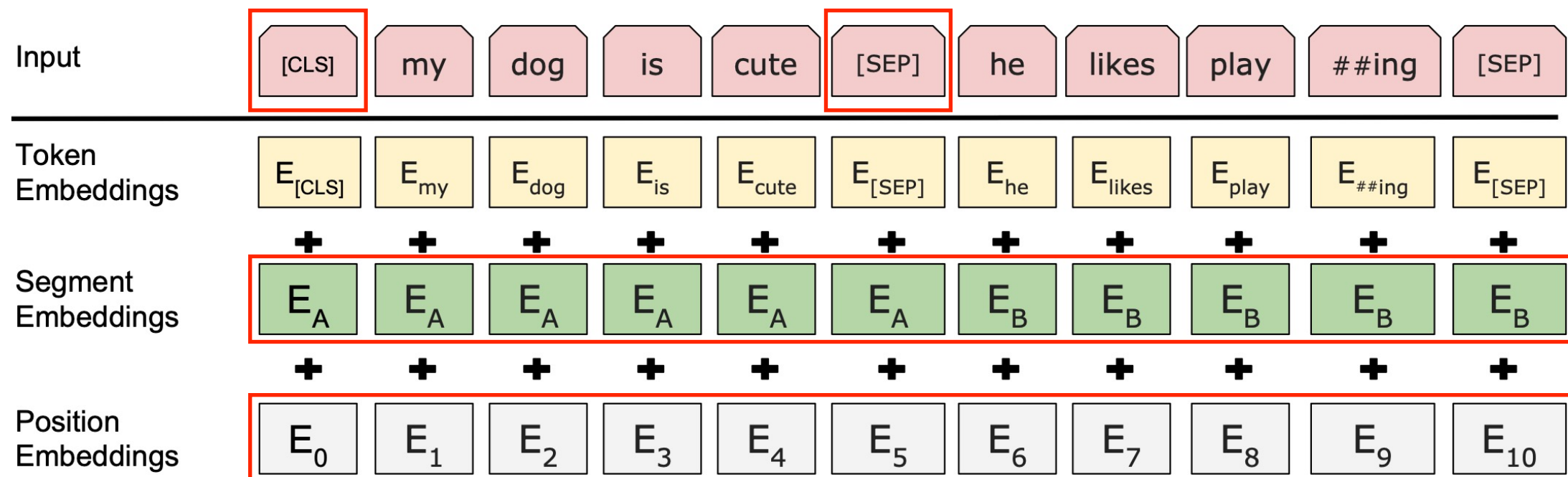
[Devlin et al., 2018]

Representations from Transformers

在每个输入序列开头添加特殊词元

特殊词元分隔两个句子

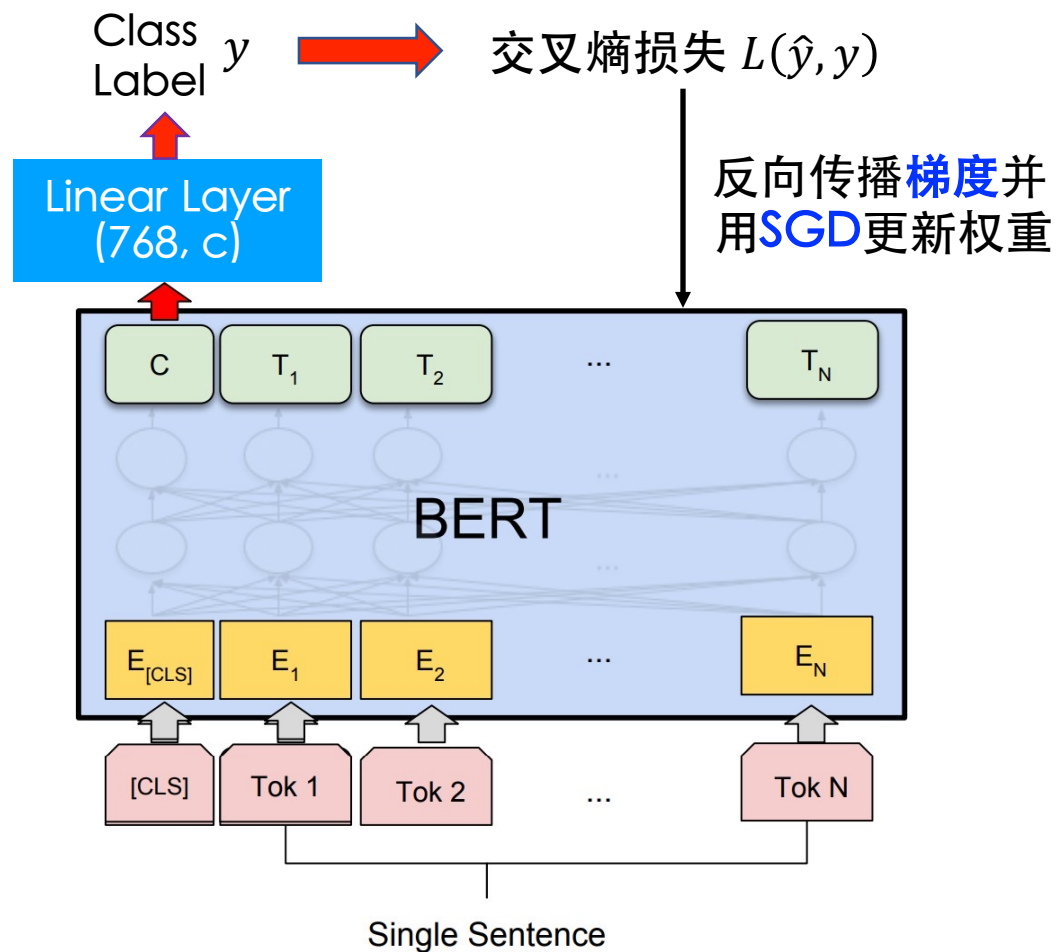
最后的词嵌入是这三者的总和！



学习每个词元的分隔嵌入，表明它是属于句子A还是句子B

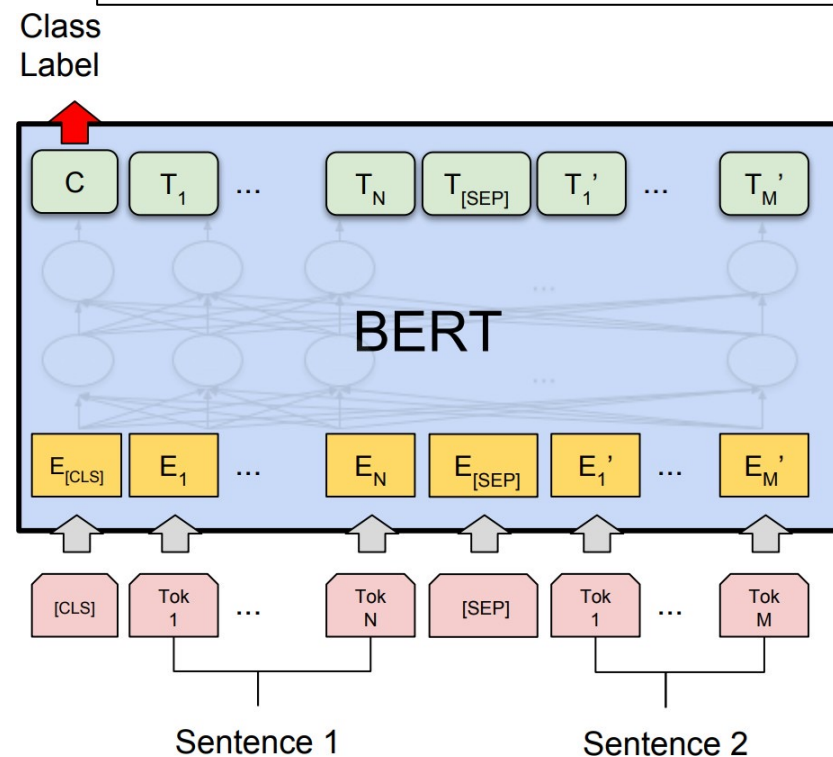
词元在整个序列中的位置

# 编码器架构: BERT (微调)



一个句子的任务, 如  
SST-2 (情感分析)

Input:  
Premise: A soccer game with multiple  
males playing  
Hypothesis: Some men are playing a sport  
Label: Entailment/Neutral/Contradiction



两个句子的任务, 如  
自然语言推理 (NLI)

# 编码器架构: BERT

Bidirectional Encoder  
Representations from Transformers [Devlin et al., 2018]

- 在当时对各种NLP任务进行微调获得SOTA的性能！

| System                | MNLI-(m/mm)<br>392k | QQP<br>363k | QNLI<br>108k | SST-2<br>67k | CoLA<br>8.5k | STS-B<br>5.7k | MRPC<br>3.5k | RTE<br>2.5k | Average<br>- |
|-----------------------|---------------------|-------------|--------------|--------------|--------------|---------------|--------------|-------------|--------------|
| Pre-OpenAI SOTA       | 80.6/80.1           | 66.1        | 82.3         | 93.2         | 35.0         | 81.0          | 86.0         | 61.7        | 74.0         |
| BiLSTM+ELMo+Attn      | 76.4/76.1           | 64.8        | 79.8         | 90.4         | 36.0         | 73.3          | 84.9         | 56.8        | 71.0         |
| OpenAI GPT            | 82.1/81.4           | 70.3        | 87.4         | 91.3         | 45.4         | 80.0          | 82.3         | 56.0        | 75.1         |
| BERT <sub>BASE</sub>  | 84.6/83.4           | 71.2        | 90.5         | 93.5         | 52.1         | 85.8          | 88.9         | 66.4        | 79.6         |
| BERT <sub>LARGE</sub> | <b>86.7/85.9</b>    | <b>72.1</b> | <b>92.7</b>  | <b>94.9</b>  | <b>60.5</b>  | <b>86.5</b>   | <b>89.3</b>  | <b>70.1</b> | <b>82.1</b>  |

- **QQP**: Quora Question Pairs (detect paraphrase questions)
- **QNLI**: natural language inference over question answering data
- **SST-2**: sentiment analysis
- **CoLA**: corpus of linguistic acceptability (detect whether sentences are grammatical)
- **STS-B**: semantic textual similarity
- **MRPC**: Microsoft paraphrase corpus
- **RTE**: a small natural language inference corpus

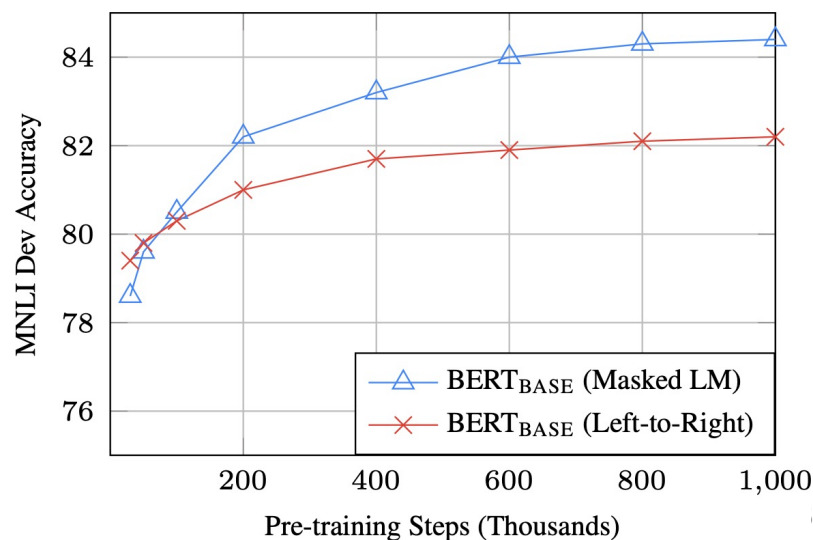


# 编码器架构: BERT

Bidirectional Encoder  
Representations from Transformers [Devlin et al., 2018]

SWAG  
(Commonsense  
inference task)

| System                             | Dev         | Test        |
|------------------------------------|-------------|-------------|
| ESIM+GloVe                         | 51.9        | 52.7        |
| ESIM+ELMo                          | 59.1        | 59.2        |
| OpenAI GPT                         | -           | 78.0        |
| BERT <sub>BASE</sub>               | 81.6        | -           |
| BERT <sub>LARGE</sub>              | <b>86.6</b> | <b>86.3</b> |
| Human (expert) <sup>†</sup>        | -           | 85.0        |
| Human (5 annotations) <sup>†</sup> | -           | 88.0        |



## • 两个模型

- **Base**: 110M, 4 Cloud TPUs, 4天
- **Large**: 340M, 16 Cloud TPUs, 4 天
- 两个模型均可在单卡上进行微调
- 模型越大效果越好

- 掩码语言模型开始性能比Left-to-Right效果差，但最终优于Left-to-Right

# 编码器架构: RoBERTa

- 原来的BERT明显训练不足！
- 更多数据 (16G  $\Rightarrow$  160G)
- 更长的预训练时间
- 更大的批次
- 移除下一句预测(NSP)目标
- 长序列训练
- 动态掩码，随机遮掩不同的词元
- 包含50K个子词单位的更大的字节级BPE词汇表



效果比BERT  
好多了

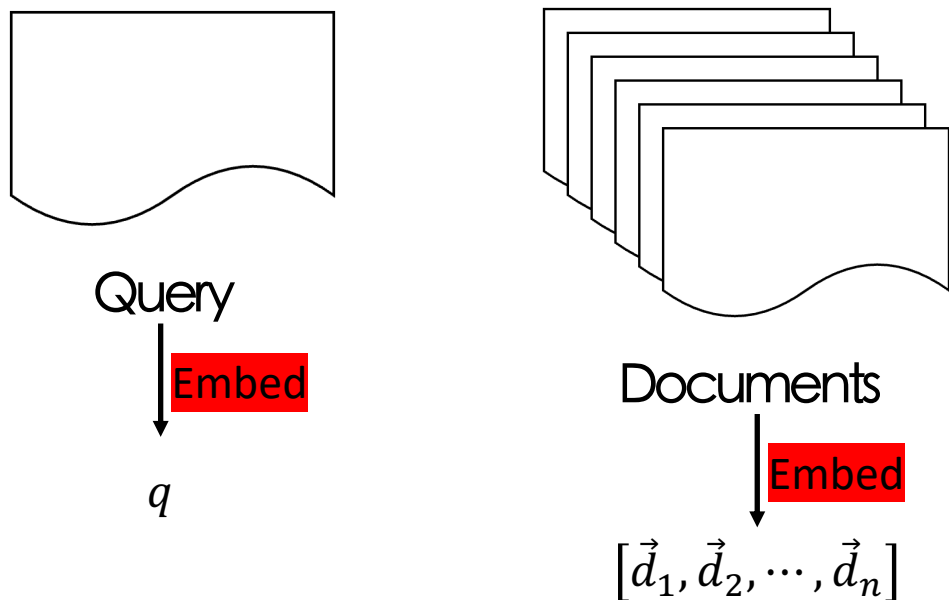
# 编码器应用于信息检索

## Encoder for Information Retrieval

给定问题/Query，查询相关  
文档集

应用：

- 搜索引擎(谷歌的工作原理！)
- 检索增强语言模型/Retrieval Augmented Language Models



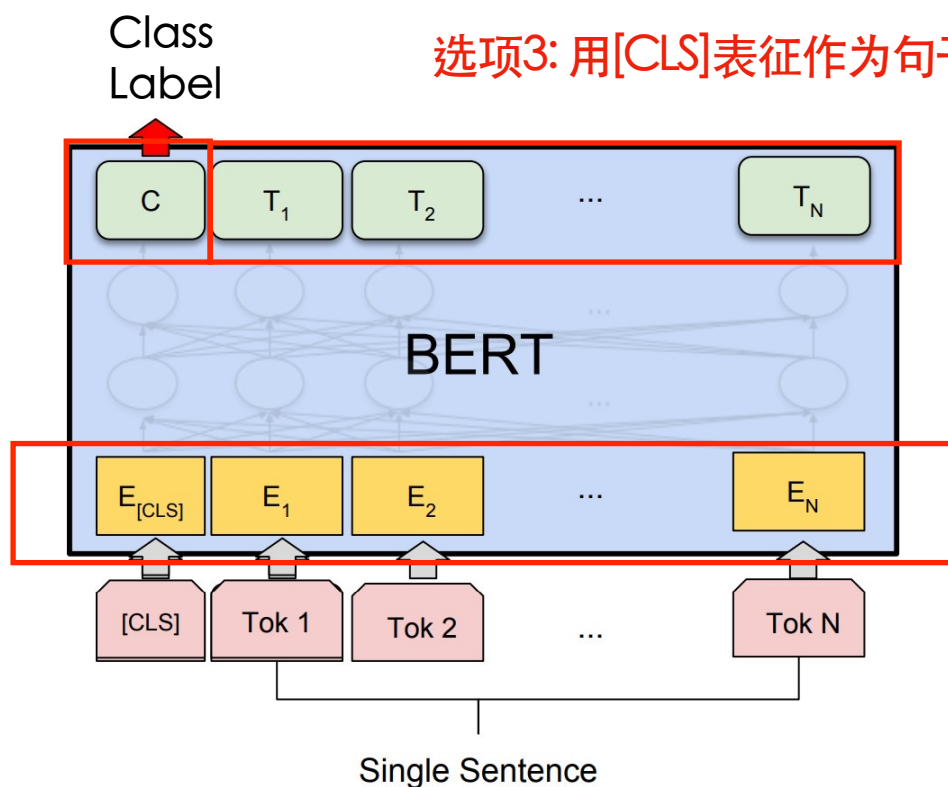
对文档相关性进行评分，如，计算问题和  
文档之间的余弦相似性

**relevance – score** $(d|q) = \cos(q, d)$

# 编码器应用于信息检索

## Encoder for Information Retrieval

- 如何从类似BERT这样基于编码器的模型中得到句子表征？



选项2: 平均所有上下文的词嵌入

选项1: 平均所有词嵌入

存在什么问题？

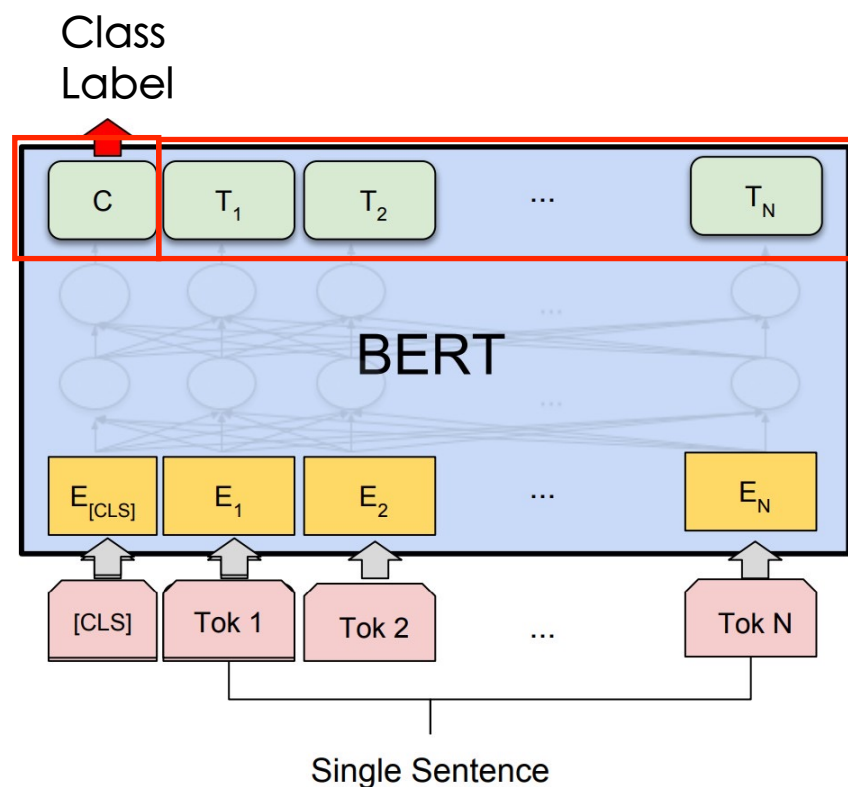
缺乏上下文，  
效果类似GloVE

# 编码器应用于信息检索

## Encoder for Information Retrieval

- 开箱即用: 甚至上下文表征也适合检索!

性能比词嵌入还差. 为什么?



选项2  
选项3

与word2vec不同, 预训练的目标不是把相似句子的嵌入空间映射的更近

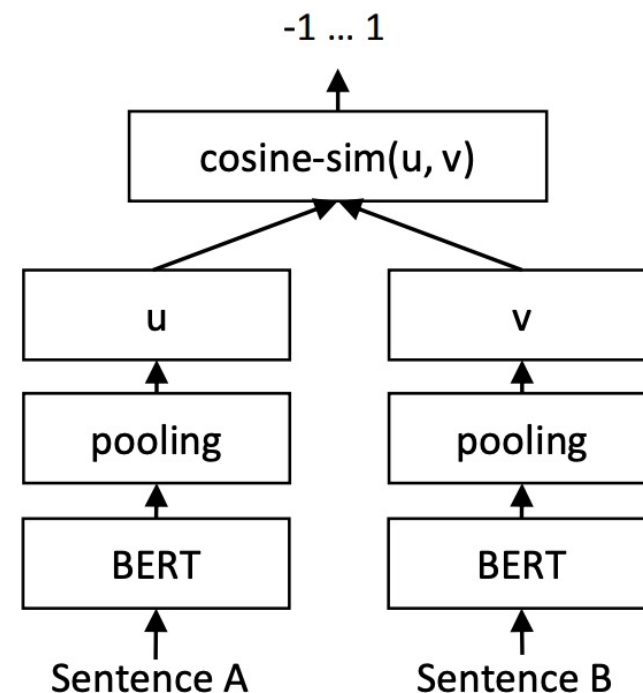
| Model                      | STS12 | STS13 | STS14 | STS15 | STS16 | STSb  | SICK-R       | Avg.  |
|----------------------------|-------|-------|-------|-------|-------|-------|--------------|-------|
| Avg. GloVe embeddings      | 55.14 | 70.66 | 59.73 | 68.25 | 63.66 | 58.02 | 53.76        | 61.32 |
| Avg. BERT embeddings       | 38.78 | 57.98 | 57.98 | 63.15 | 61.06 | 46.35 | 58.40        | 54.81 |
| BERT CLS-vector            | 20.16 | 30.01 | 20.09 | 36.88 | 38.08 | 16.50 | 42.63        | 29.19 |
| InferSent - Glove          | 52.86 | 66.75 | 62.15 | 72.77 | 66.87 | 68.03 | 65.65        | 65.01 |
| Universal Sentence Encoder | 64.49 | 67.80 | 64.61 | 76.83 | 73.18 | 74.92 | <b>76.69</b> | 71.22 |

Spearman correlations for Textual Similarity (STS) tasks (higher is better)

# 编码器应用于信息检索: Sentence BERT (S-BERT)

[N.Reimers & I. Gurevych 2019]

- 微调BERT/RoBERTa学习句子级的表征，使得相似的句子在嵌入空间中位置更近
- 采用三元组目标函数/triplet objective function:
  - 给定一个锚定/anchor句子、一个正样本句子和一个负样本句子，用三元组损失函数对网络进行调优，使锚定句子与正样本句子间的距离小于锚定句子与负样本句子间的距离



Triplet objective function  
 $\max(\|s_a - s_p\| - \|s_a - s_n\| + \epsilon, 0)$

# 编码器应用于信息检索: Sentence BERT (S-BERT)

[N.Reimers & I. Gurevych 2019]

| Model                      | STS12        | STS13        | STS14        | STS15        | STS16        | STSb         | SICK-R       | Avg.         |
|----------------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| Avg. GloVe embeddings      | 55.14        | 70.66        | 59.73        | 68.25        | 63.66        | 58.02        | 53.76        | 61.32        |
| Avg. BERT embeddings       | 38.78        | 57.98        | 57.98        | 63.15        | 61.06        | 46.35        | 58.40        | 54.81        |
| BERT CLS-vector            | 20.16        | 30.01        | 20.09        | 36.88        | 38.08        | 16.50        | 42.63        | 29.19        |
| InferSent - Glove          | 52.86        | 66.75        | 62.15        | 72.77        | 66.87        | 68.03        | 65.65        | 65.01        |
| Universal Sentence Encoder | 64.49        | 67.80        | 64.61        | 76.83        | 73.18        | 74.92        | <b>76.69</b> | 71.22        |
| SBERT-NLI-base             | 70.97        | 76.53        | 73.19        | 79.09        | 74.30        | 77.03        | 72.91        | 74.89        |
| SBERT-NLI-large            | 72.27        | <b>78.46</b> | <b>74.90</b> | 80.99        | 76.25        | <b>79.23</b> | 73.75        | 76.55        |
| SRoBERTa-NLI-base          | 71.54        | 72.49        | 70.80        | 78.74        | 73.69        | 77.77        | 74.46        | 74.21        |
| SRoBERTa-NLI-large         | <b>74.53</b> | 77.00        | 73.18        | <b>81.85</b> | <b>76.82</b> | 79.10        | 74.29        | <b>76.68</b> |

Spearman correlations for Textual Similarity (STS) tasks (higher is better)

- SBERT/SRoBERTa性能显著优于现有方法！



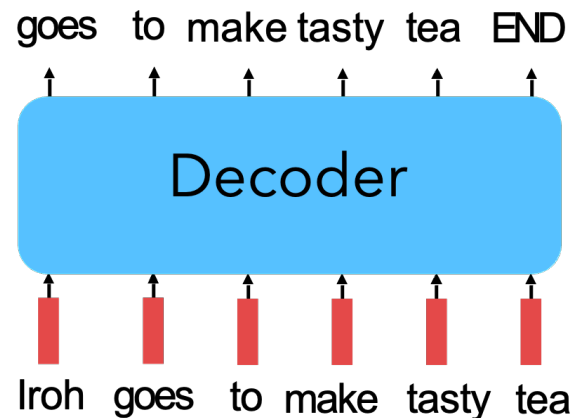
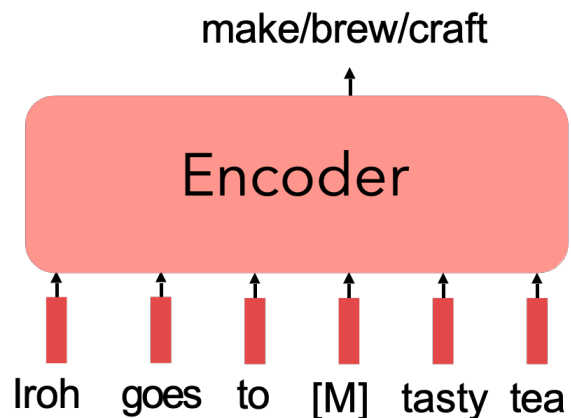
# 编码器架构: 优劣势



- 同时考虑左右上下文
- 捕捉复杂的上下文关系



- 不擅长从左到右生成开放文本(一次一个词元)





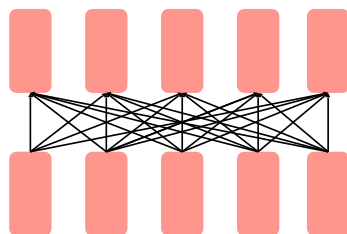
# 大纲

- 预训练(自监督学习)的动机
- 预训练的架构和训练目标
  - 编码器架构
  - 编码器-解码器架构
  - 解码器架构
- 附录
  - Transformer
  - 开放式文本生成

# 三种预训练模式/架构

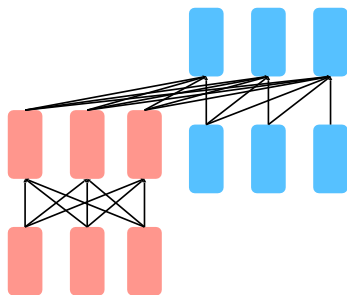
## 3 Pre-training Paradigms/Architectures

Encoder  
/编码器



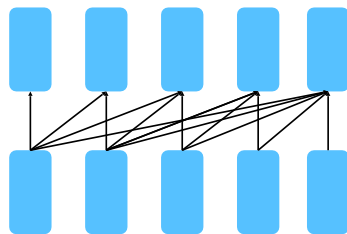
- **双向的/Bidirectional**: 可以基于未来上下文/context为条件生成

Encoder — Decoder  
/编码器 — /解码器



- 将两个不同长度的**序列**映射在一起

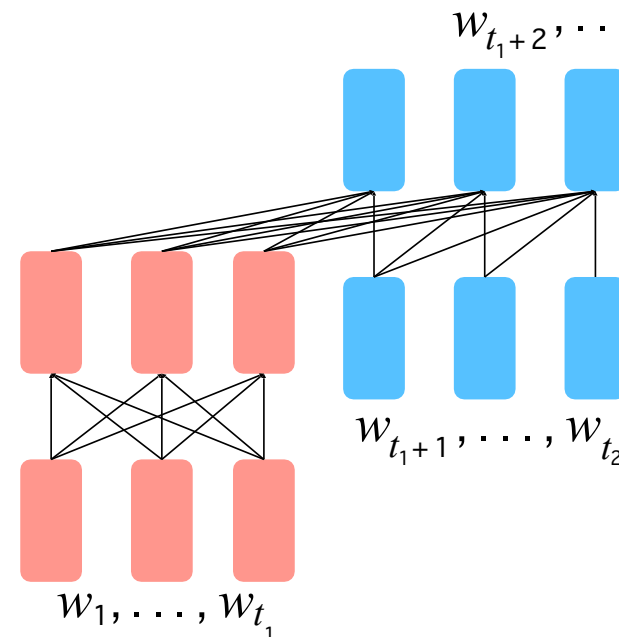
Decoder  
/解码器



- **语言建模/Language modelling**: 仅能根据过去的上下文/context为条件生成

# 编码器-解码器架构

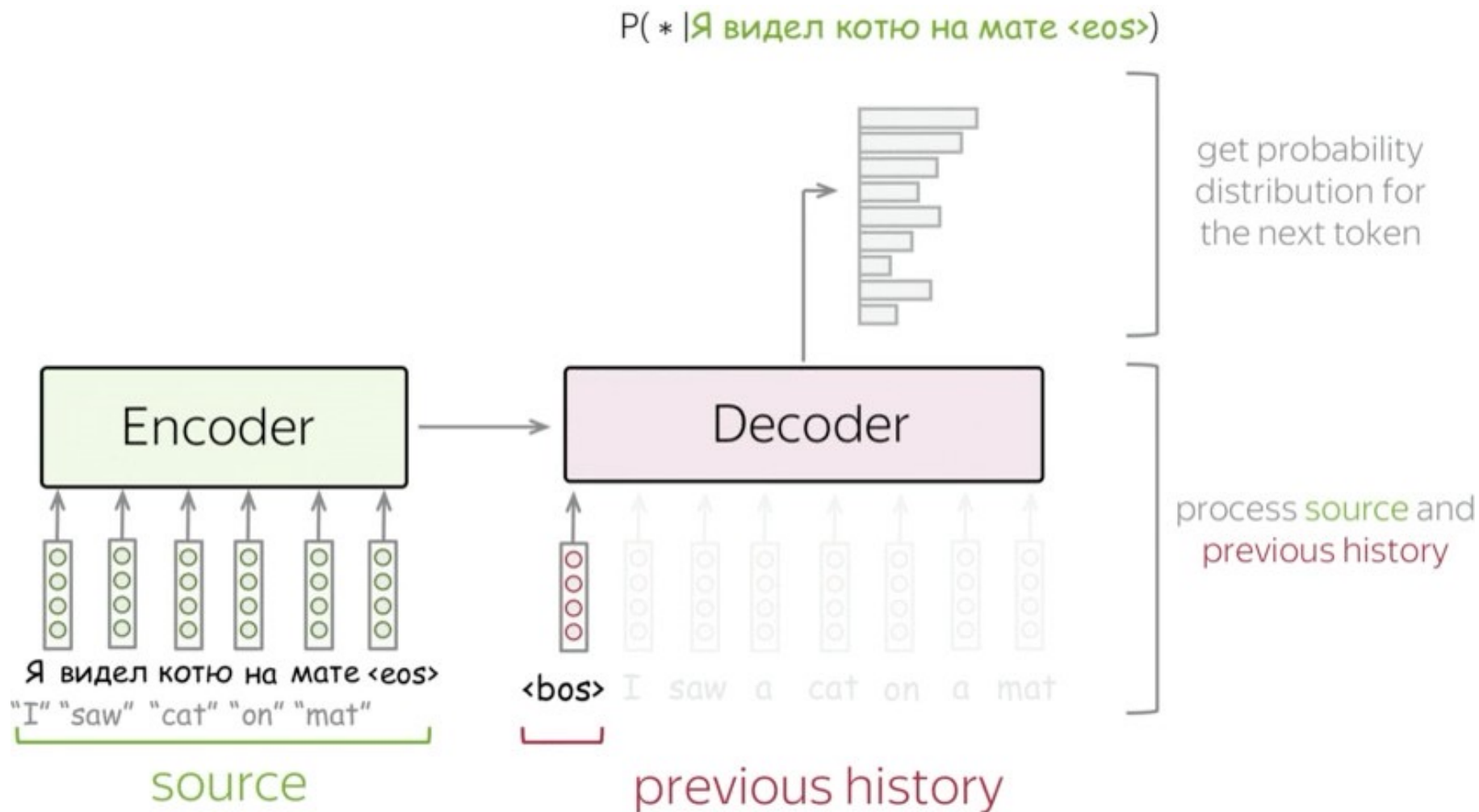
- 迈向**开放文本生成/Open-text Generation** .....
- **编码器**构建源文本的表征并将其传递至**解码器**
- **解码器**利用该源文本表征生成目标语句
- **编码器**得益于**双向**上下文信息; **解码器**则通过**语言模型**任务进行模型训练



$$\begin{aligned}h_1, \dots, h_{t_1} &= \text{Encoder}(w_1, \dots, w_{t_1}) \\h_{t_1+1}, \dots, h_{t_2} &= \text{Decoder}(w_{t_1+1}, \dots, w_{t_2}, h_1, \dots, h_{t_1}) \\y_i &\sim Ah_i + b, i > t\end{aligned}$$

[[Raffel et al., 2018](#)]

# 编码器-解码器架构: 一个机器翻译的例子



[\[Lena Viota Blog\]](#)

# 编码器-解码器架构: 训练目标

- 我们能否在此处应用语言建模？
- **可以这样操作**: 给定一段文本，随机选取一个分割点，将其拆分为前缀部分与目标部分
  - 编码器将前缀作为输入，解码器则基于该前缀进行训练，以生成对应的目标文本

Original text

Thank you for inviting me to your party last week.

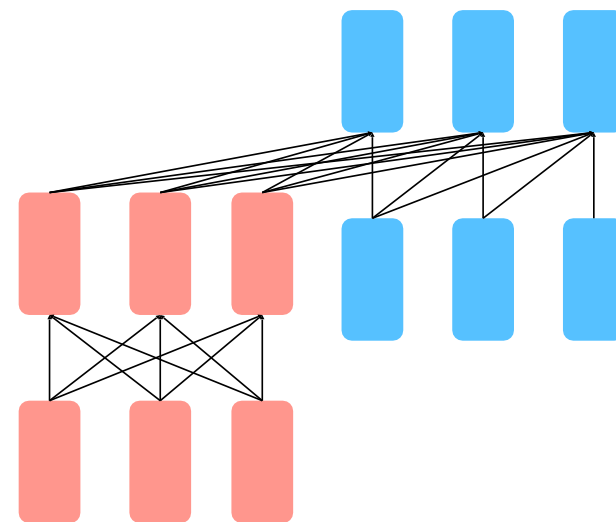
↑  
如: 在此切分

Inputs

Thank you for inviting me

Targets

to your party last week

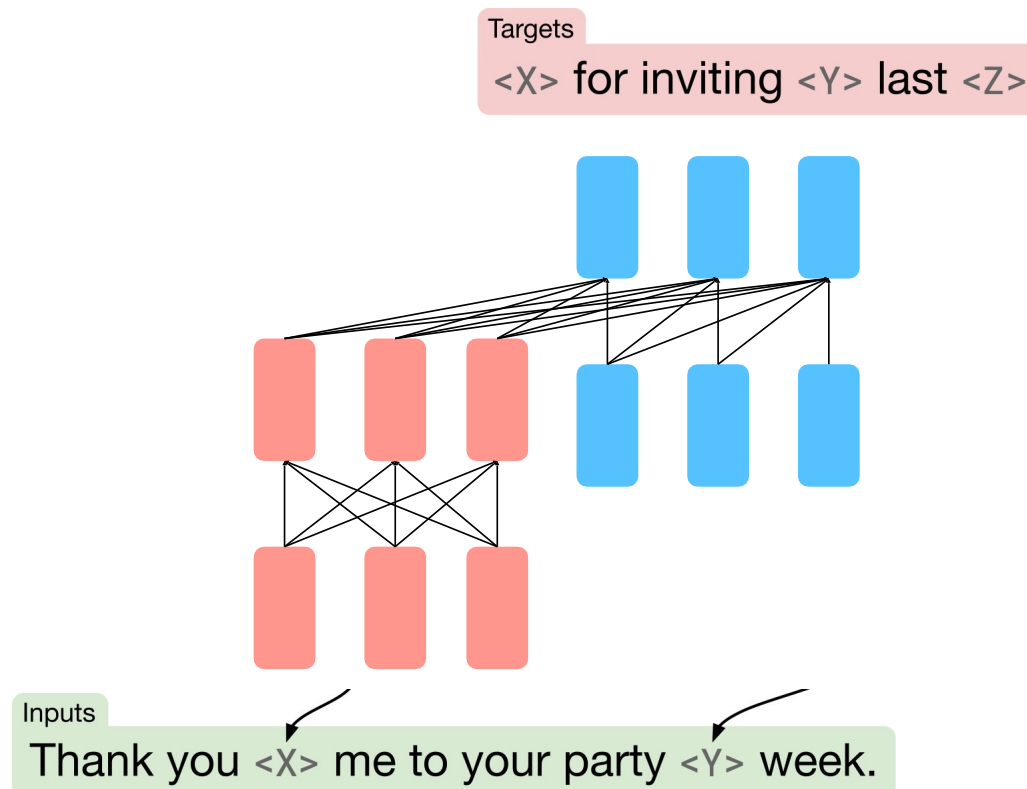


# 编码器-解码器架构: 训练目标

- **T5 [Raffel et al., 2018]**
- **文本片段掩码(去噪)**: 将输入文本中不同长度的片段替换为特定占位符/placeholders (如 <extra\_id\_0>); 解码器需还原出被掩码的文本片段/masked spans
  - 该操作在**文本预处理阶段**完成: 模型训练时, 解码器端采用**语言模型**作为优化目标

Original text

Thank you ~~for inviting~~ me to your party ~~last~~ week.



# 编码器-解码器架构: T5

Text to Text Transfer  
Transformer

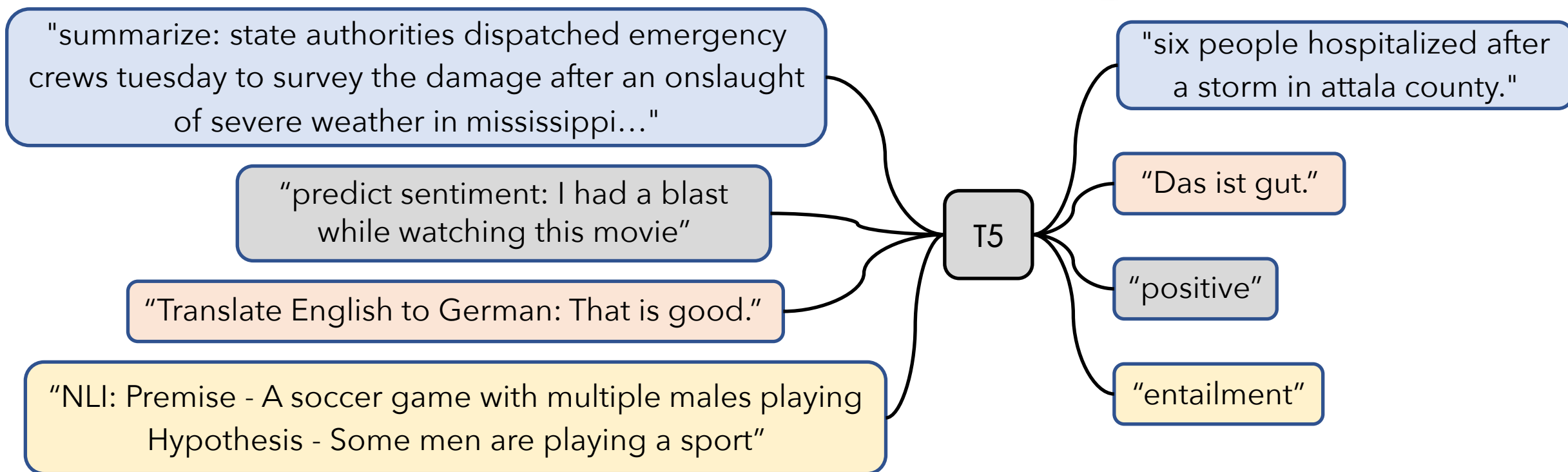
| Architecture      | Objective | Params | Cost  | GLUE         | CNNDM        | SQuAD        | SGLUE        | EnDe         | EnFr         | EnRo         |
|-------------------|-----------|--------|-------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| ★ Encoder-decoder | Denoising | $2P$   | $M$   | <b>83.28</b> | <b>19.24</b> | <b>80.88</b> | <b>71.36</b> | <b>26.98</b> | <b>39.82</b> | <b>27.65</b> |
| Enc-dec, shared   | Denoising | $P$    | $M$   | 82.81        | 18.78        | <b>80.63</b> | <b>70.73</b> | 26.72        | 39.03        | <b>27.46</b> |
| Enc-dec, 6 layers | Denoising | $P$    | $M/2$ | 80.88        | 18.97        | 77.59        | 68.42        | 26.38        | 38.40        | 26.95        |
| Language model    | Denoising | $P$    | $M$   | 74.70        | 17.93        | 61.14        | 55.02        | 25.09        | 35.28        | 25.86        |
| Prefix LM         | Denoising | $P$    | $M$   | 81.82        | 18.61        | 78.94        | 68.11        | 26.43        | 37.98        | 27.39        |
| Encoder-decoder   | LM        | $2P$   | $M$   | 79.56        | 18.59        | 76.02        | 64.29        | 26.27        | 39.17        | 26.86        |
| Enc-dec, shared   | LM        | $P$    | $M$   | 79.60        | 18.13        | 76.35        | 63.50        | 26.62        | 39.17        | 27.05        |
| Enc-dec, 6 layers | LM        | $P$    | $M/2$ | 78.67        | 18.26        | 75.32        | 64.06        | 26.13        | 38.42        | 26.89        |
| Language model    | LM        | $P$    | $M$   | 73.78        | 17.54        | 53.81        | 56.51        | 25.23        | 34.31        | 25.38        |
| Prefix LM         | LM        | $P$    | $M$   | 79.68        | 17.84        | 76.87        | 64.86        | 26.28        | 37.51        | 26.76        |

- 文本片段掩码(去噪)比语言模型效果更好！
- 编码器-解码器架构比解码器架构效果更好

[[Raffel et al., 2018](#)]

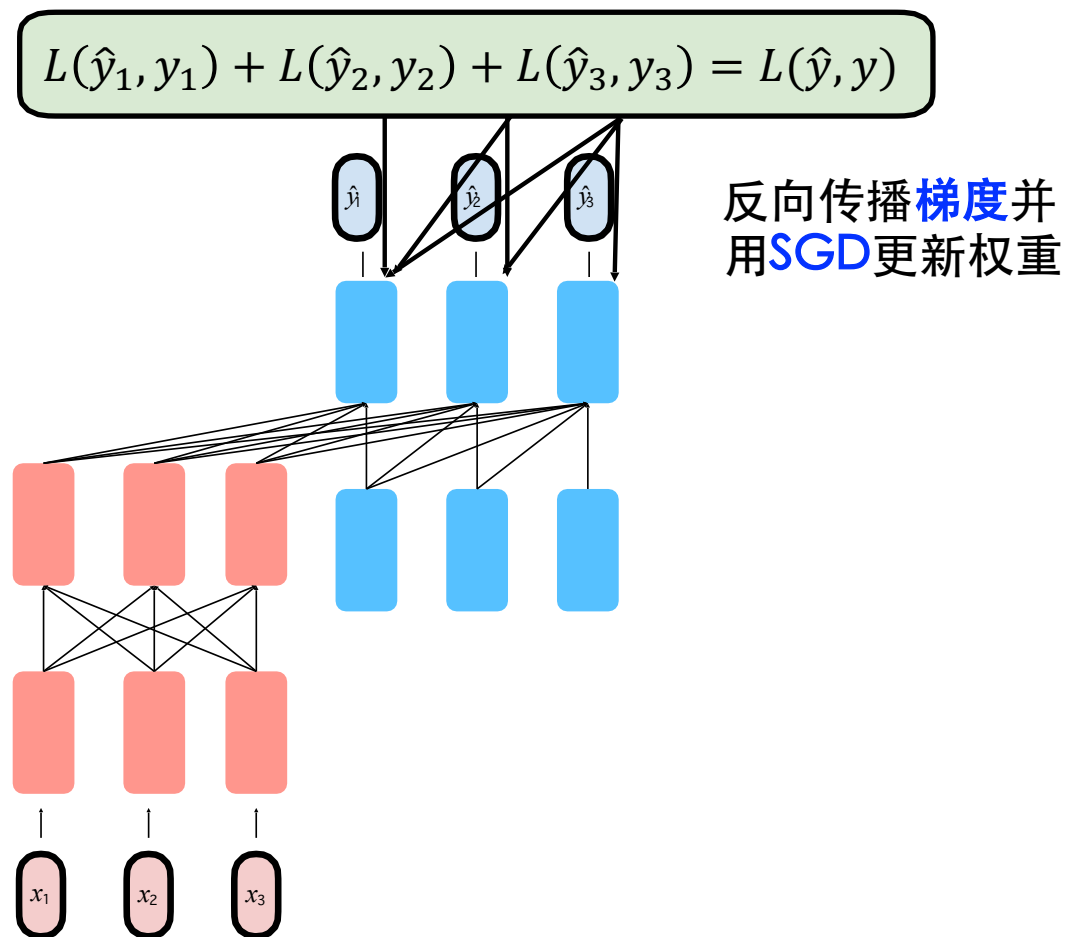
# 编码器-解码器架构: T5 (微调)

**核心思想:** 将当前的任意自然语言处理任务，都转化为基于给定输入文本的**文本生成**问题！





# 编码器-解码器架构: T5 (微调)



# 编码器-解码器架构: T5

- Text-to-Text: 将各类自然语言处理任务转化为输入/输出文本序列的形式
- 数据集: Colossal Clean Crawled Corpus (C4), 750G文本数据!
- 多种规模的模型版本:
  - Base (222M)
  - Small (60M)
  - Large (770M)
  - 3B
  - 11B
- 通过扩充和净化数据，该模型达到SOTA



[\[Google Blog\]](#)

- 其他模型: [BART](#)

# 编码器-解码器架构: 优劣势



- 一种兼顾**双向**上下文信息利用与**开放式文本**生成的理想折中方案
- 适用于**多任务**微调



- 需要投入更多**文本预处理/text wrangling**工作
- **训练难度更高**
- 在自然语言生成任务中的**灵活性相对不足**

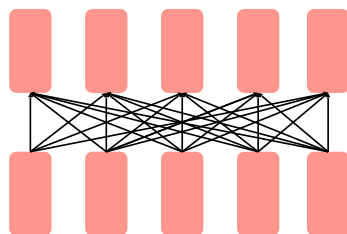
# 大纲

- 预训练(自监督学习)的动机
- 预训练的架构和训练目标
  - 编码器架构
  - 编码器-解码器架构
  - 解码器架构
- 附录
  - Transformer
  - 开放式文本生成

# 三种预训练模式/架构

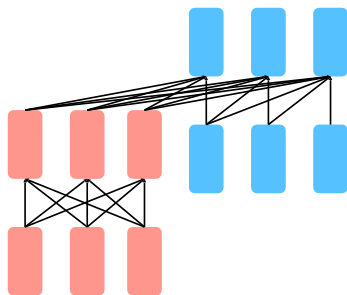
## 3 Pre-training Paradigms/Architectures

Encoder  
/编码器



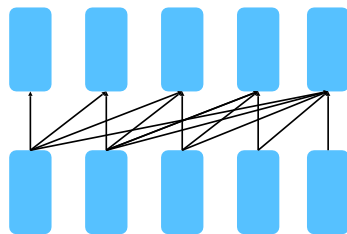
- 双向的/Bidirectional: 可以基于未来上下文/context为条件生成

Encoder — Decoder  
/编码器 — /解码器



- 将两个不同长度的序列映射在一起

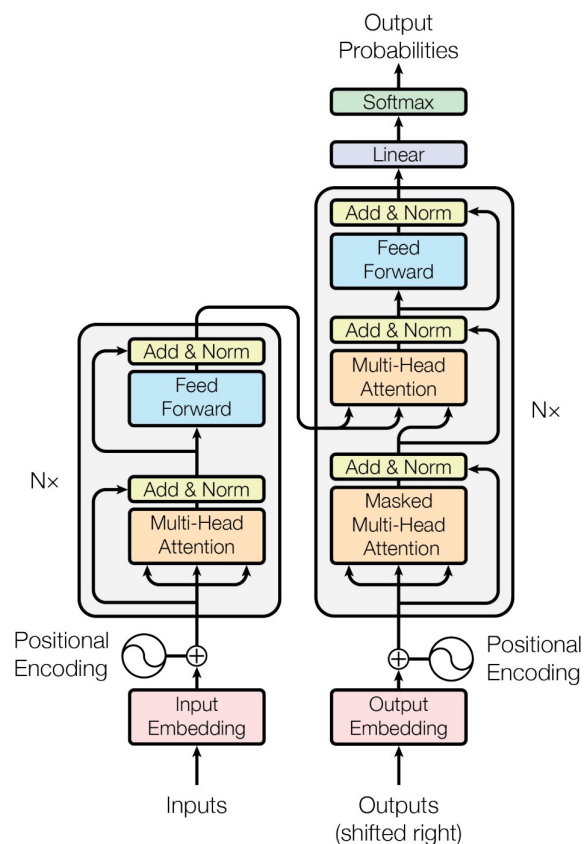
Decoder  
/解码器



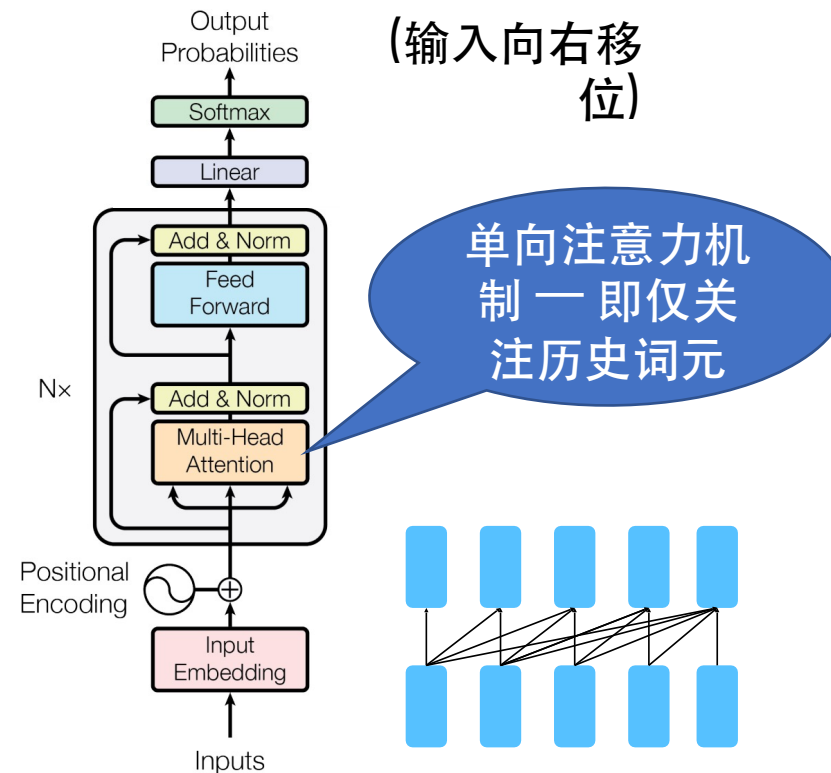
- 语言建模/Language modelling: 仅能根据过去的上下文/context为条件生成

# 解码器架构 Decoder Architecture

Full-Transformer Architecture  
(Encoder-Decoder)

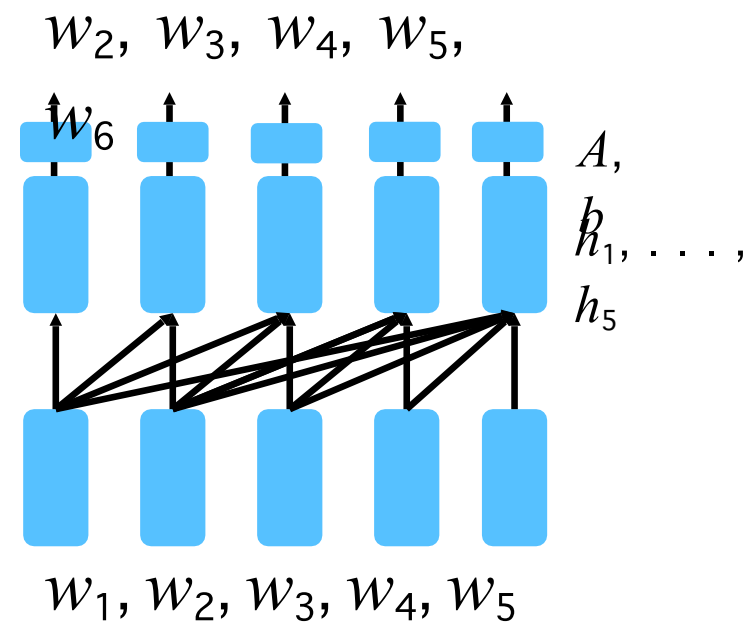


Decoder-Only Transformer  
Architecture



# 解码器架构: 训练目标

- 许多知名的生成式大语言模型均为**纯解码器架构**
  - 如: GPT-1/2/3/4、Llama-1/2系列等
- 以**语言建模**为核心训练目标，天然适用于**开放式文本生成**任务
- **条件语言建模**:  $p(w_t | w_1, \dots, w_{t-1}, x)$ 
  - 给定源上下文信息，模型**从左到右**的自回归方式生成文本
- 该类模型可针对各类**自然语言生成**任务进行微调
  - 如: 对话生成、文本摘要等



# 解码器架构: GPT

Improving Language Understanding  
by **G**enerative **P**re-**T**raining

[[Radford et al., 2018](#)]

- 2018 年推出的 GPT 在解码器预训练领域取得了重大突破！
- 该模型采用 Transformer 解码器架构，包含 12 层，共 1.17 亿个参数
- 其训练数据集为 **BooksCorpus**，**涵盖超过 7000 本不同的书**
  - 该语料库包含大量连续的长文本片段，可助力模型学习长距离依赖关系
- 值得一提的是，缩写词“GPT”并未出现在其原始论文中
  - 它可以指代“**生成式预训练 (Generative PreTraining)**”或“**生成式预训练Transformer (Generative Pretrained Transformer)**”



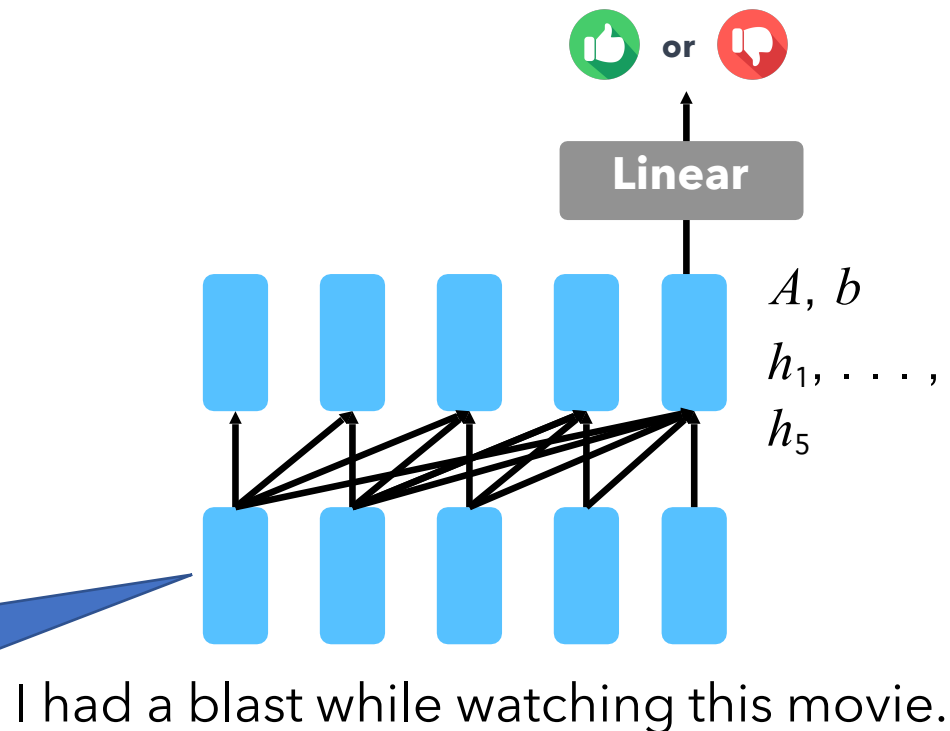
# 解码器架构: GPT (微调)

[Radford et al., 2018]

- 针对下游任务对预训练模型进行定制:

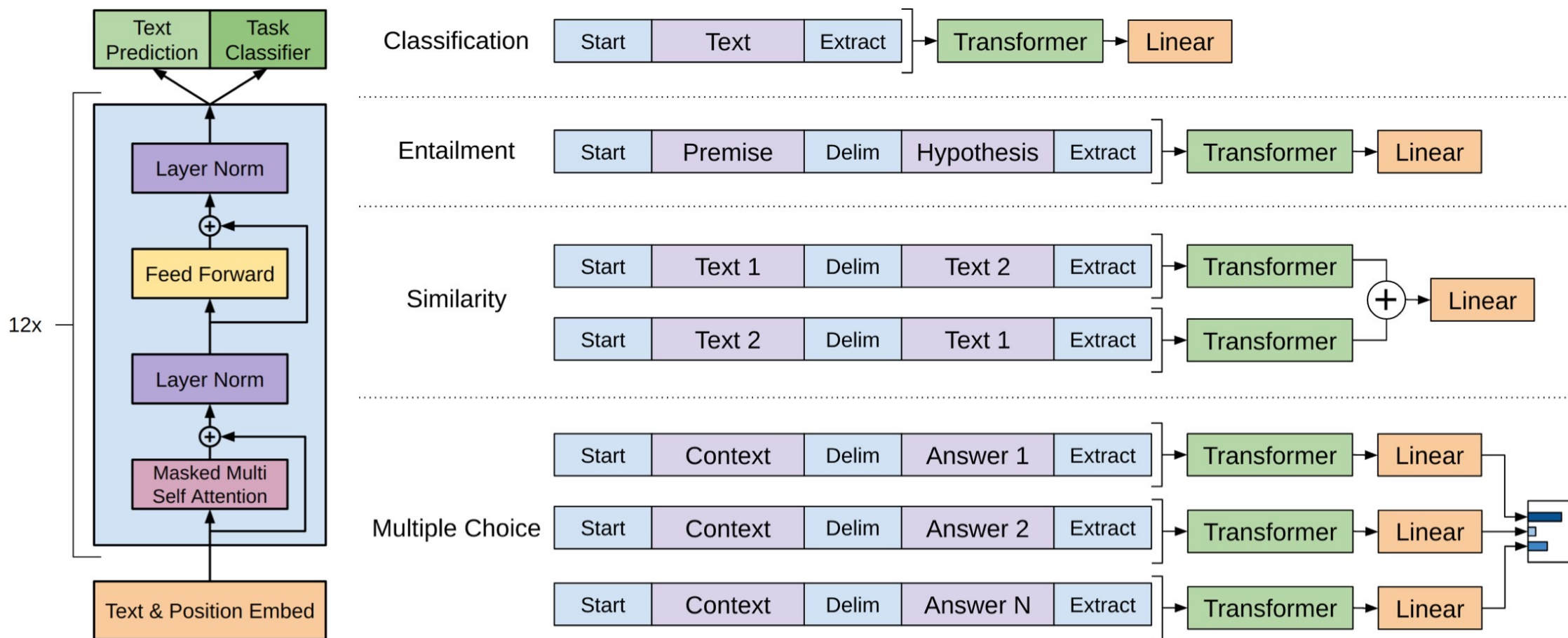
- 在模型的最后一个隐藏层之上增设一个**线性层**，使其成为分类器！
- 微调阶段，不仅对**随机初始化的线性层**进行训练，同时对网络进行**全参数训练**

尽管最初的设计并非如此，但我们可在此处采用 T5 风格的文本到文本范式微调，以适配各类任务  
事实上，这种方式如今已成为行业通用标准！



# 解码器架构: GPT (微调)

[[Radford et al., 2018](#)]



# 解码器架构: GPT-2

Language Models are Unsupervised  
Multitask Learners [\[Radford et al., 2019\]](#)

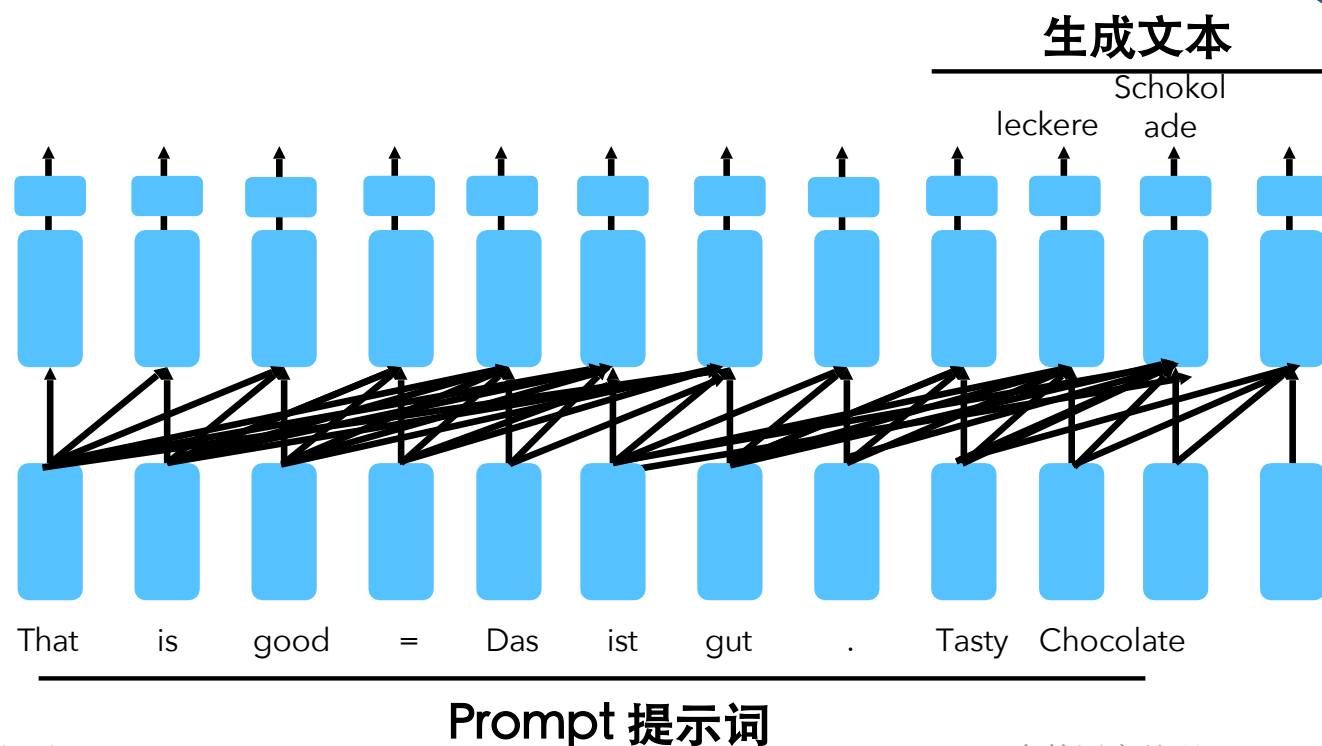
- GPT 的规模化升级版本: 最大的 GPT-2 模型拥有**15.6 亿参数**与 **48 个网络层**
- 该模型基于体量更大的数据集完成训练
  - **WebText 语料库**: 其文本均经过筛选以保证高质量
    - 语料来源为 Reddit 平台上获得至少 3 次点赞的外链网页爬取数据
    - 从4500万个链接筛选至 800万份文档，最终形成 40GB 的文本数据量

# 解码器架构: GPT-2

Language Models are **Unsupervised Multitask Learners**

[[Radford et al., 2019](#)]

- GPT-2 最令人称道的一点在于，它能够在诸多自然语言处理数据集上实现出色的**零样本学习/Zero-shot**性能！

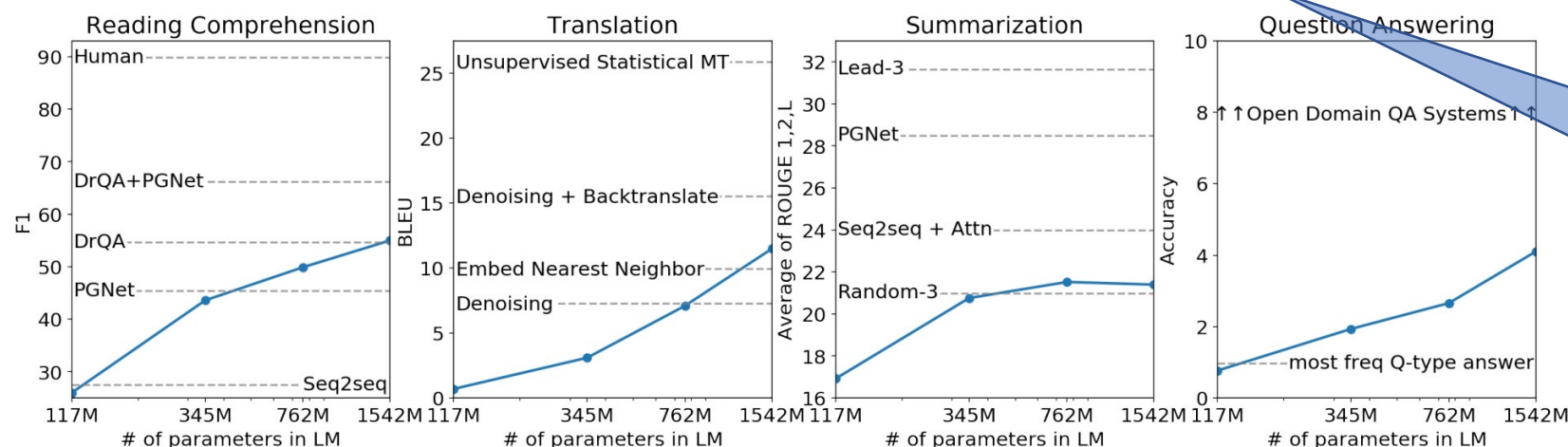


# 解码器架构: GPT-2

Language Models are **Unsupervised Multitask Learners**

[[Radford et al., 2019](#)]

- GPT-2 最令人称道的一点在于，它能够在诸多自然语言处理数据集上实现出色的**零样本学习/Zero-shot**性能！



开启“提示词时代” (范式4)

Figure 1. Zero-shot task performance of WebText LMs as a function of model size on many NLP tasks. Reading Comprehension results are on CoQA (Reddy et al., 2018), translation on WMT-14 Fr-En (Artetxe et al., 2017), summarization on CNN and Daily Mail (See et al., 2017), and Question Answering on Natural Questions (Kwiatkowski et al., 2019). Section 3 contains detailed descriptions of each result.

**GPT-2 的论文中甚至完全没有做任何微调实验！**

# 解码器架构: GPT-2 (文本生成) Text Generation

[[Radford et al., 2019](#)]

- 实验还表明，该模型能够生成极为逼真的自然语言文本样本

**Context (human-written):** In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

**GPT-2:** The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science.

Now, after almost two centuries, the mystery of what sparked this odd phenomenon is finally solved.

Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans. Pérez noticed that the valley had what appeared to be a natural fountain, surrounded by two peaks of rock and silver snow.

Pérez and the others then ventured further into the valley. "By the time we reached the top of one peak, the water looked blue, with some crystals on top," said Pérez.

Pérez and his friends were astonished to see the unicorn herd. These creatures could be seen from the air without having to move too much to see them – they were so close they could touch their horns.

GPT-2 发布之时，OpenAI 并未公开其15 亿参数版本，目的是防止该模型被大规模用于生成具有欺骗性、偏见性或侮辱性的文本内容

February 14, 2019

## Better language models and their implications

[Read paper ↗](#)

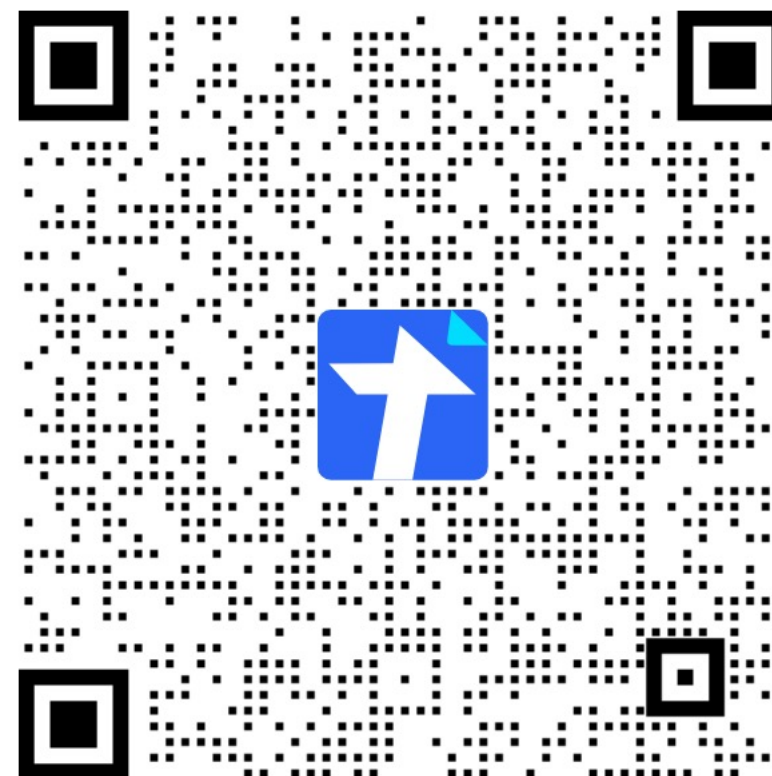
[View code ↗](#)

# 给定一个任务， 如何选择合适的架构？

- 目前，**纯解码器架构**模型似乎主导了该领域
  - 如: GPT-1/2/3/4、Mistral、Llama-1/2/3/3.1、Gemini、Claude 等
  - 此类模型是**文本生成**任务的最佳选择
- 若你需要用于自然语言理解类任务的轻量级模型，或需要生成用于检索的句子嵌入，编码器架构模型(如BERT)的表现更为出色
- T5(序列到序列架构)则在多任务场景下表现优异
  - 有研究表明，在自然语言理解任务上，T5 的性能优于纯解码器模型[Tay et al. 2022. [UL2](#)]
- 当然，选择最优的模型架构，至今仍是一个开放的研究问题



# 一句话总结



- 课外阅读
  - [SLP3] Ch. 9-12



# 附录

1. Transformer
2. 开放式文本生成 Open Ended Text Generation

# 长距离依赖

## Long Range Dependencies



I'm want to watch Wicked! How does the weather in NYC look next week?

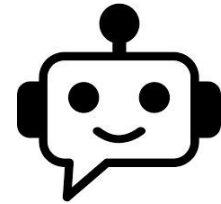
It looks sunny with some light rain during the weekend.

Oh! But I don't have a rain jacket 😞 Is there a store nearby?

There's a marshall's a mile away. They have the navy blue jacket you have been eyeing for a while!

⋮

Ok! Looks like I can actually go! Book the tickets for next Wed!



# 长距离依赖

## Long Range Dependencies



I'm want to watch **Wicked**! How does the weather in NYC look next week?

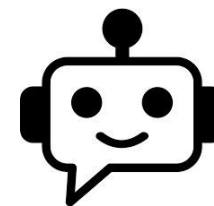
It looks sunny with some light rain during the weekend.

Oh! But I don't have a rain jacket 😞 Is there a store nearby?

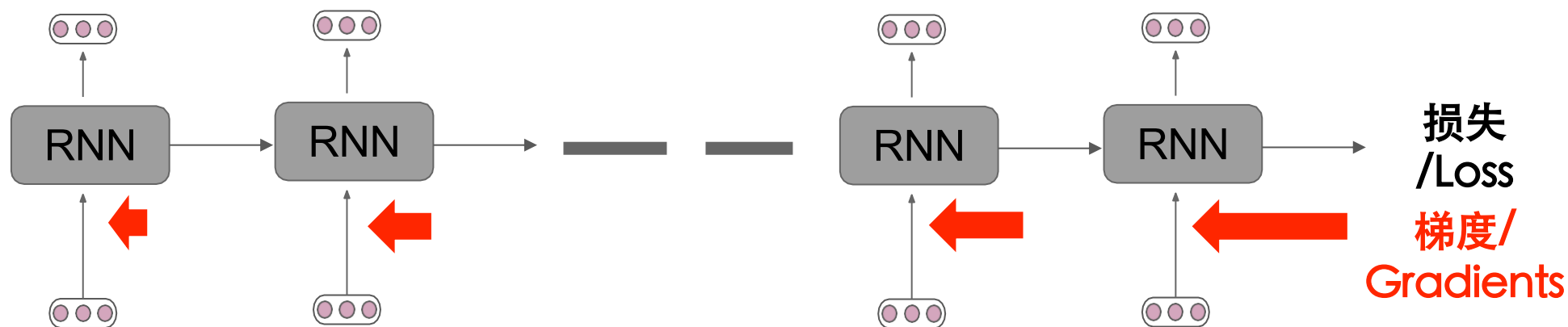
There's a marshall's a mile away. They have the navy blue jacket you have been eyeing for a while!

⋮

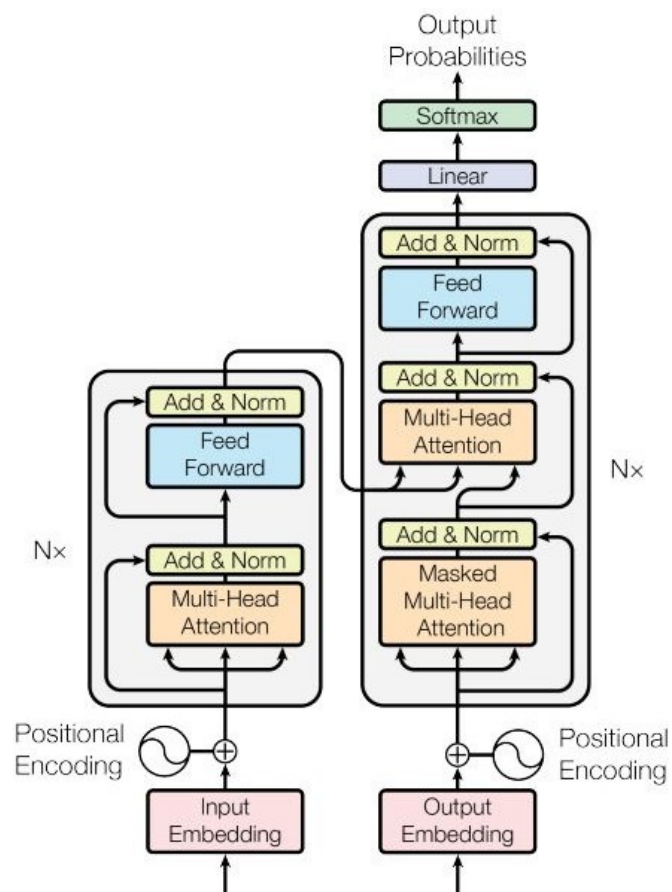
Ok! Looks like I can actually go! Book the **tickets** for next Wed!



# 循环神经网络问题：梯度消失/爆炸



# Transformer



Google Research

[Attention is all you need](#) (Vaswani et.al, 2017)

# Transformer作者去向

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

## Attention Is All You Need

**Ashish Vaswani\***  
Google Brain  
avaswani@google.com

**Noam Shazeer\***  
Google Brain  
noam@google.com

**Niki Parmar\***  
Google Research  
nikip@google.com

**Jakob Uszkoreit\***  
Google Research  
usz@google.com

**Llion Jones\***  
Google Research  
llion@google.com

**Aidan N. Gomez\* †**  
University of Toronto  
aidan@cs.toronto.edu

**Lukasz Kaiser\***  
Google Brain  
lukaszkaizer@google.com

**Illia Polosukhin\* ‡**  
illia.polosukhin@gmail.com



### Google's \$4 Billion 'Transformer' Club

Nearly all eight authors of a seminal AI research paper have left to start their own companies



#### Ashish Vaswani

COMPANY Essential.ai  
ROLE Co-founder  
RAISED \$8 million  
VALUATION Unknown



#### Noam Shazeer

COMPANY Character.ai  
ROLE Co-founder  
RAISED \$193 million  
VALUATION \$850 million



#### Niki Parmar

COMPANY Essential.ai  
ROLE Co-founder  
RAISED \$8 million  
VALUATION Unknown



#### Jakob Uszkoreit

COMPANY Inceptive Nucleics  
ROLE Co-founder  
RAISED \$20 million  
VALUATION \$90 million



#### Llion Jones

COMPANY Stealth startup  
ROLE Co-founder  
RAISED NA  
VALUATION NA



#### Aidan N. Gomez

COMPANY Cohere  
ROLE Co-founder and CEO  
RAISED \$440 million  
VALUATION \$2 billion



#### Lukasz Kaiser

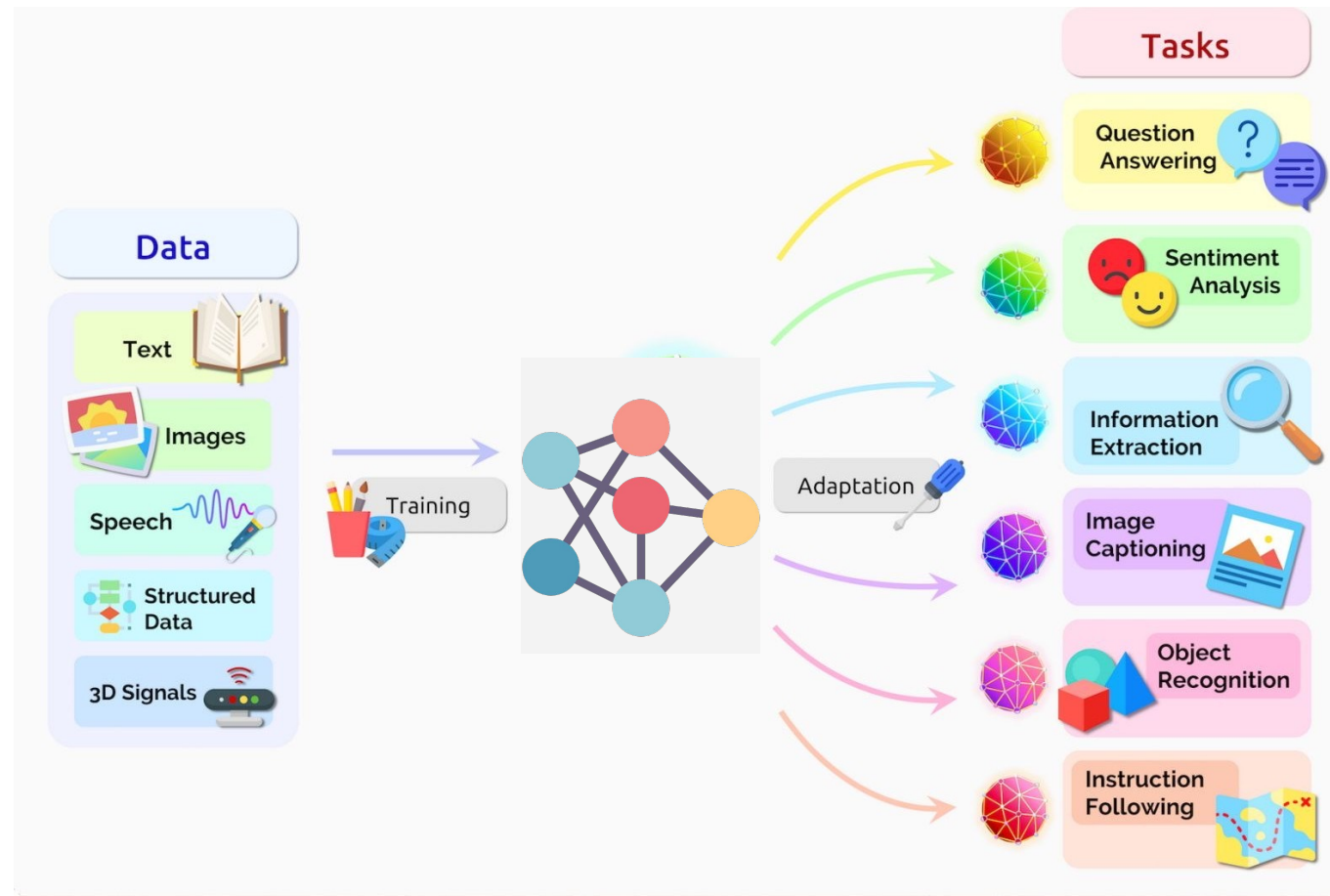
COMPANY OpenAI  
ROLE Researcher  
RAISED NA  
VALUATION NA



#### Illia Polosukhin

COMPANY NEAR Protocol  
ROLE Co-founder  
RAISED \$800 million in endowments  
VALUATION \$1.2 billion

# 广泛应用



<https://blogs.nvidia.com/blog/2022/03/25/what-is-a-transformer-model/>

# 现实世界的影响



Machine Translation



Smart Assistants



Search Engines



Auto Transcription



Health Record Analysis

google-research/  
pegasus



5 Contributions 79 Issues 1k Stars 261 Forks

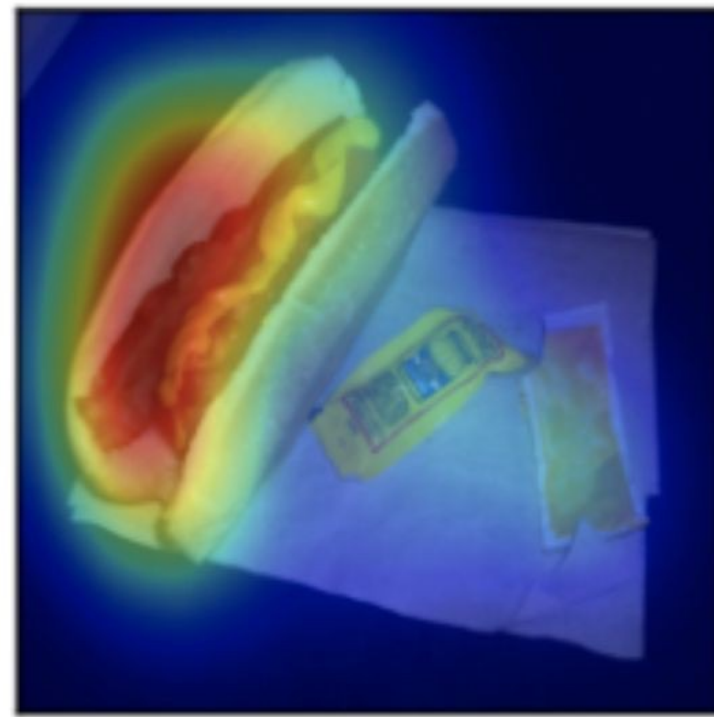
Summarization Engines

and many more ....



# 视觉注意力 Visual Attention

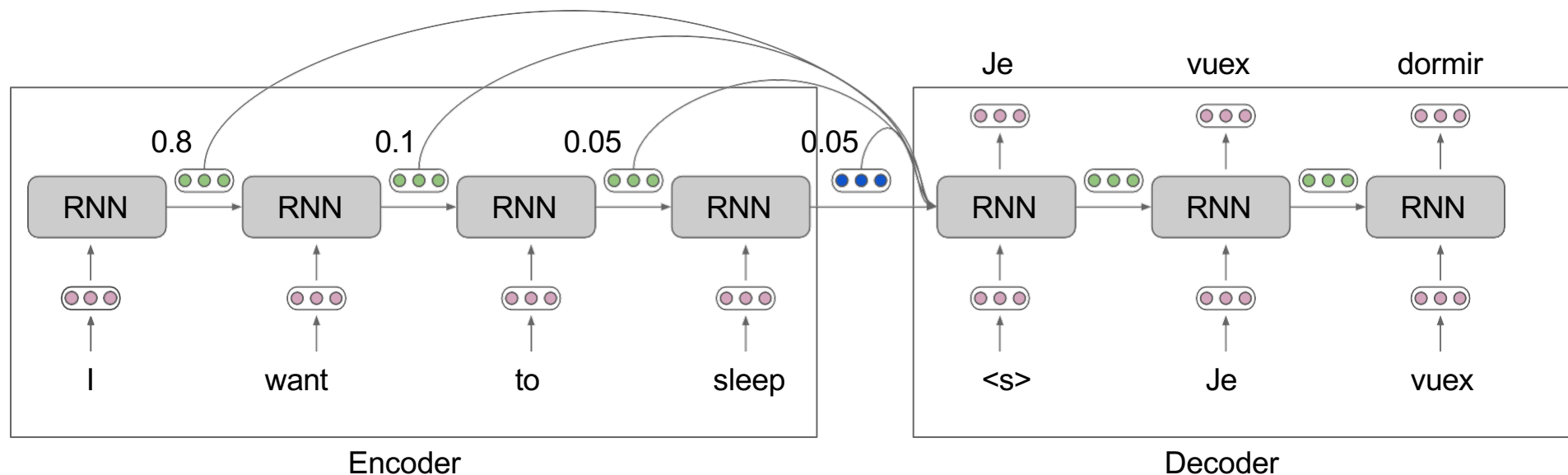
- 这个热狗加了什么配料？ / What toppings are on the hot dog?



[Differential Attention for Visual Question Answering](#) (Patro et.al, 2018)

# 神经机器翻译中的交叉注意力机制

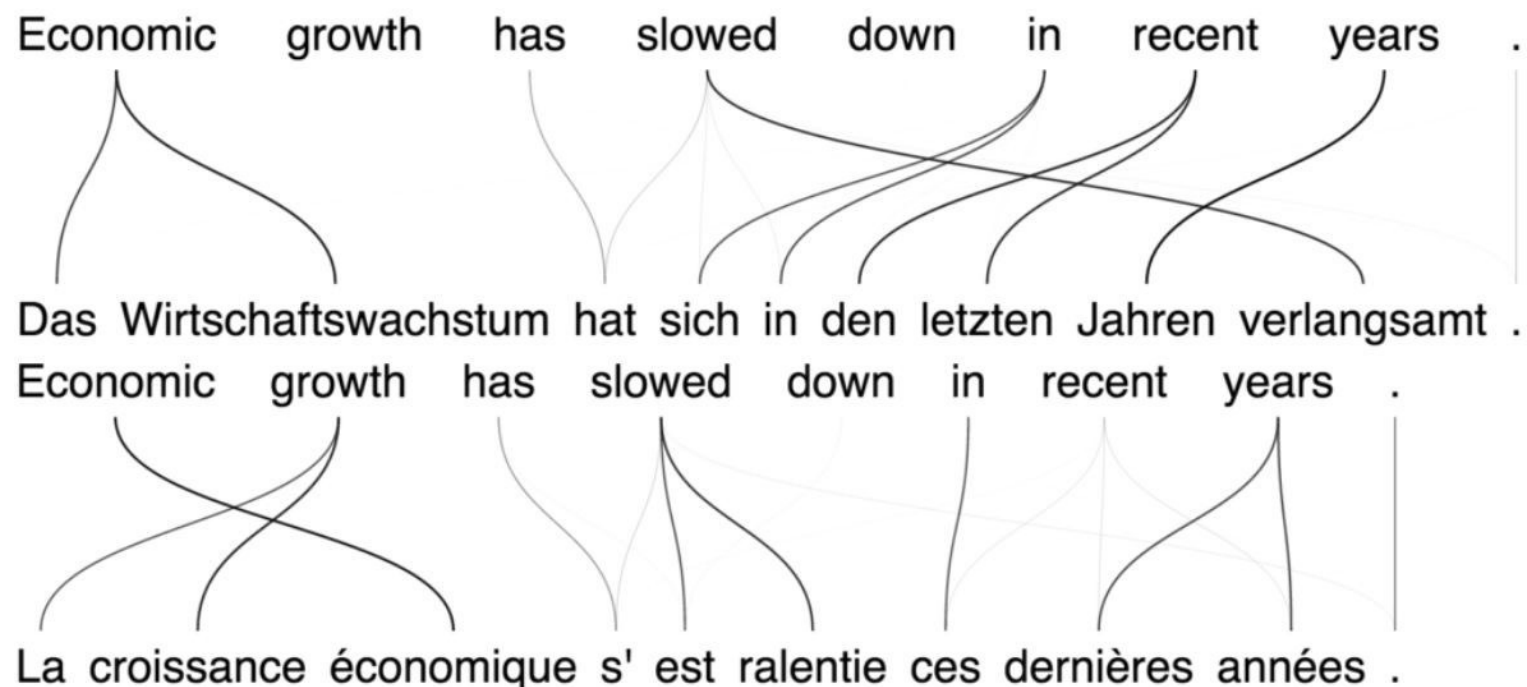
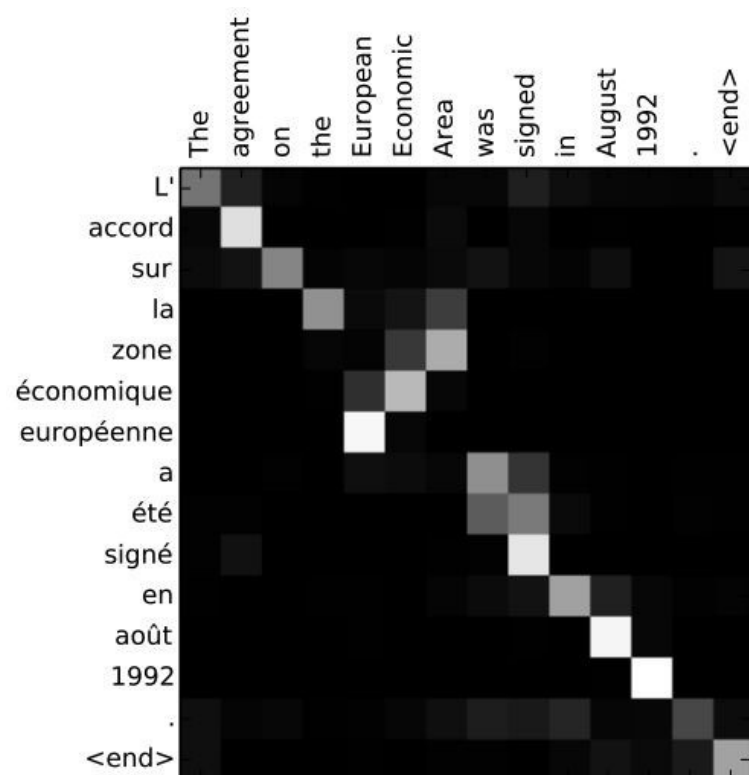
## Cross Attention in NMT



[Neural Machine Translation by Jointly Learning to Align and Translate](#). Bahdanau et al, 2015

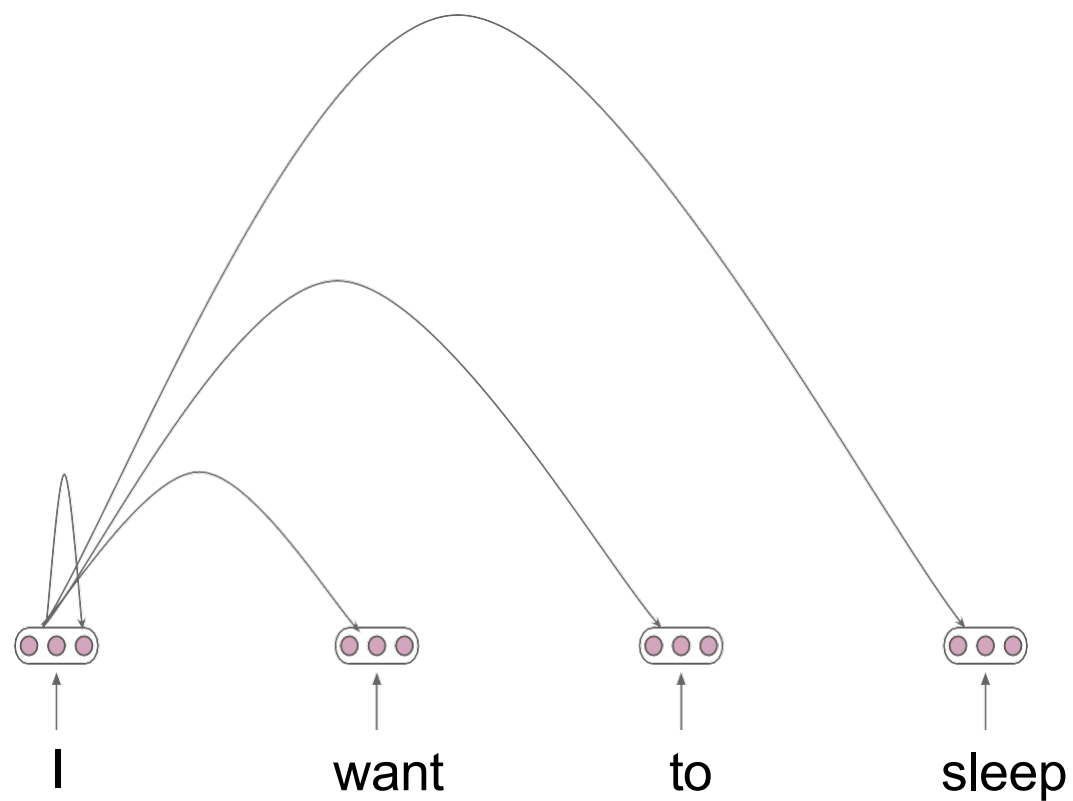
# 神经机器翻译中的注意力机制

## Attention in NMT



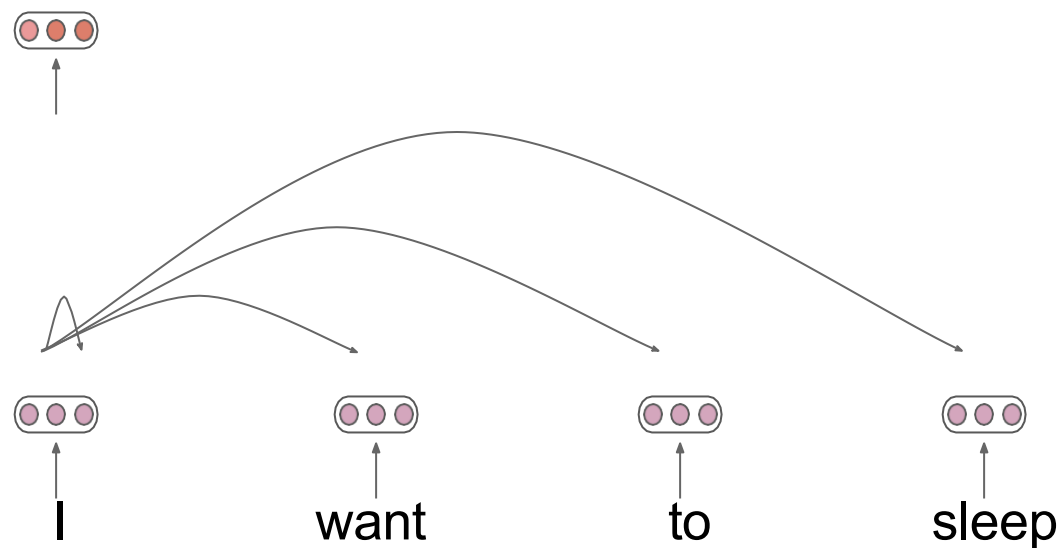
[Neural Machine Translation by Jointly Learning to Align and Translate](https://developer.nvidia.com/blog/introduction-neural-machine-translation-gpus-part-3/). Bahdanau et al, 2015  
<https://developer.nvidia.com/blog/introduction-neural-machine-translation-gpus-part-3/>

# 自注意力机制 Self Attention



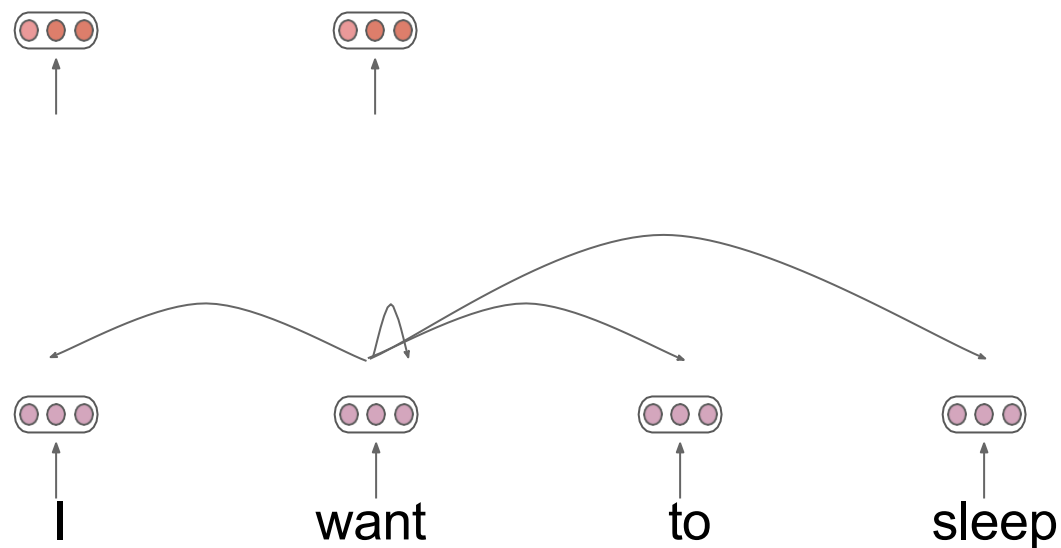
# 自注意力机制-告别循环结构

## Self Attention - No more recurrence



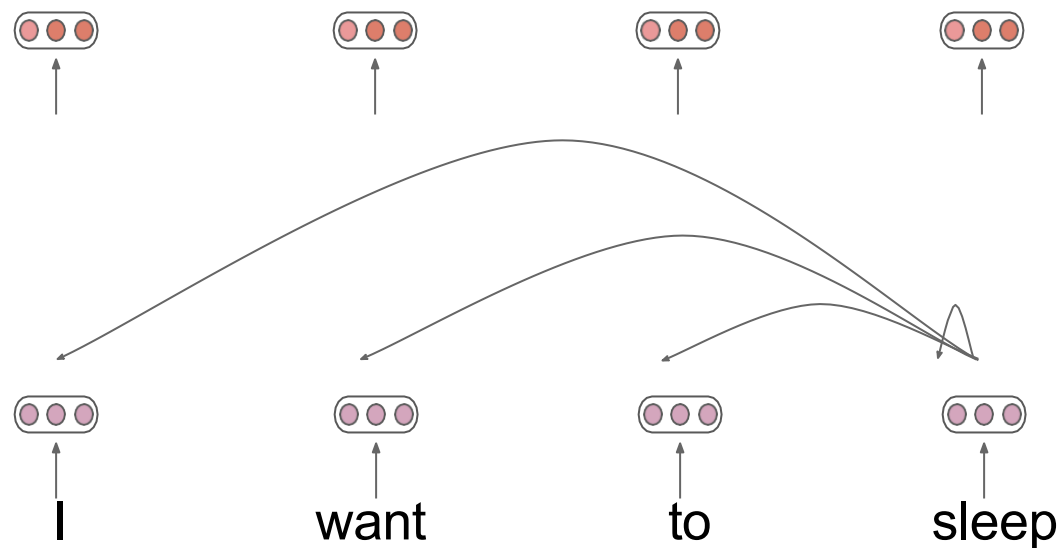
# 自注意力机制-告别循环结构

## Self Attention - No more recurrence



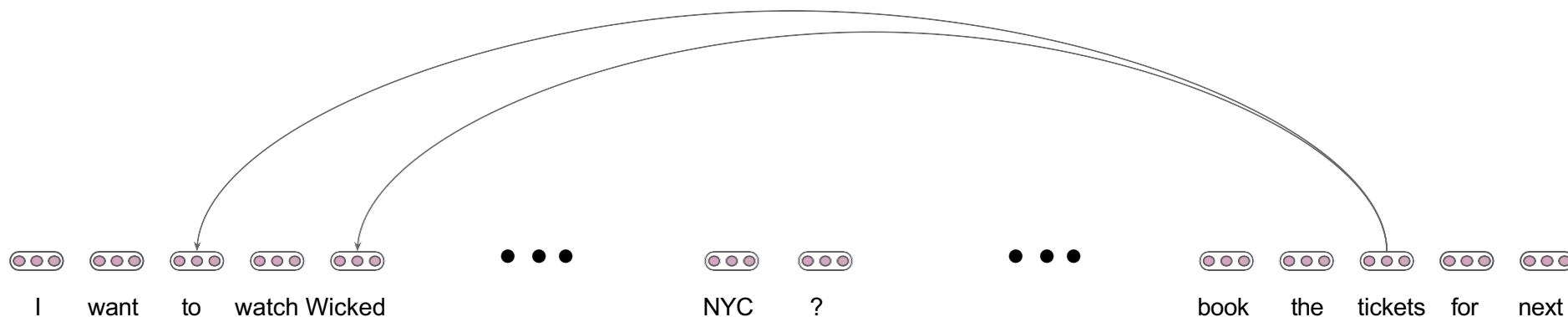
# 自注意力机制-告别循环结构

## Self Attention - No more recurrence



# 长序列自注意力机制

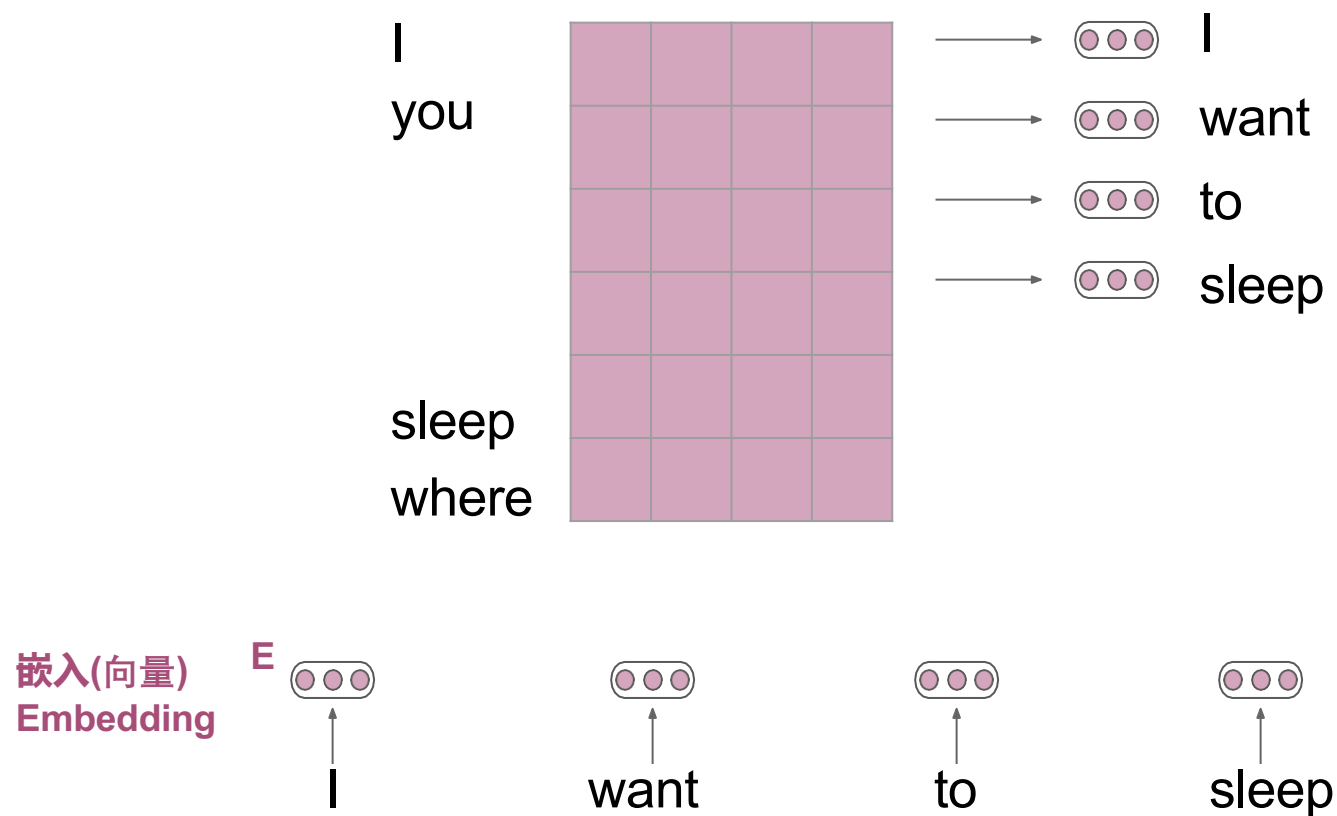
## Self Attention for Long Sequences





# 自注意力机制 – 词嵌入

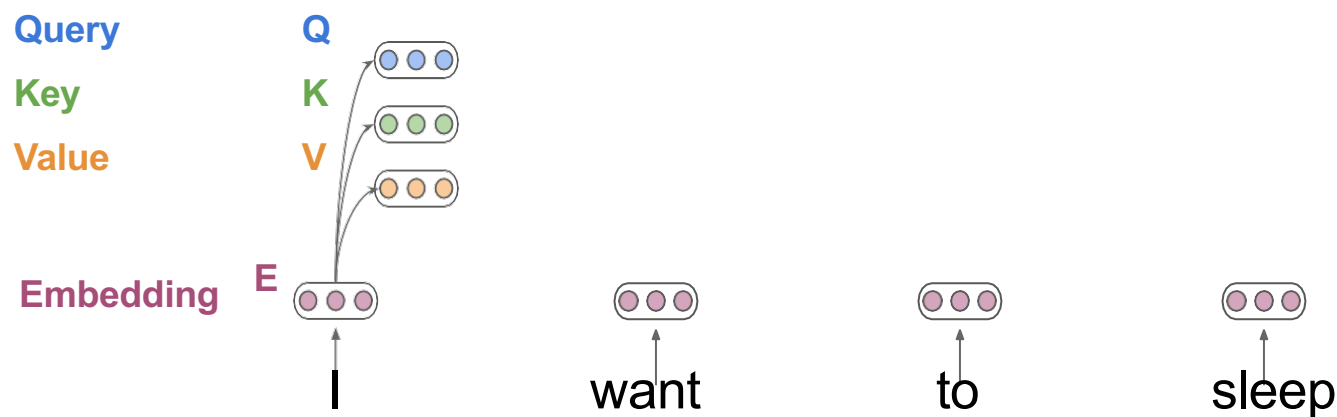
## Self Attention - Word Embedding



# 自注意力机制 – 投影层

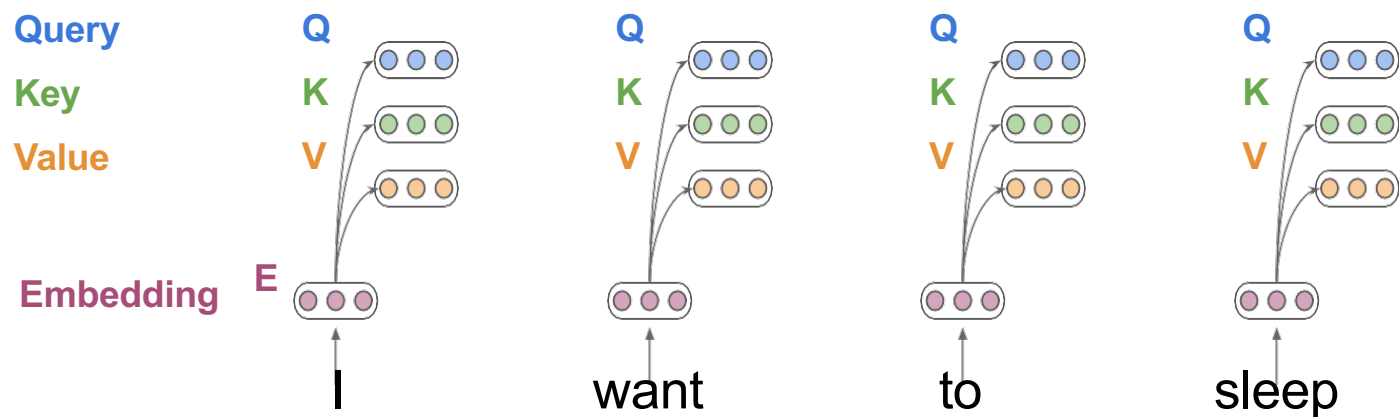
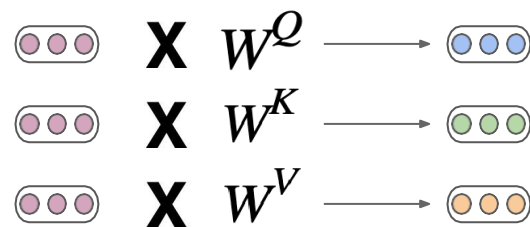
## Self Attention - Projection Layer

$$\begin{array}{lcl} \text{[3 purple circles]} & \mathbf{X} & \mathbf{W}^Q \longrightarrow \text{[3 blue circles]} \\ \text{[3 purple circles]} & \mathbf{X} & \mathbf{W}^K \longrightarrow \text{[3 green circles]} \\ \text{[3 purple circles]} & \mathbf{X} & \mathbf{W}^V \longrightarrow \text{[3 orange circles]} \end{array}$$

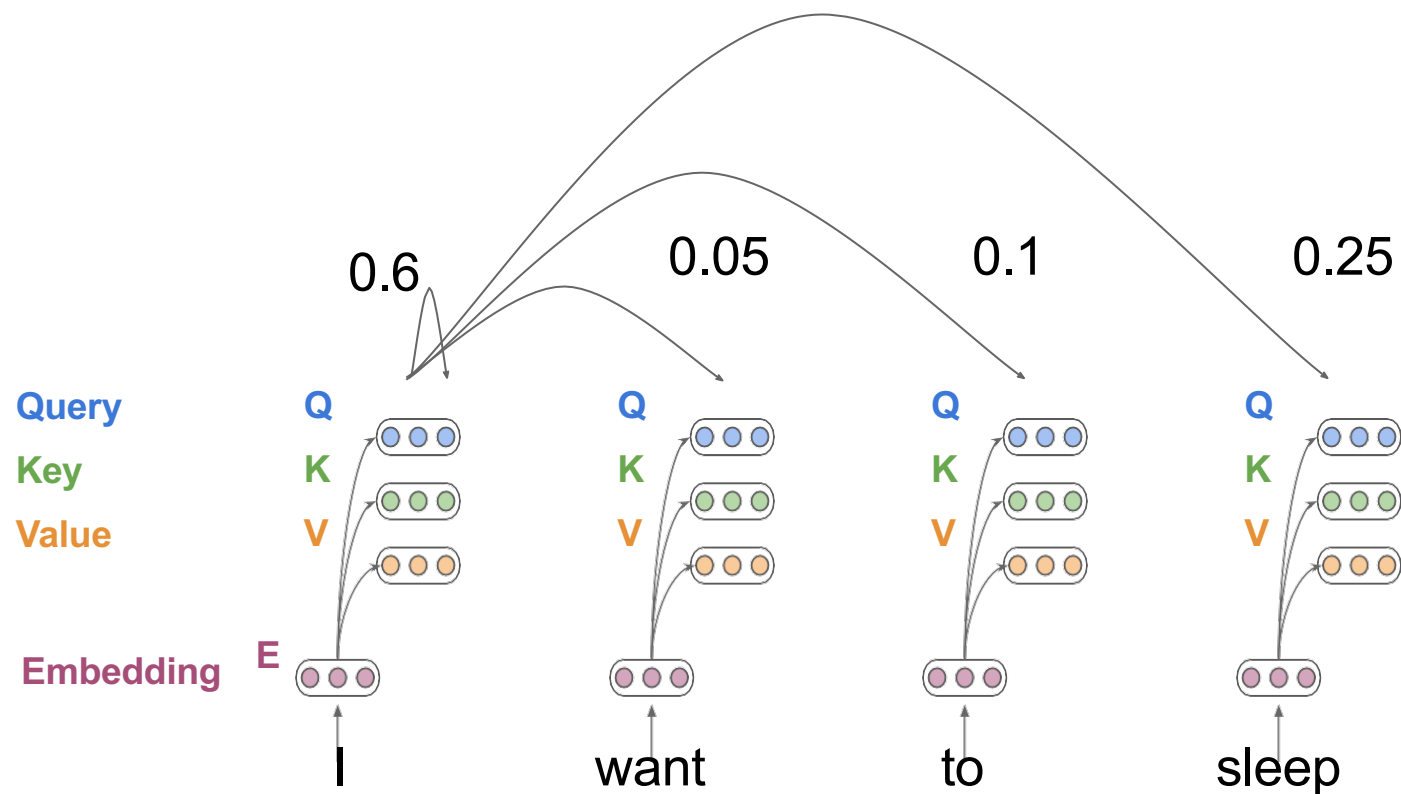


# 自注意力机制 – 投影层

## Self Attention - Projection Layer

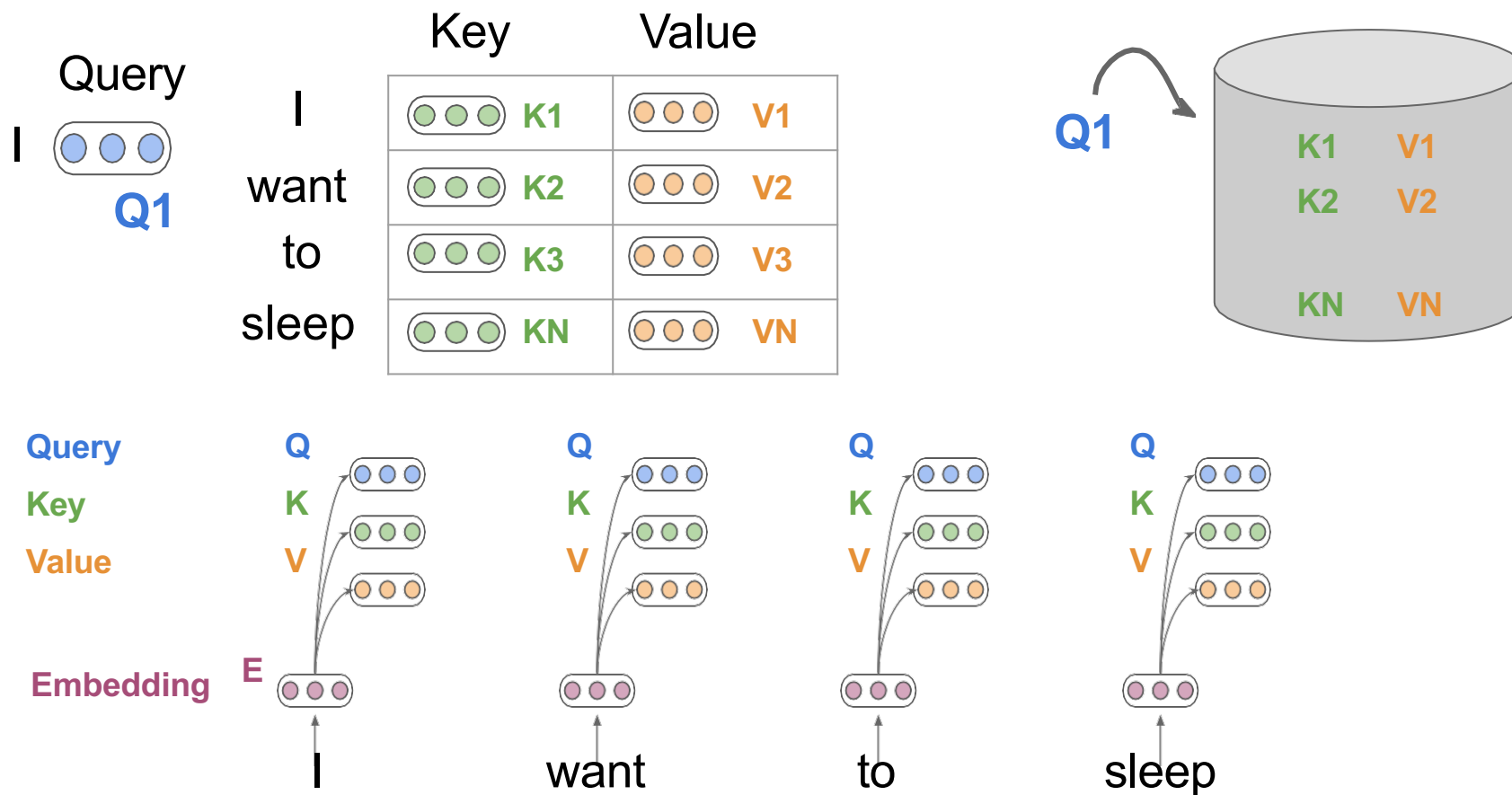


# 自注意力机制 – 注意力分数 Self Attention – Attention Score



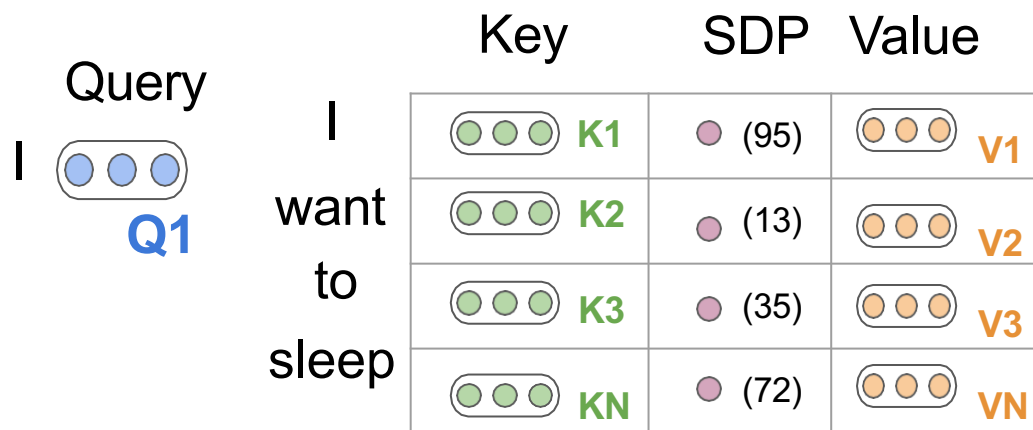
# 自注意力机制

## Self Attention



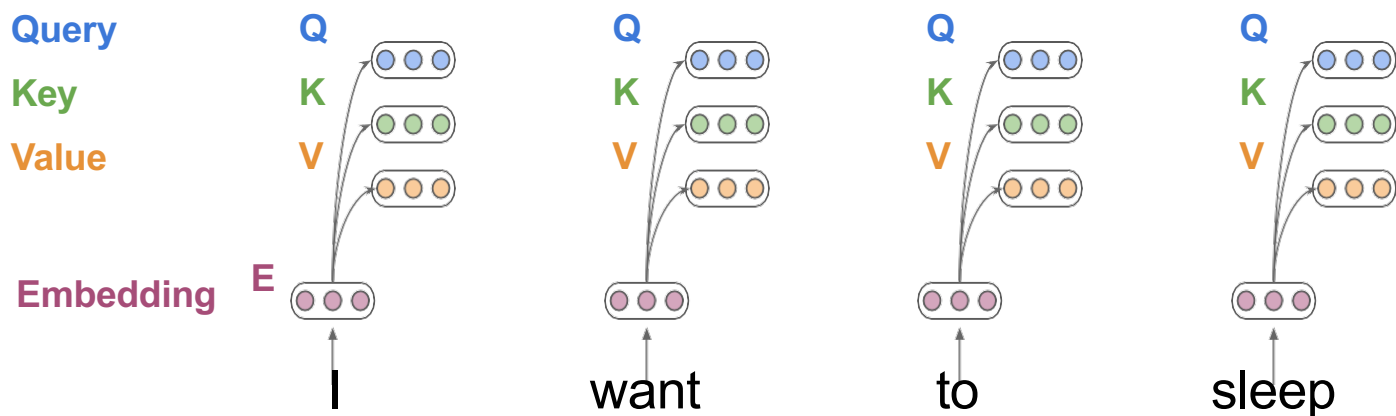
# 自注意力机制 – 缩放点积

## Self Attention – Scaled Dot Product



Scaled Dot Product

$$\text{SDP} = \frac{(QK^T)}{\sqrt{d_k}}$$

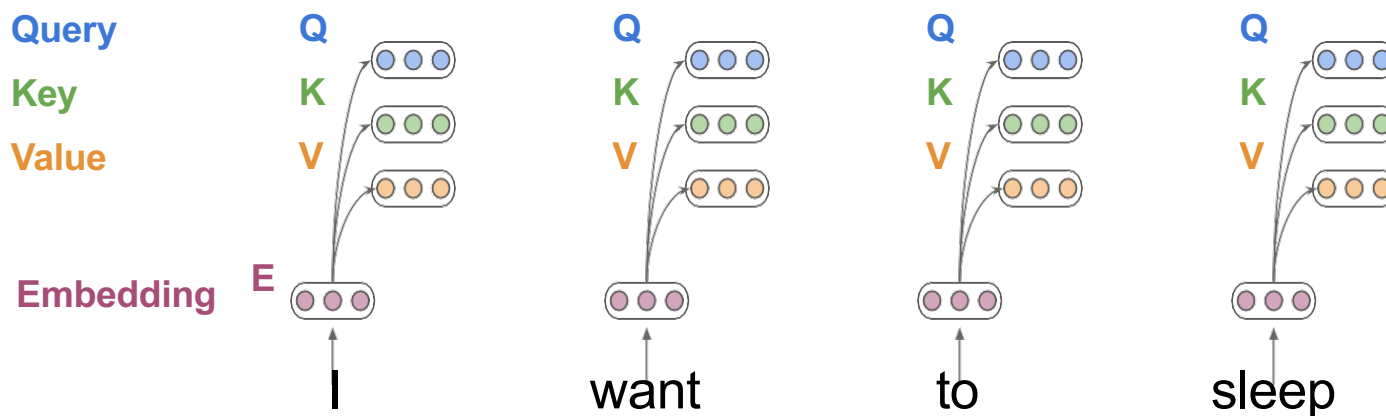


# 自注意力机制 – Softmax函数

## Self Attention – Softmax

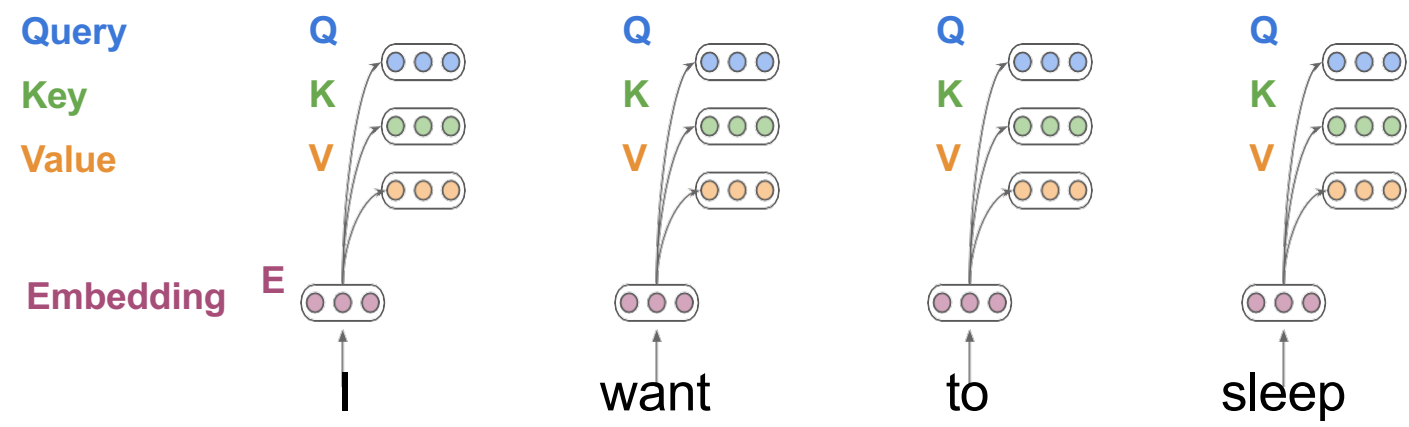
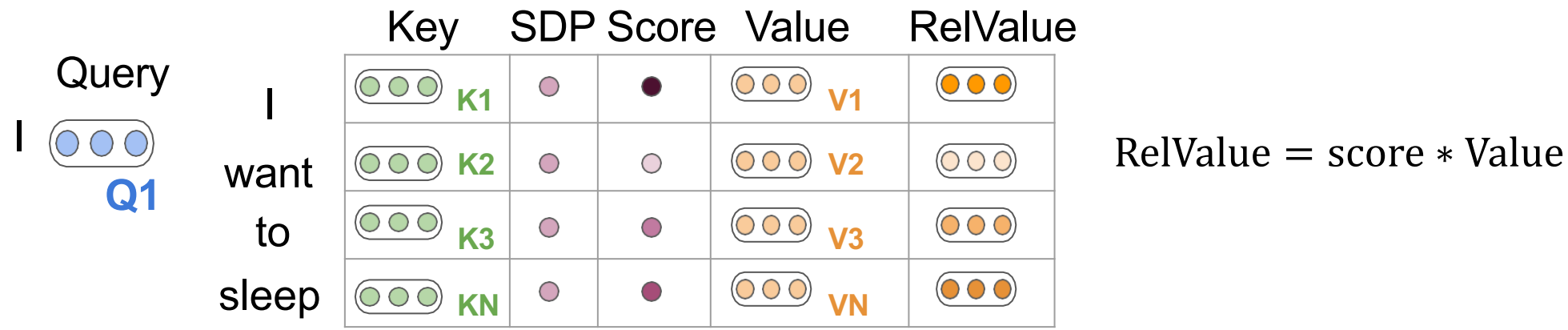
|                  | Key                                                                                   | SDP                                                                                      | Score                                                                                      | Value                                                                                  |
|------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Query<br>I<br>Q1 |  K1 |  (95) |  (0.6)  |  V1 |
| want             |  K2 |  (13) |  (0.05) |  V2 |
| to               |  K3 |  (35) |  (0.1)  |  V3 |
| sleep            |  KN |  (72) |  (0.25) |  VN |

$$\text{score} = \text{softmax}\left(\frac{(QK^T)}{\sqrt{d_k}}\right)$$



# 自注意力机制 – 柔性(相对)值

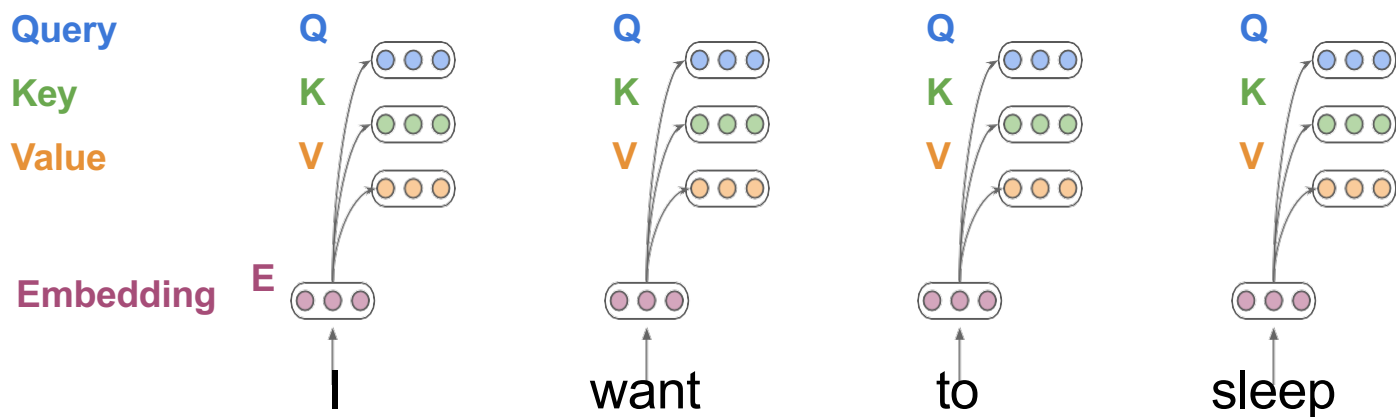
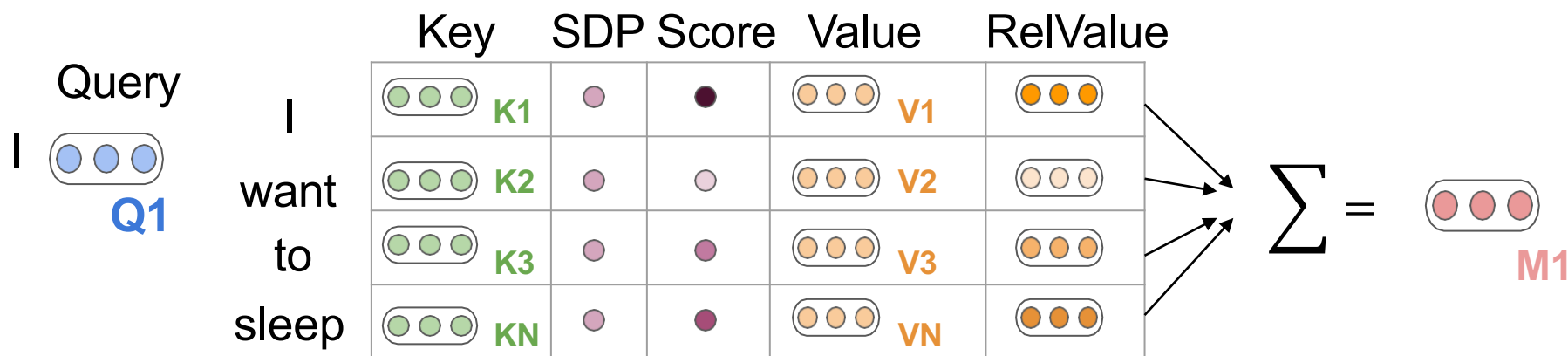
## Self Attention – Soft (Relative) Values





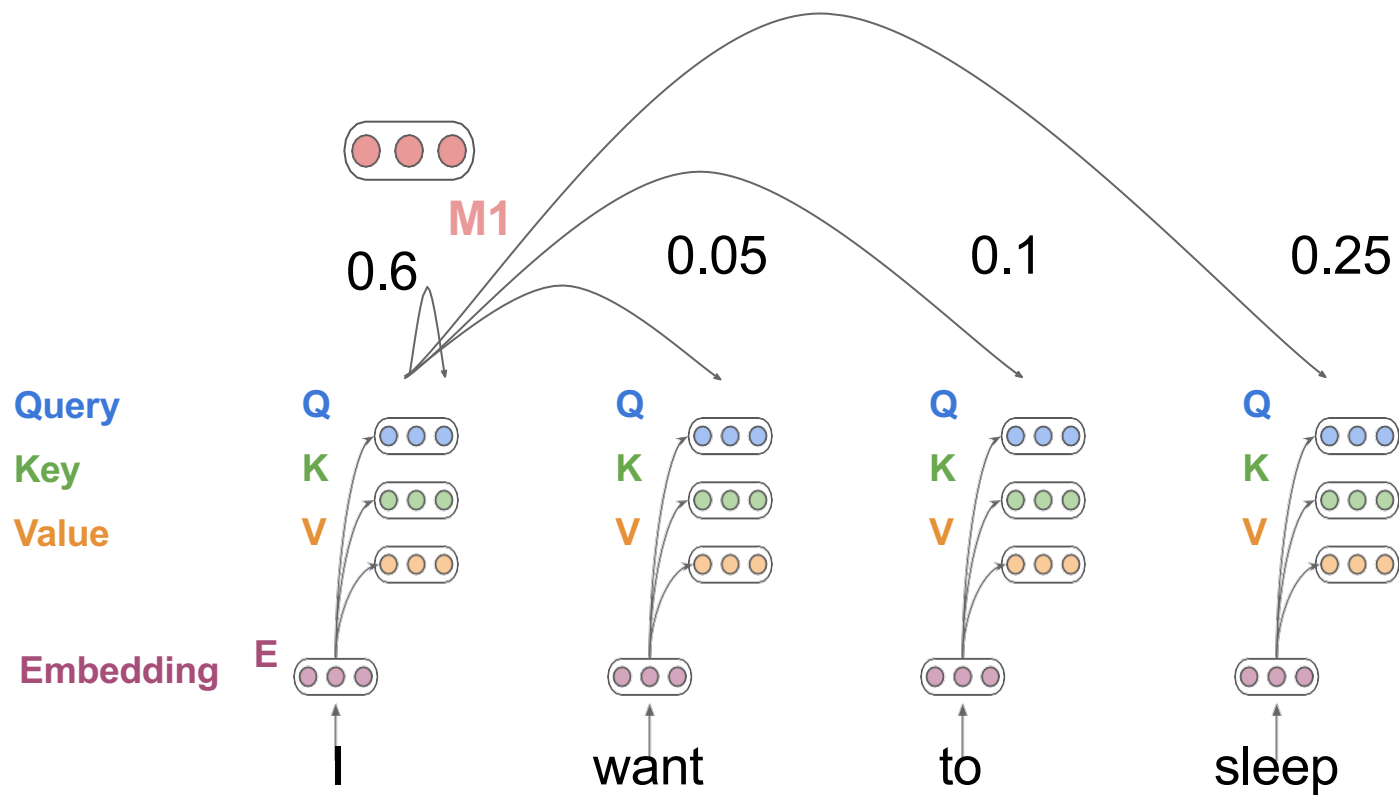
# 自注意力机制 – 注意力表征

## Self Attention – Attended Representation



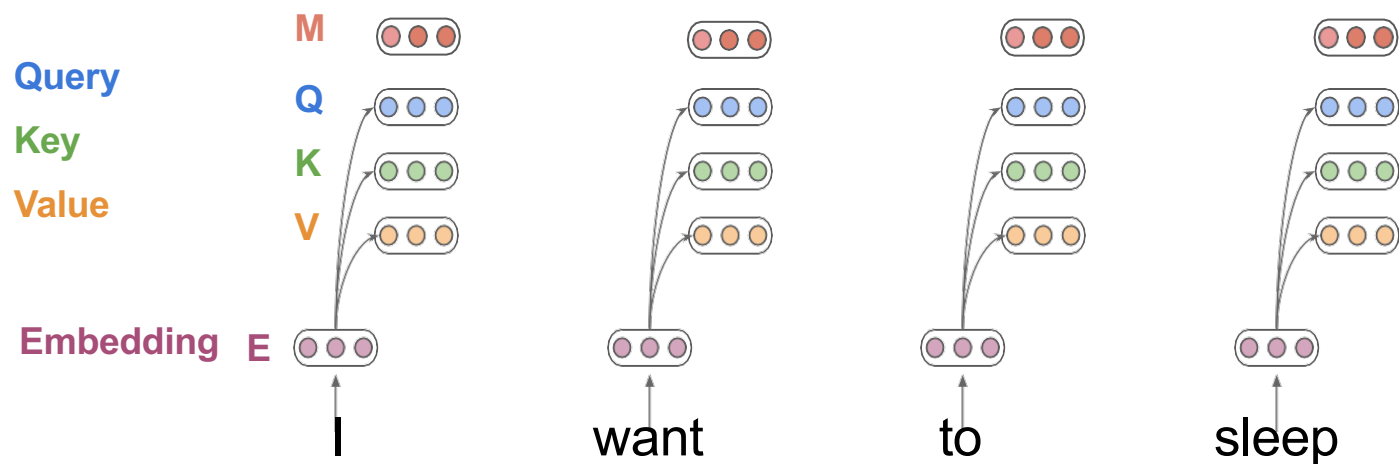
# 自注意力机制 – 注意力上下文表征

## Self Attention – Attended Contextual Rep



# 自注意力机制 – 注意力上下文表征

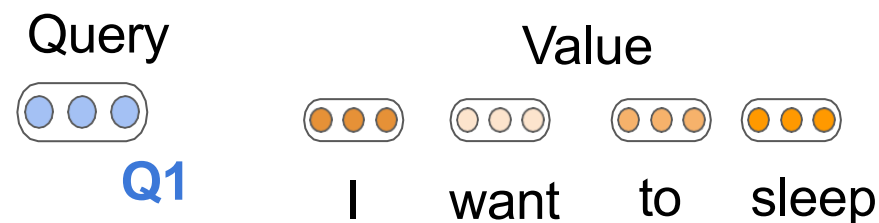
## Self Attention – Attended Contextual Rep



# 自注意力机制的问题

## Problem with Self Attention

- 自注意力机制会对同一个词分配极高的注意力权重！



- 单一表征

I like **Harry Potter**  
(Book)

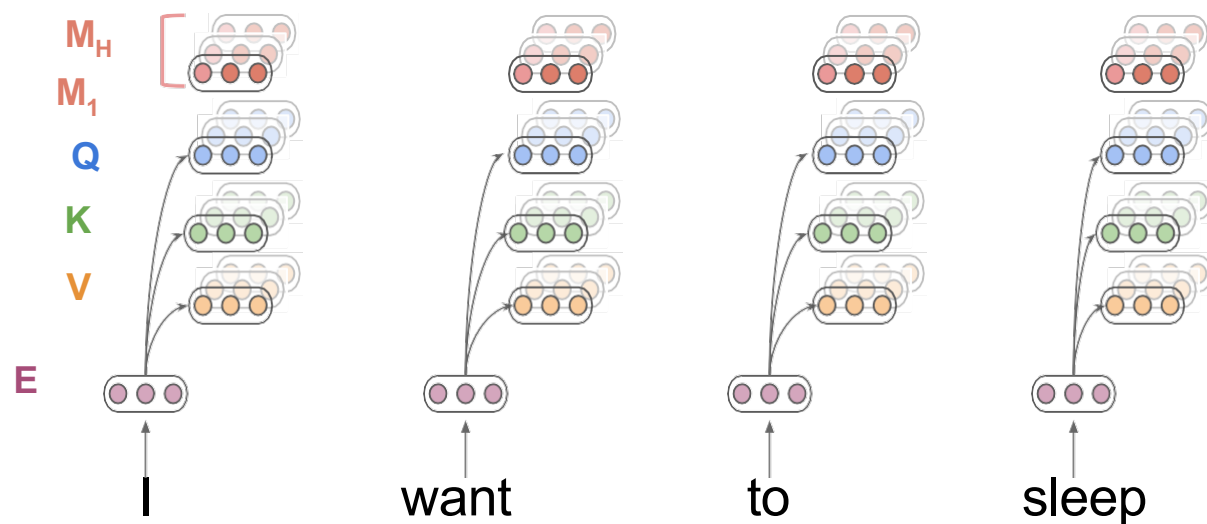
vs.

I like **Harry Potter**  
(Movie)

# 多头自注意力机制

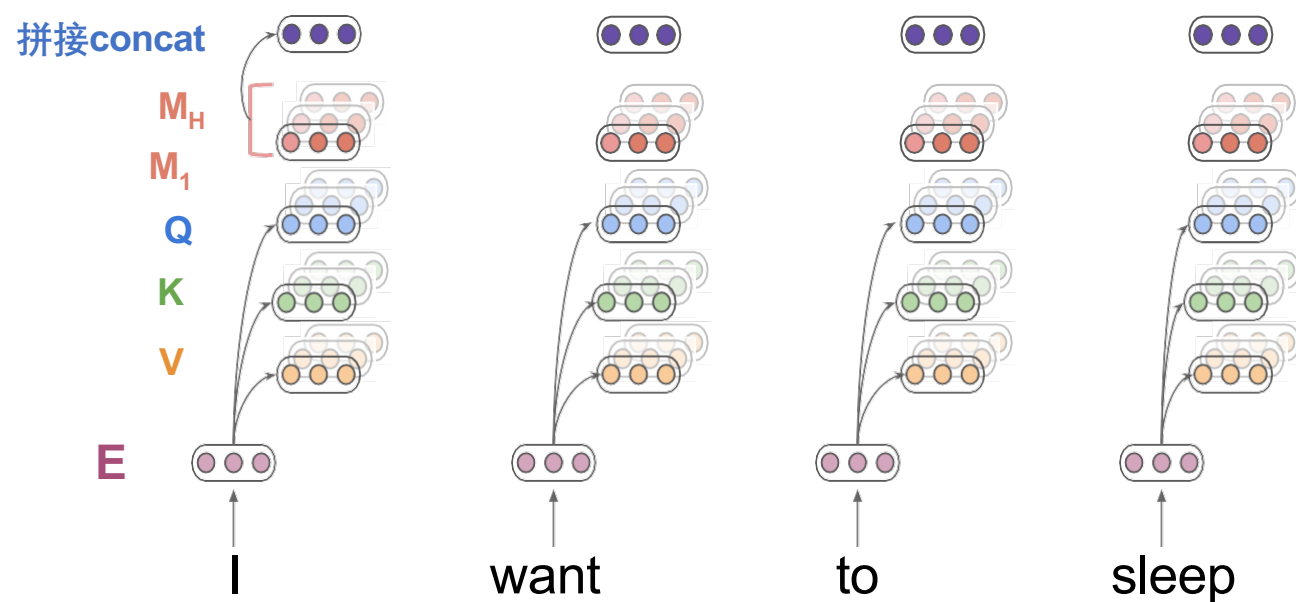
## Multi-Headed Self Attention

- $H$  (no: of heads) 不同版本的 $Q, K, V$
- 每个不同表征  $\rightarrow$  不同的注意力表征



# 多头自注意力机制

## Multi-Headed Self Attention



# 多头自注意力机制

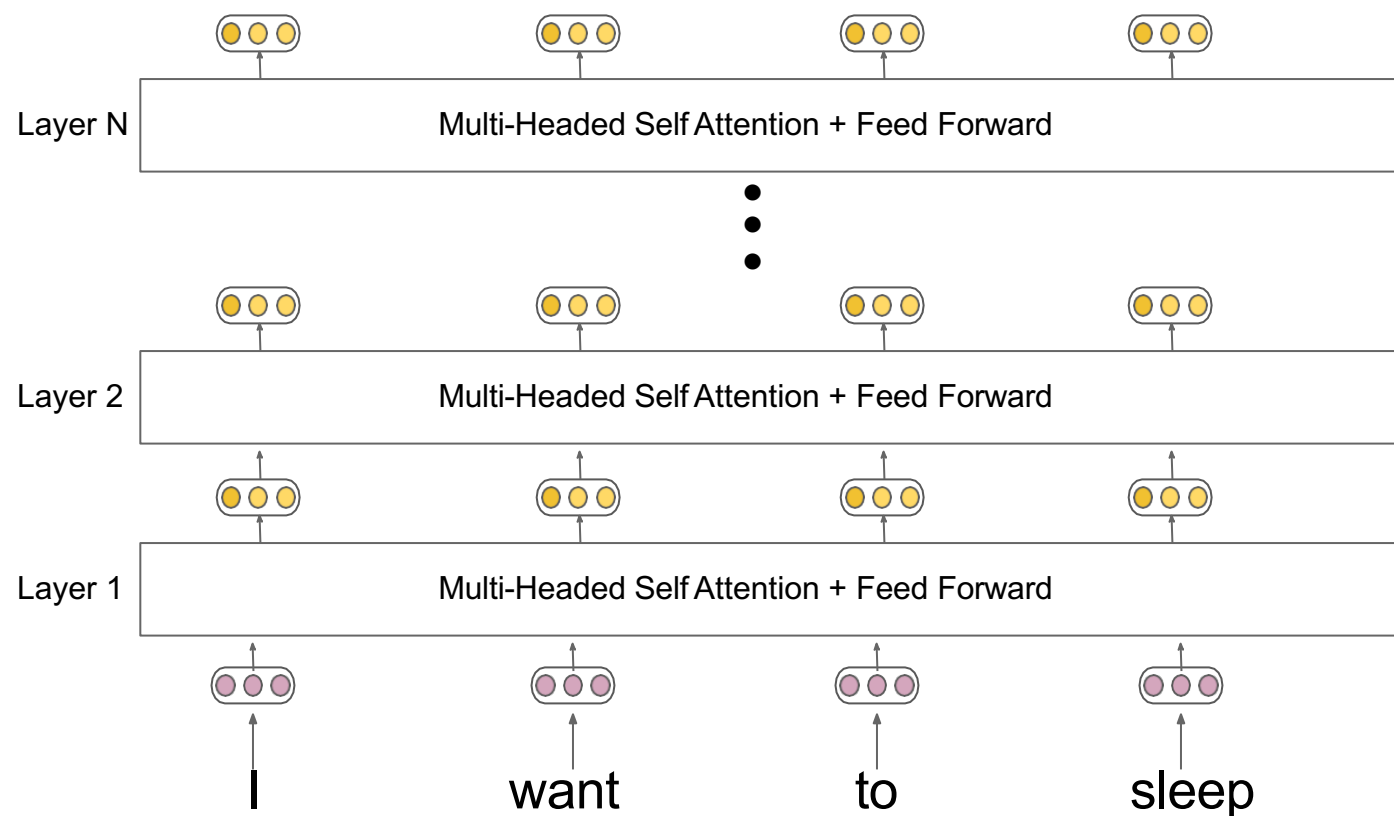
## Multi-Headed Self Attention

Multi-Headed  
Self Attention +  
Feed Forward



# 多头自注意力机制

## Multi-Headed Self Attention

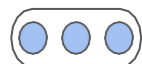




# 重审自注意力机制

## Revisiting Self Attention

Query (I)



|                          | Key                                                                                          | SDP                                                                                | Score                                                                              | Value                                                                                        | RelValue                                                                            |
|--------------------------|----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| I<br>want<br>to<br>sleep |  <b>K1</b>  |   |   |  <b>V1</b>  |   |
|                          |  <b>K2</b>  |   |   |  <b>V2</b>  |   |
|                          |  <b>K3</b>  |   |   |  <b>V3</b>  |   |
|                          |  <b>KN</b> |  |  |  <b>VN</b> |  |

$$\sum = \text{ M}$$

I want to sleep

# 重审自注意力机制

## Revisiting Self Attention

Query (I)

 两个句子的表征相同—但位置信息至关重要！ 

|                          | Key                                                                                   | SDP                                                                               | Score                                                                             | Value                                                                                 | RelValue                                                                            |
|--------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| I<br>want<br>to<br>sleep |  K1  |  |  |  V1  |   |
|                          |  K2  |  |  |  V2  |   |
|                          |  K3  |  |  |  V3  |   |
|                          |  KN |  |  |  VN |  |

$$\sum = \text{ M}$$

I want to sleep

Query (I)

|                          | Key                                                                                    | SDP                                                                                 | Score                                                                               | Value                                                                                  | RelValue                                                                            |
|--------------------------|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| sleep<br>to<br>want<br>I |  K1 |  |  |  V1 |  |
|                          |  K2 |  |  |  V2 |  |
|                          |  K3 |  |  |  V3 |  |
|                          |  KN |  |  |  VN |  |

$$\sum = \text{ M}$$

sleep to want I

# 位置编码 Positional Encoding

- 位置嵌入/position embedding: 每个位置编号都对应一个关联嵌入向量
  - 正弦位置嵌入 一可泛化至任意序列长度

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

## 优点

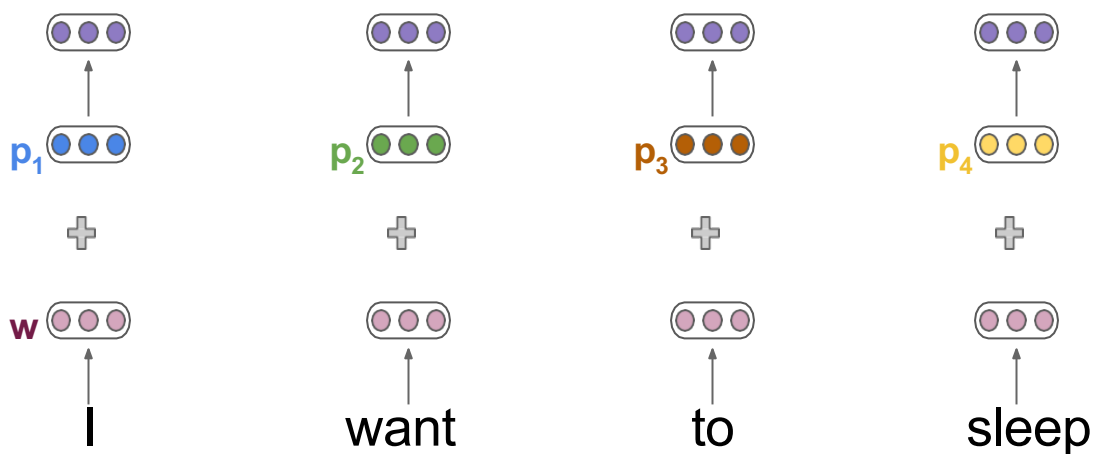
- 低频分量对应高维度
- 高频分量对应低维度

## 为什么是相加？

- 高维正交性

还有其他方案吗？

- [Rotary Position Embedding \(RoPE\)](#)



# Transformer编码器

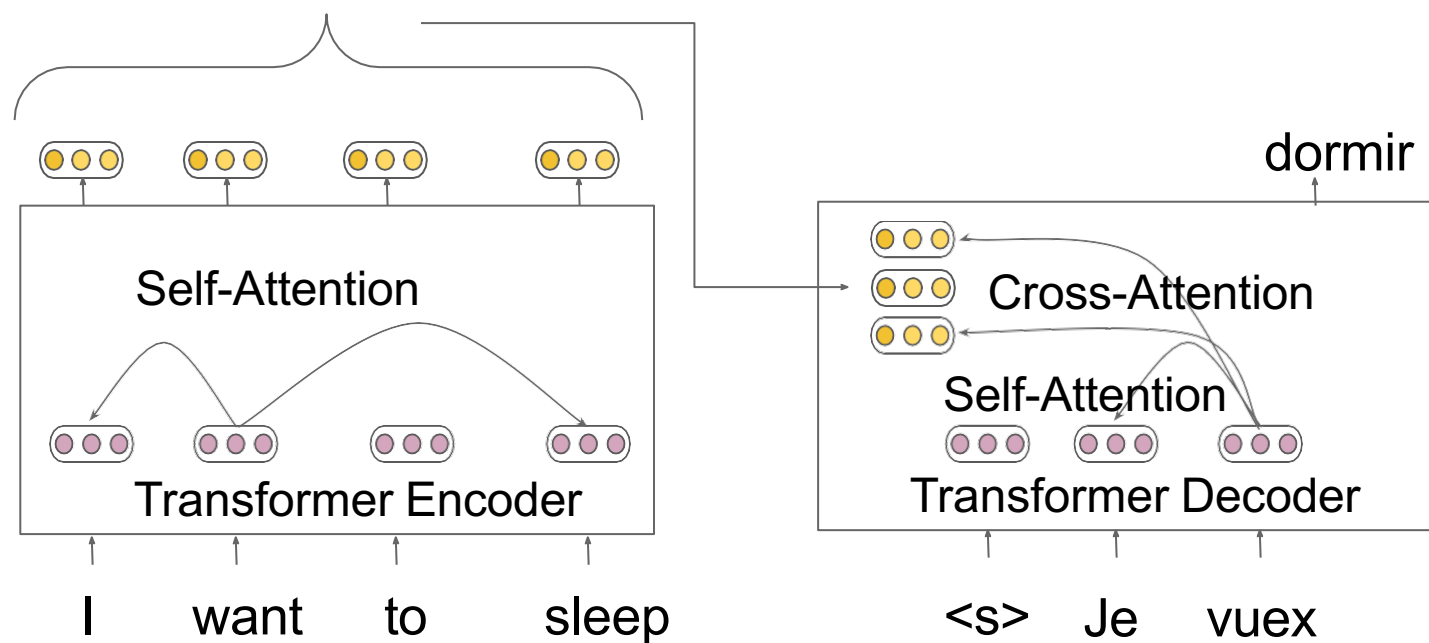
## Transformer Encoder

N-Layer  
Transformer  
Encoder



# Transformer编码器-解码器

## Transformer Encoder-Decoder

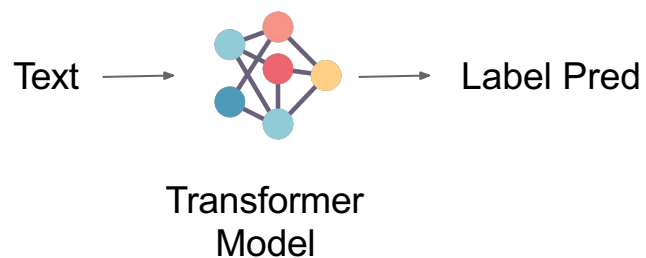


# Transformer的优势

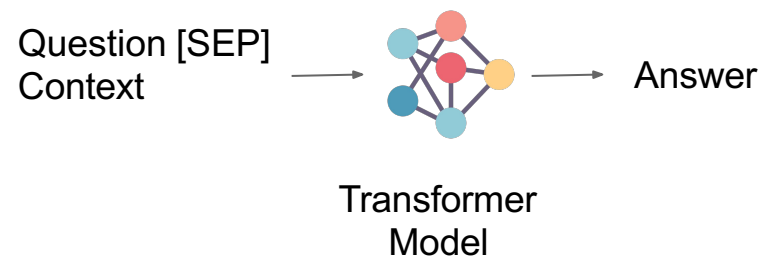
- 可并行化计算
  - 全序列、所有查询向量、所有注意力头均可并行计算
  - 借助 GPU 的高速矩阵乘法可实现性能增益
- 具备丰富的表征能力
  - 每个词元/token均与其余所有词元建立关联
  - 可构建长距离依赖关系
- 模型深度与序列长度无关
  - 缓解梯度爆炸/消失问题
  - 收敛速度更快

# 影响 - 广泛的应用

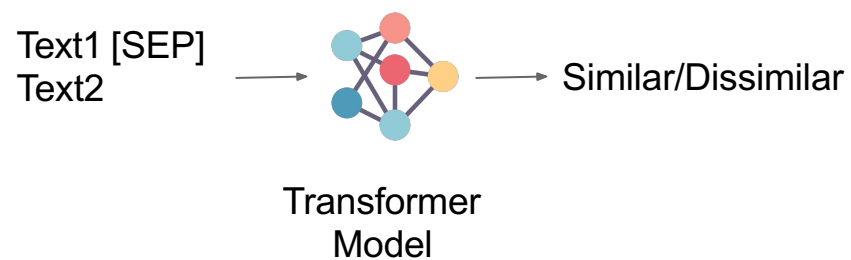
## Classification



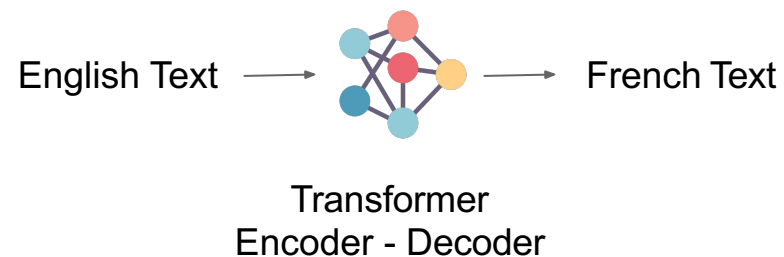
## Question Answering



## Sentence Similarity

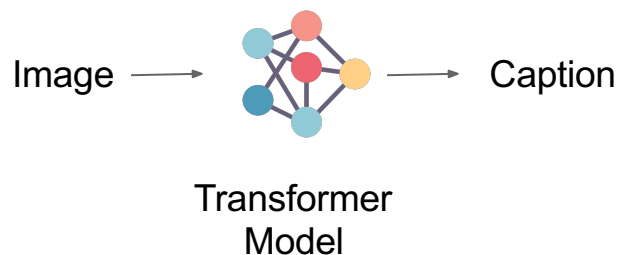


## Translation

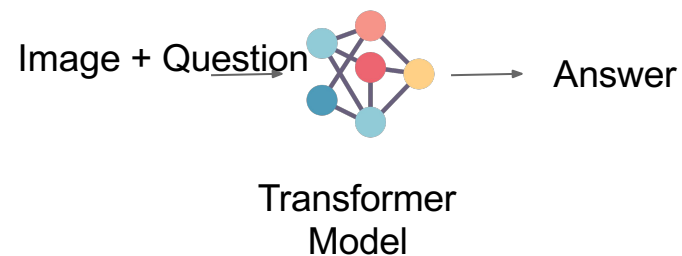


# 影响 - 广泛的应用

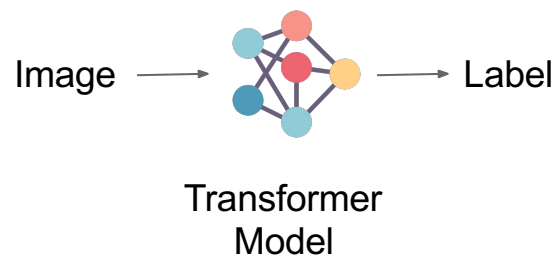
## Captioning



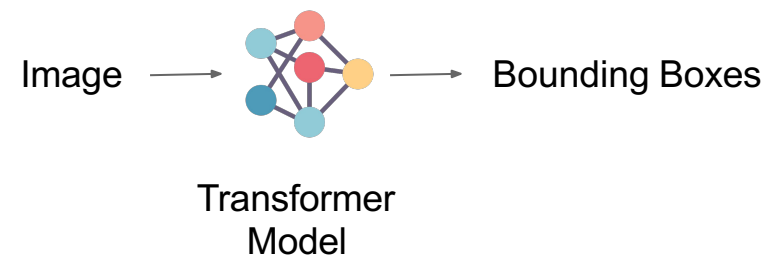
## Visual Question Answering



## Image Classification



## Object Detection



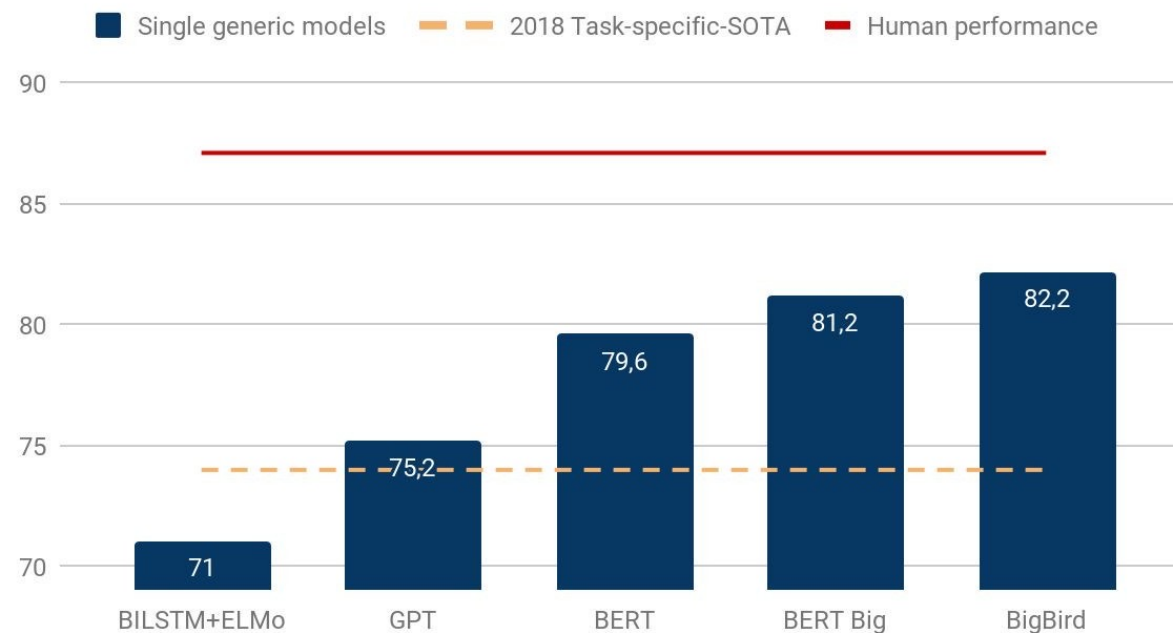


# 更大的影响

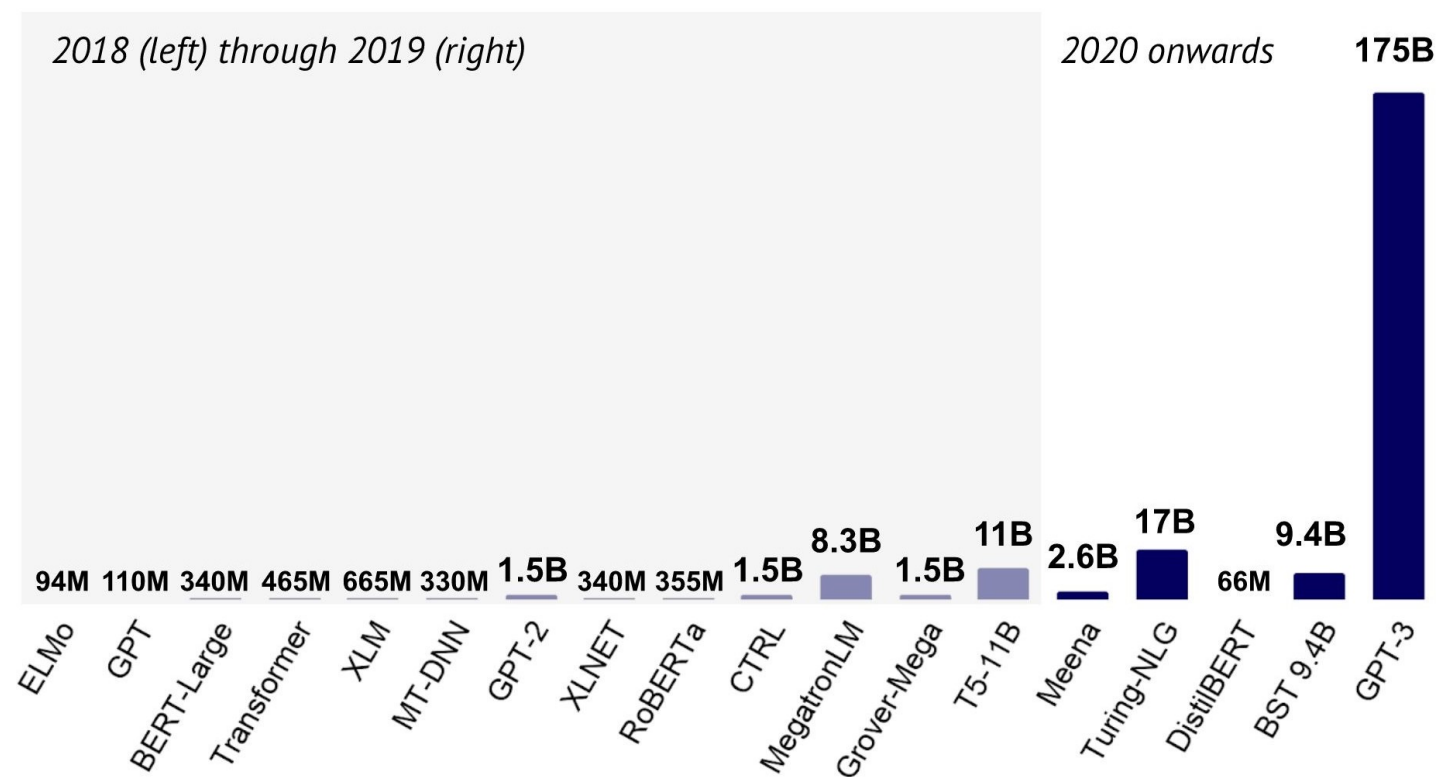


# 更大的影响

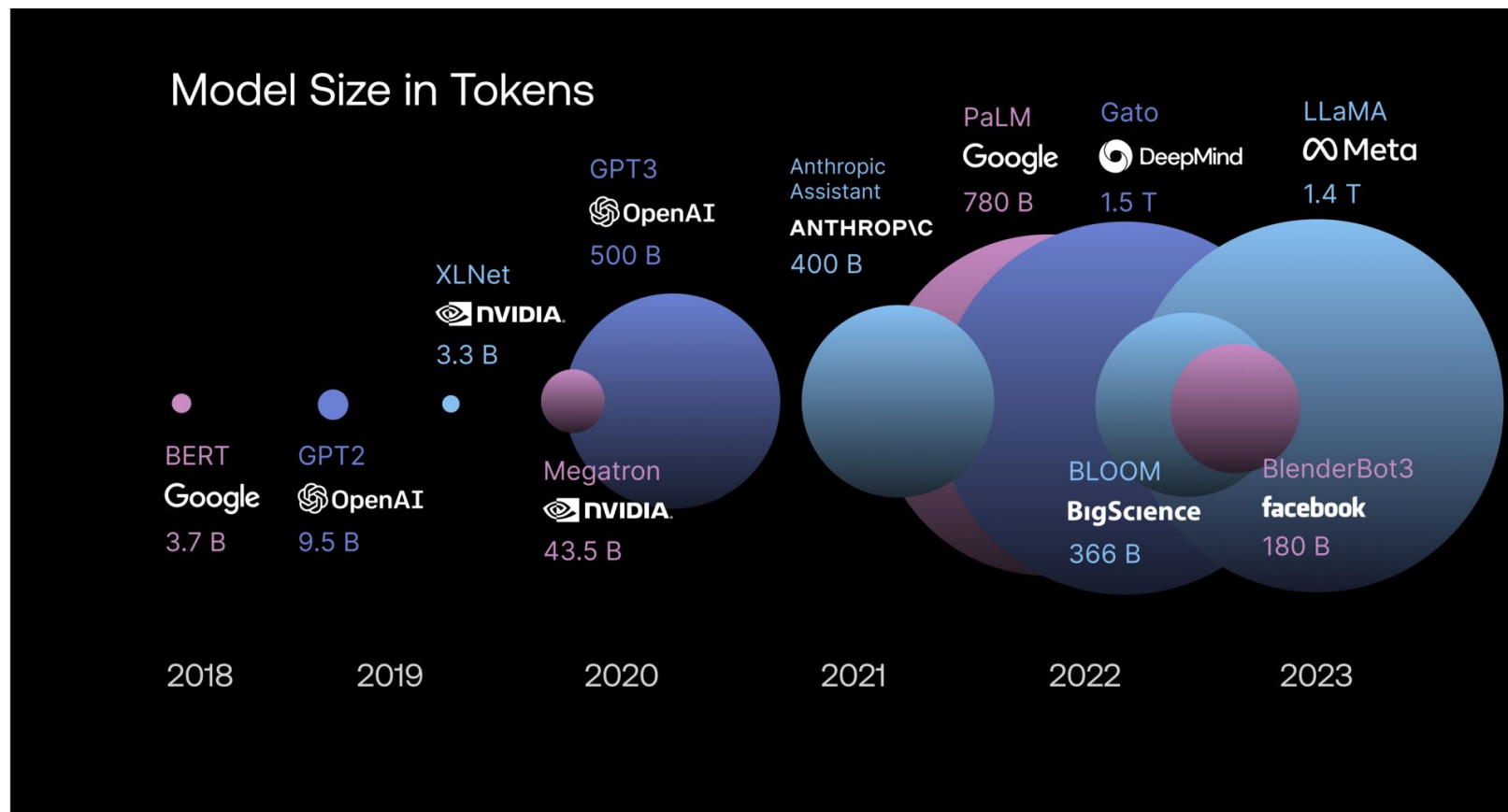
GLUE scores evolution over 2018-2019



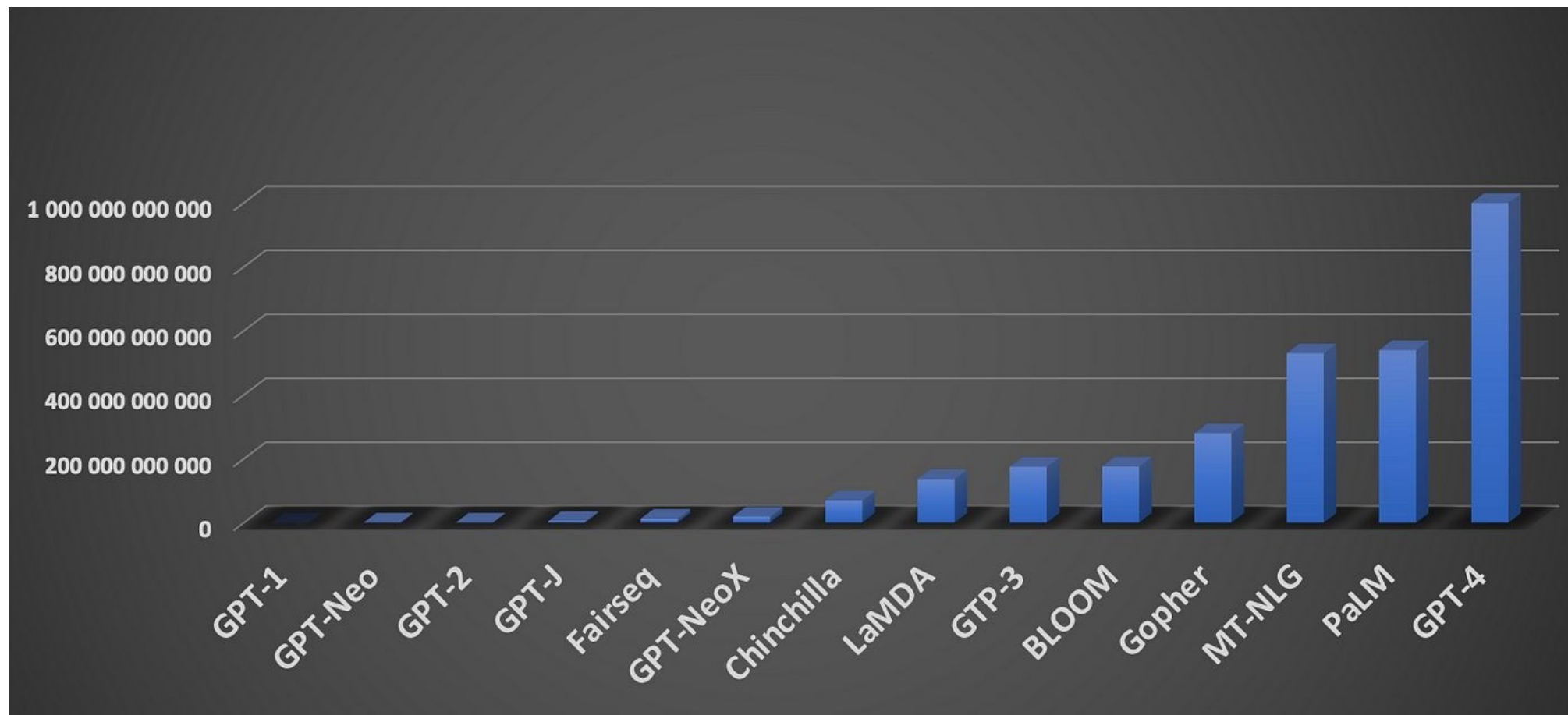
# 更大的影响



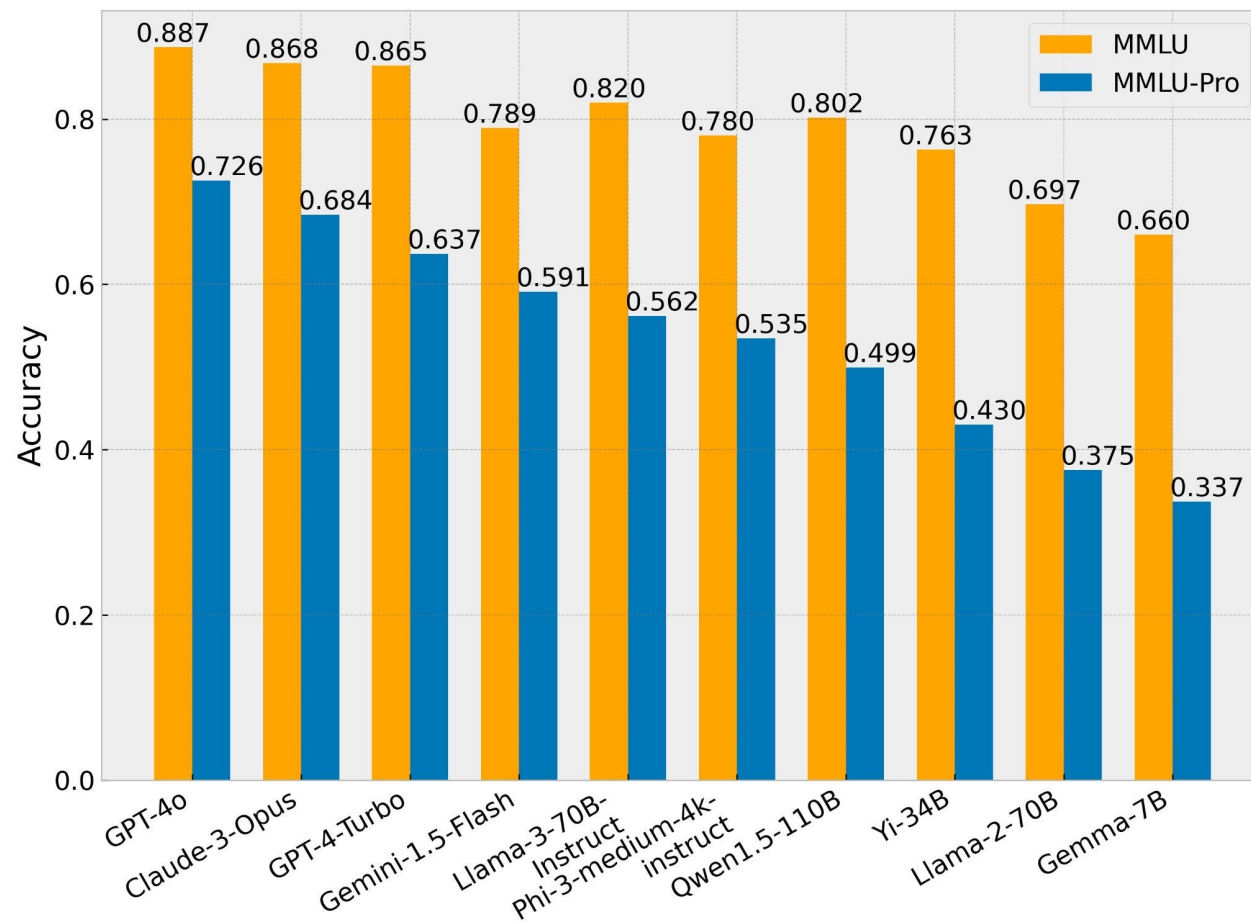
# 更大的影响



# 更大的影响



# 更大的影响



# 更大的影响



AI will revolutionise research. But could it transform science altogether?



Announcing Microsoft Copilot, your everyday AI companion



**More schools want your kids to use ChatGPT. Really.**

Education leaders are embracing technology that set off a plagiarism panic just months ago.



How will leveraging AI change the future of legal services?



**NYU Langone Health LLM can predict hospital readmissions**



**GPT-4's potential in shaping the future of radiology**



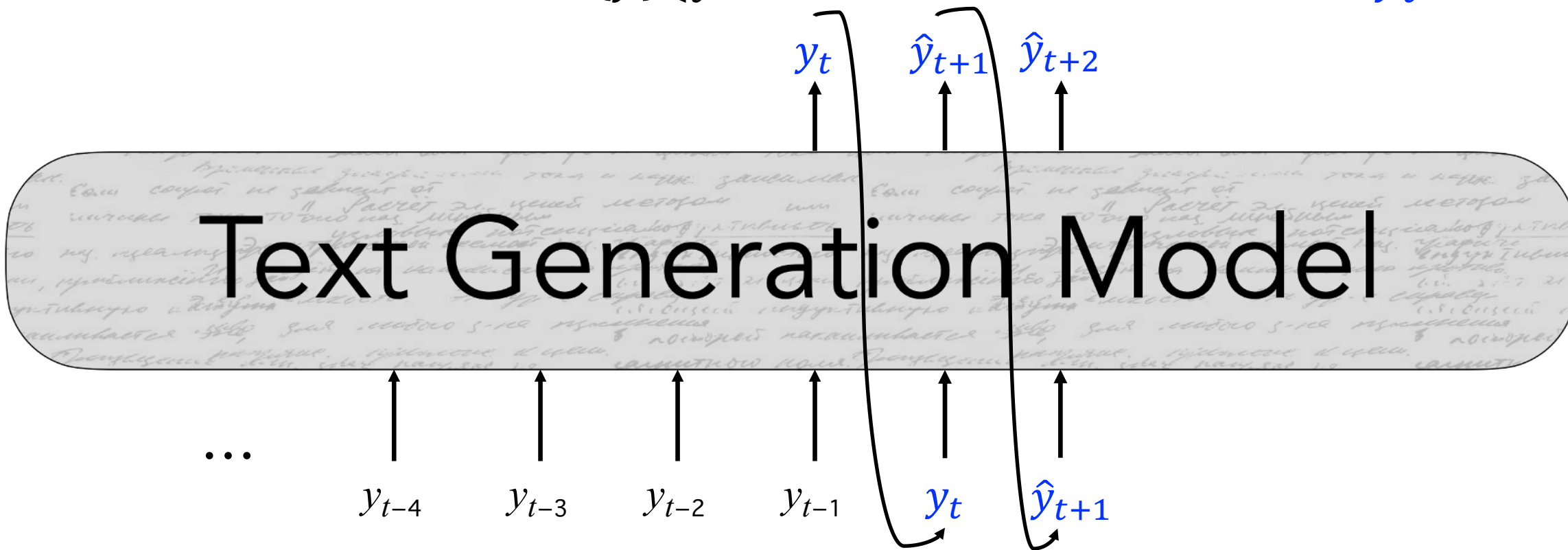
**Bing, Bard, and ChatGPT: How AI is rewriting the internet**

## 2. Open Ended Text Generation



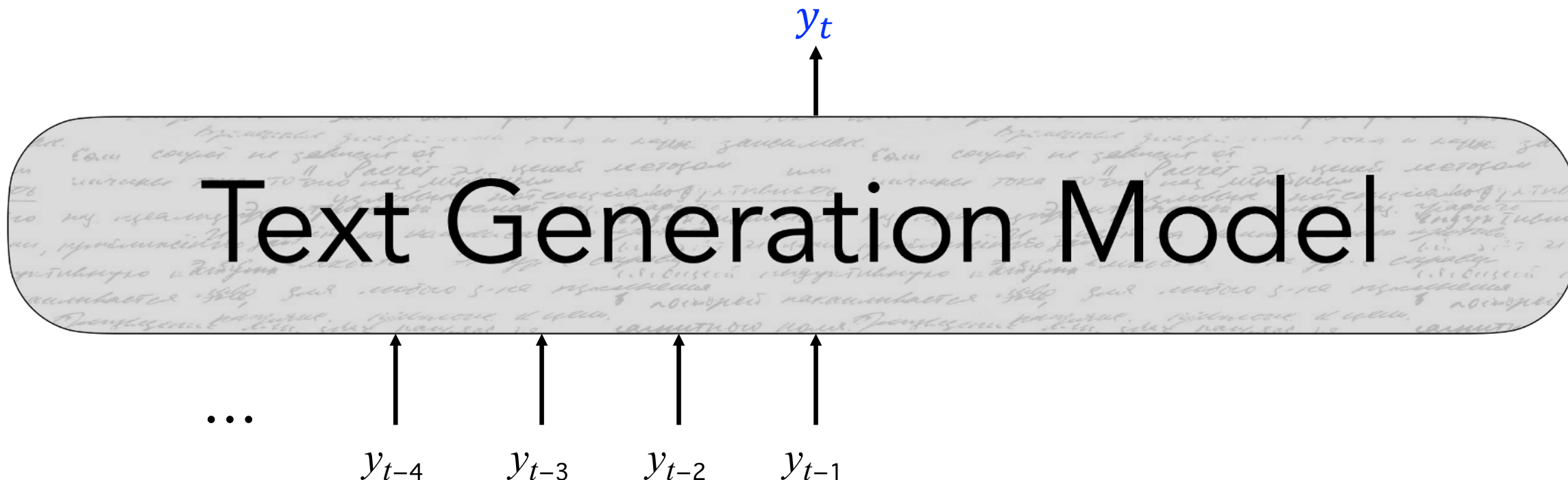
# 自然语言生成基础

- 在自回归文本生成模型中，每一时间步 $t$ ，模型会接收一个未包含当前位置的标记序列 $\{y\}_{<t}$ 作为输入，并输出一个新标记 $y_t$



# 单步过程解析

- 在自回归文本生成模型中，每一时间步 $t$ ，模型会接收一个未包含当前位置的标记序列 $\{y\}_{<t}$ 作为输入，并输出一个新标记 $y_t$



# 自然语言生成基础

- 在每一时间步 $t$ ，模型会为词表中的每个标记计算一个得分向量  $S \in R^{|V|}$ ：

$$S = \underline{f(\{y\}_{<t}; \theta)} \quad f(\cdot; \theta) \text{ 为模型}$$

- 随后，我们基于这些得分，计算词表 $V$ 中每个词元 $w$ 对应的概率分布：

$$P(y_t = w | \{y\}_{<t}) = \frac{\exp(S_w)}{\sum_{w' \in V} \exp(S_{w'})}$$

# 单步过程解析

- 在每一时间步 $t$ ，模型会为词表中的每个标记计算一个得分向量  $S \in R^{|V|}$ 。随后，我们基于这些得分，计算词表 $V$ 中每个词元 $w$ 对应的概率分布：

$$P(y_t | \{y\}_{<t})$$

softmax

$S$

Text Generation Model

...

$y_{t-4}$

$y_{t-3}$

$y_{t-2}$

$y_{t-1}$

# 什么是解码?

## Decoding

- 在每一时间步 $t$ ，模型会为词表中的每个标记计算一个得分向量 $S \in R^{|V|}$ ：

$$S = \underline{f(\{y\}_{<t}; \theta)} \quad f(\cdot; \theta) \text{ 为模型}$$

- 随后，我们基于这些得分，计算词表 $V$ 中每个词元 $w$ 对应的概率分布：

$$P(y_t = w | \{y\}_{<t}) = \frac{\exp(S_w)}{\sum_{w' \in V} \exp(S_{w'})}$$

- 解码算法**会定义一个函数，从上述概率分布中选取一个词元：

$$\hat{y}_t = \underline{g(P(y_t | \{y\}_{<t}))} \quad g(\cdot) \text{ 是解码算法}$$

# 如何找到最可能的字符串呢？

- 直观方法: 贪心解码

- 根据条件概率 $P(y_t|\{y\}_{<t})$ ，选择概率最高的词元
$$\hat{y}_t = \operatorname{argmax}_{w \in V} P(y_t = w|\{y\}_{<t})$$

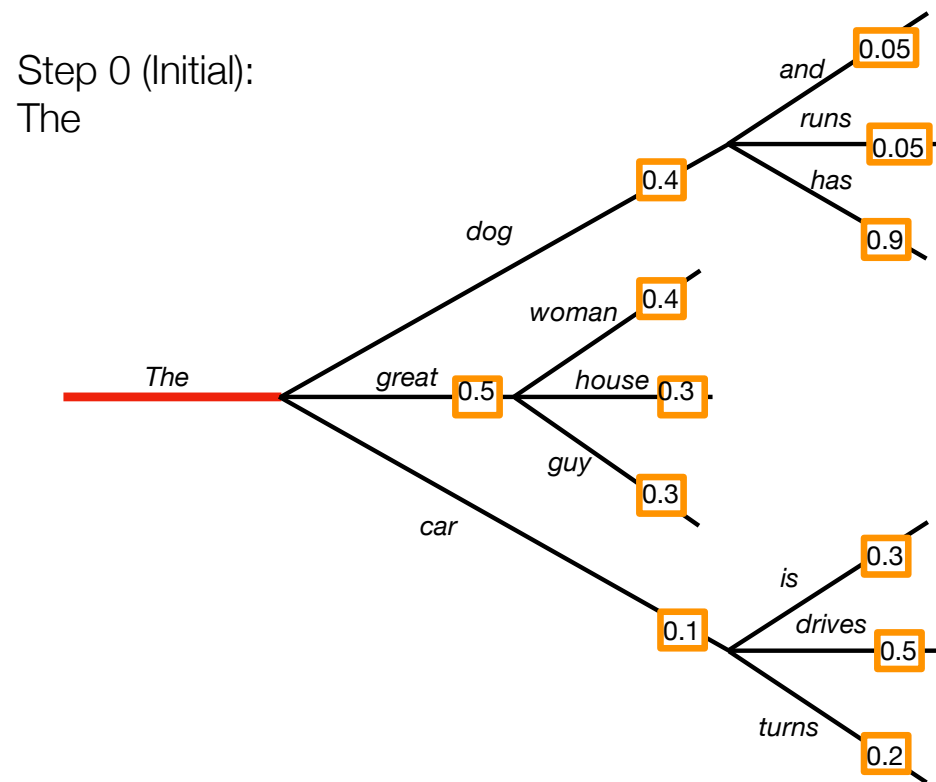
- 束搜索/Beam Search

- 同样以寻找概率最高的字符串为目标，但会对候选序列进行更全面的搜索

# 贪心解码 vs. 束搜索 Greedy Decoding vs. Beam Search

- 贪心解码

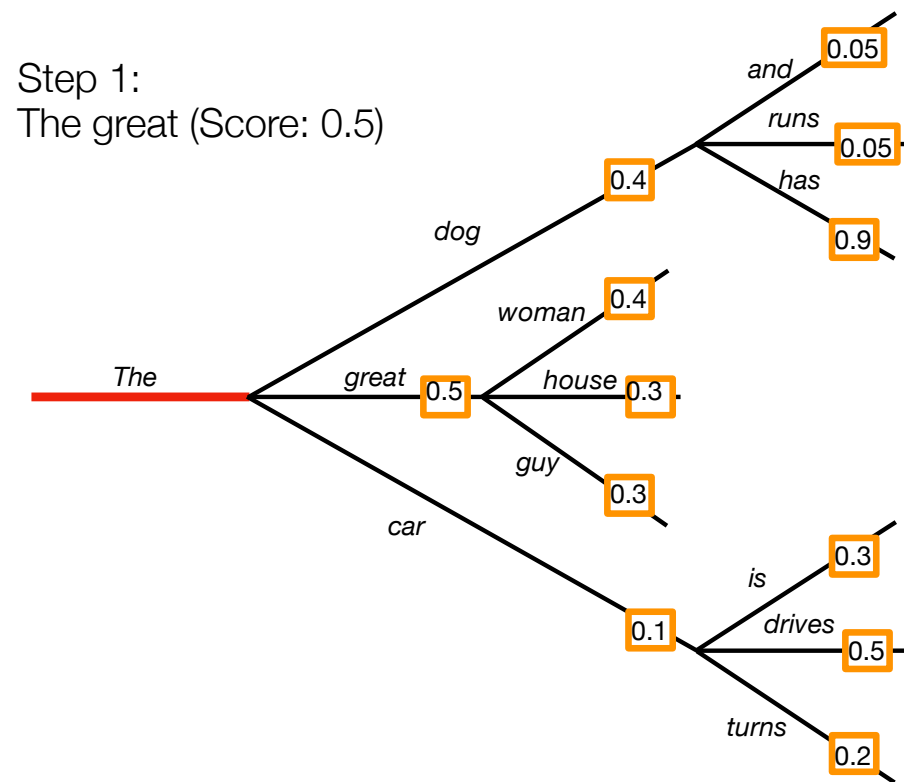
- 选择 “当前最好” 的词元



# 贪心解码 vs. 束搜索 Greedy Decoding vs. Beam Search

- 贪心解码

- 选择“当前最好”的词元



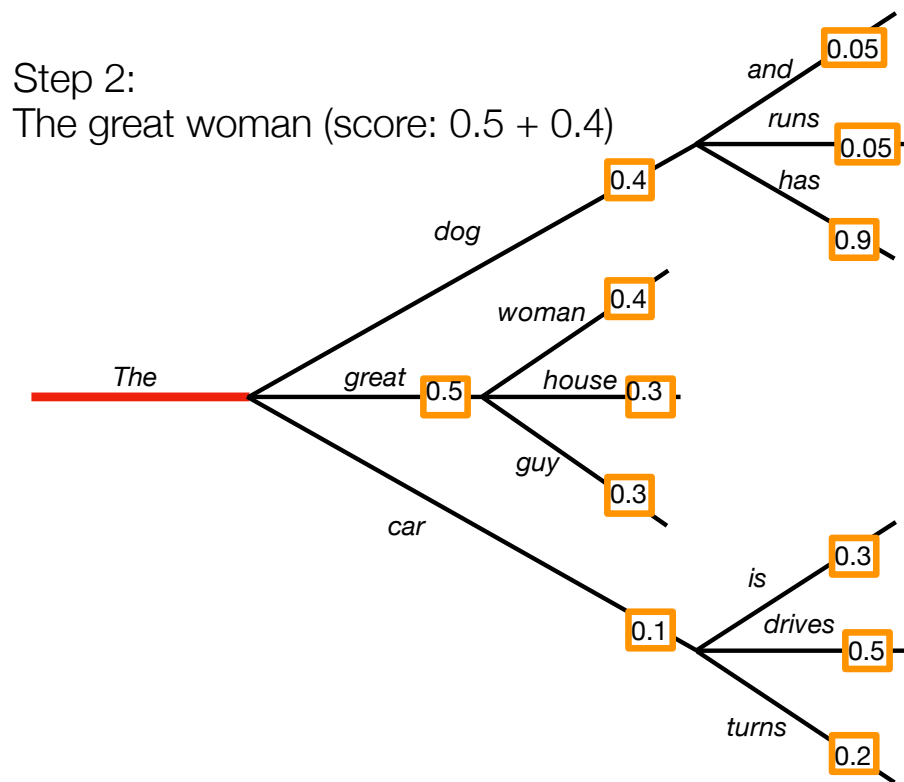


# 贪心解码 vs. 束搜索

## Greedy Decoding vs. Beam Search

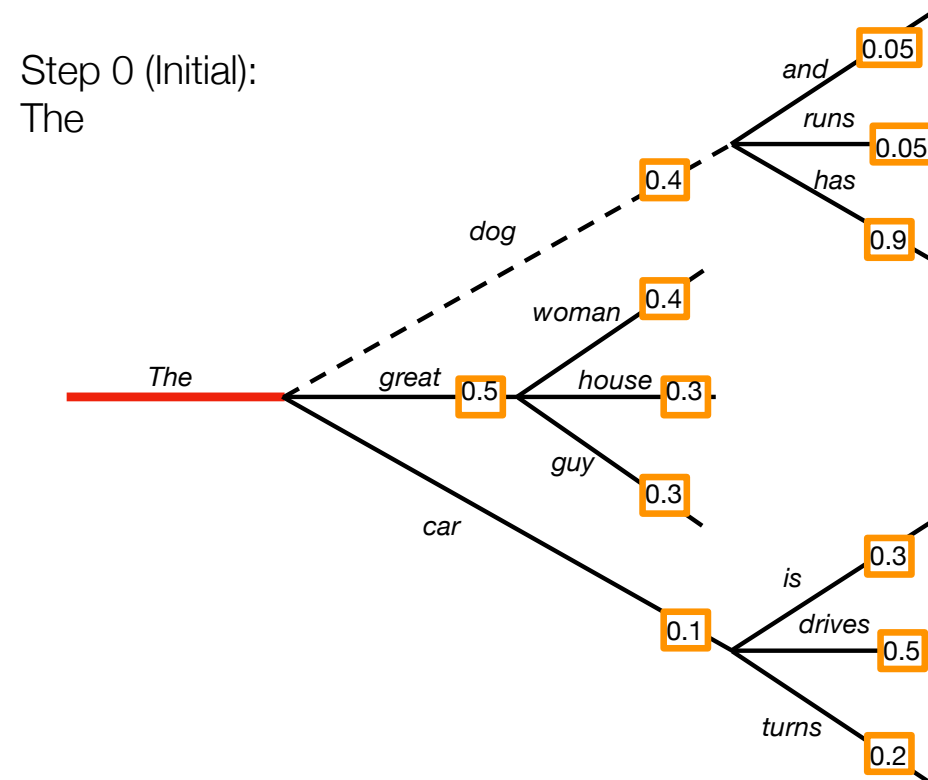
- 贪心解码

- 选择 “当前最好” 的词元



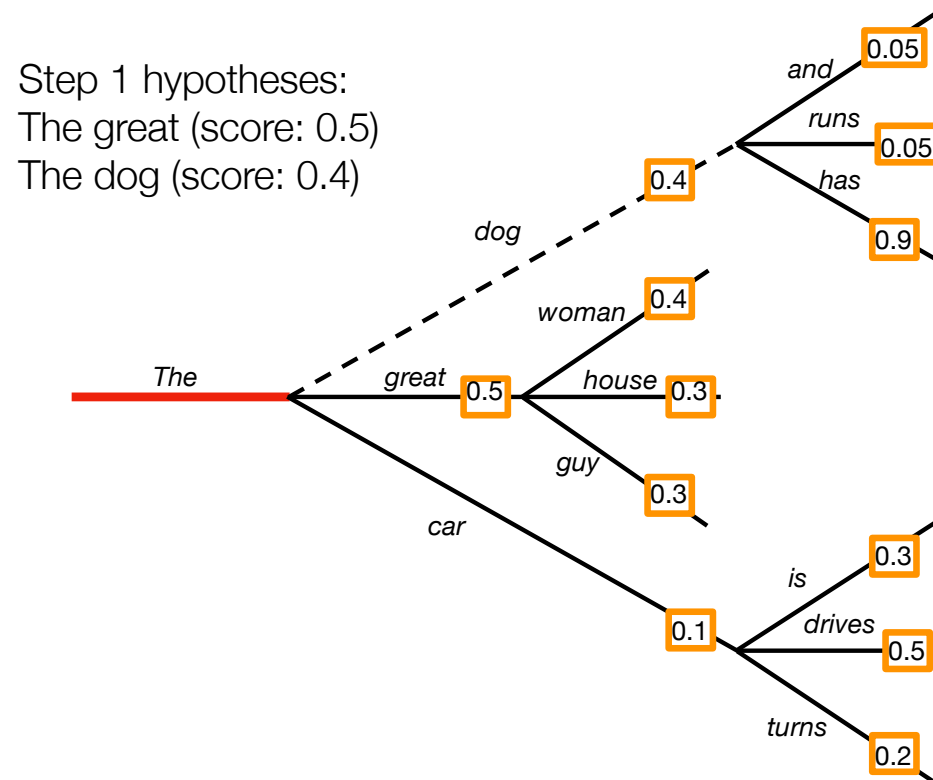
# 贪心解码 vs. 束搜索 Greedy Decoding vs. Beam Search

- 束搜索 (假设 *beam\_width* = 2)
  - 每一步保留概率最高的 2 个候选序列



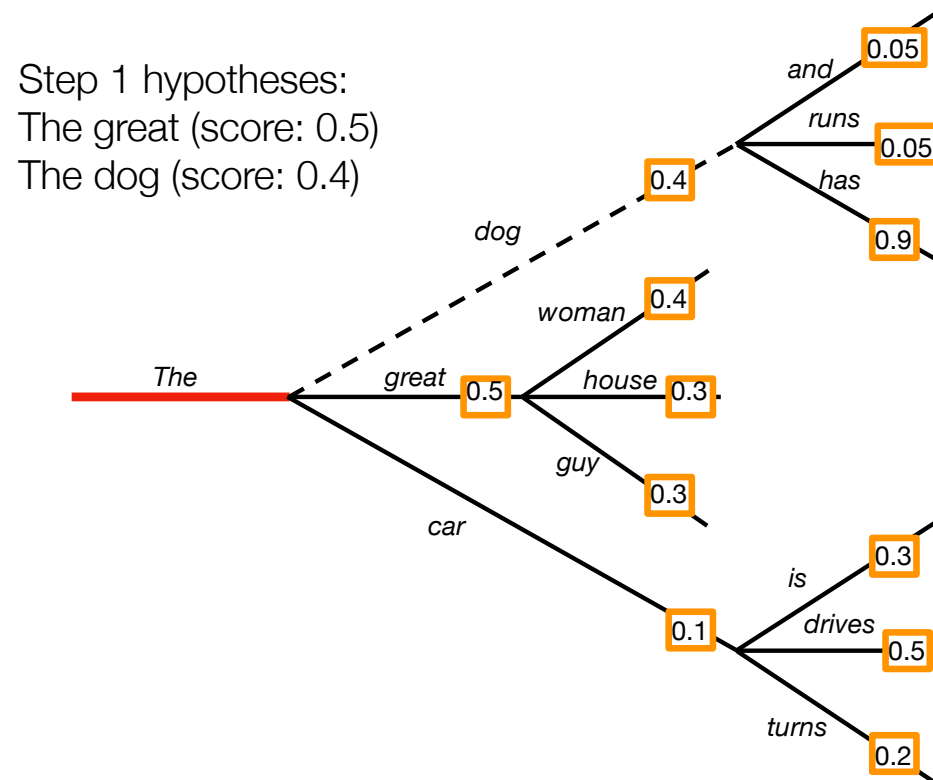
# 贪心解码 vs. 束搜索 Greedy Decoding vs. Beam Search

- 束搜索 (假设 *beam\_width* = 2)
  - 每一步保留概率最高的 2 个候选序列



# 贪心解码 vs. 束搜索 Greedy Decoding vs. Beam Search

- 束搜索 (假设 *beam\_width* = 2)
  - 每一步保留概率最高的 2 个候选序列



# 贪心解码 vs. 束搜索 Greedy Decoding vs. Beam Search

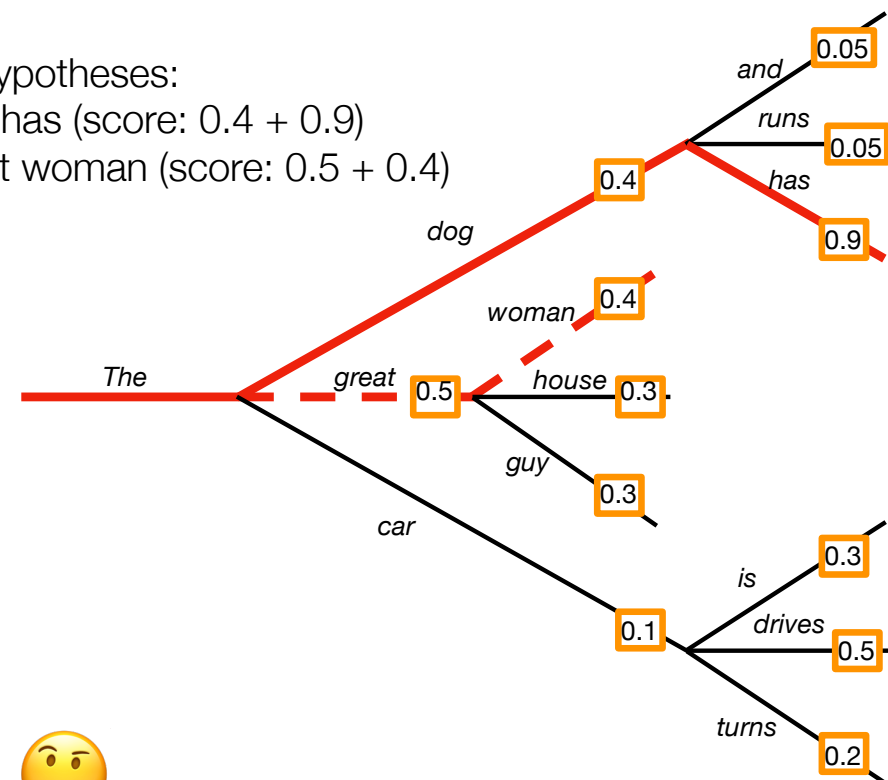
- 束搜索 (假设  $beam\_width = 2$ )

- 每一步保留概率最高的 2 个候选序列

Step 2 hypotheses:

The dog has (score:  $0.4 + 0.9$ )

The great woman (score:  $0.5 + 0.4$ )



注: 总体而言, 贪心搜索/束搜索被广泛应用于机器翻译、文本摘要等低熵/low-entropy任务中

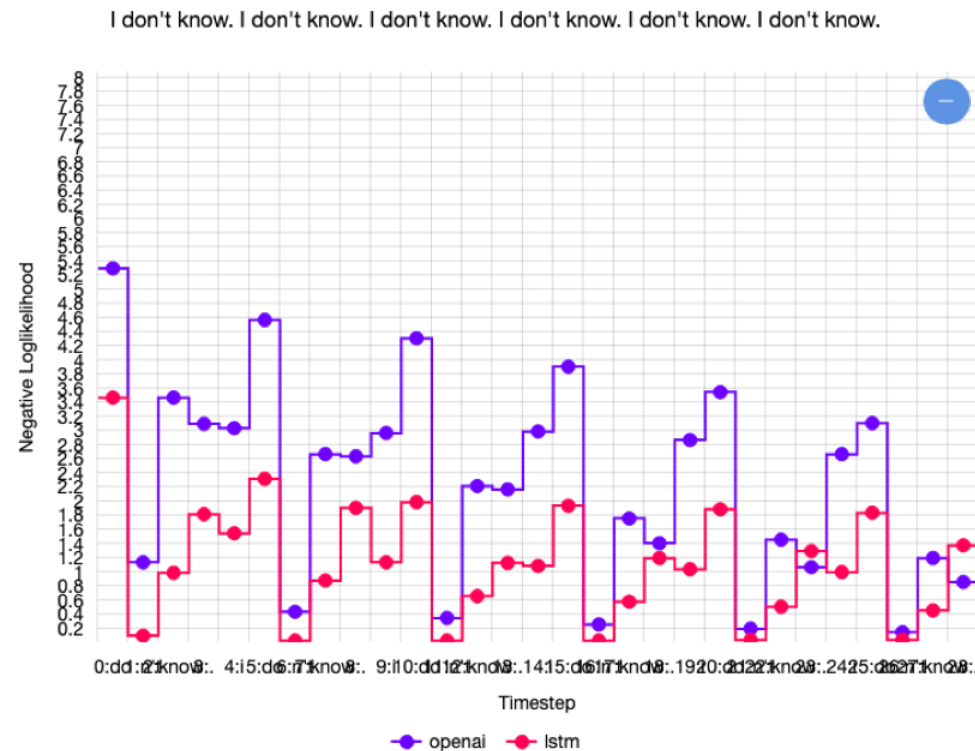
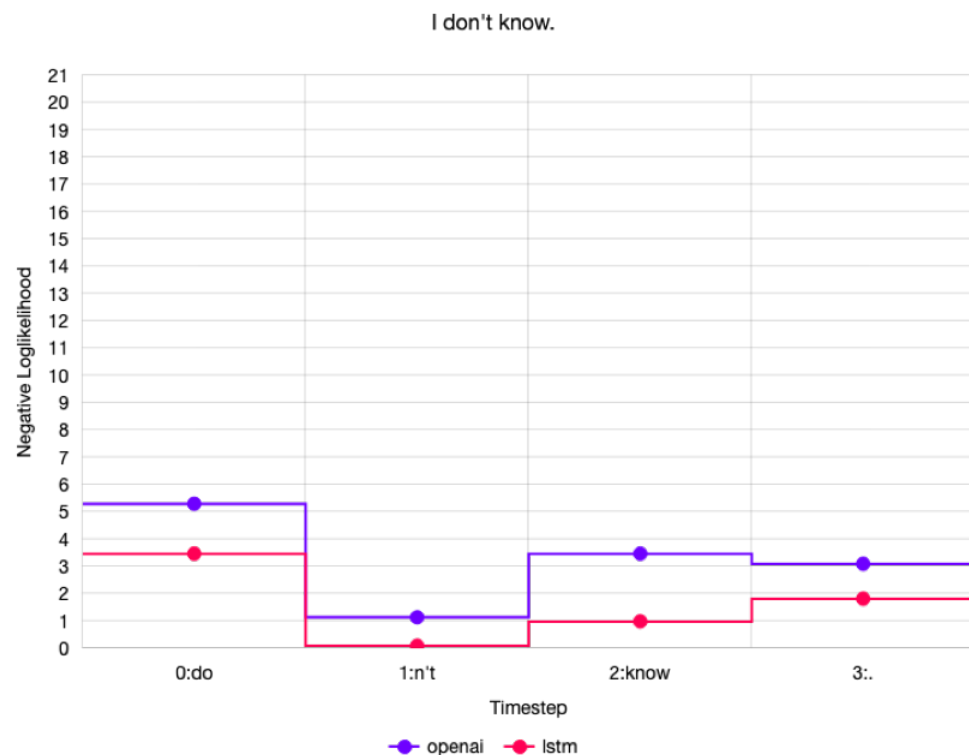
- 但问题是: 贪心搜索得到的序列就一定是最优吗? 🤔

# 问题:最可能的序列重复出现

**Context:** In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

**Continuation:** The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from **the Universidad Nacional Autónoma de México (UNAM)** and **the Universidad Nacional Autónoma de México (UNAM/ Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México...** [\[Holtzman et al. ICLR 2020\]](#)

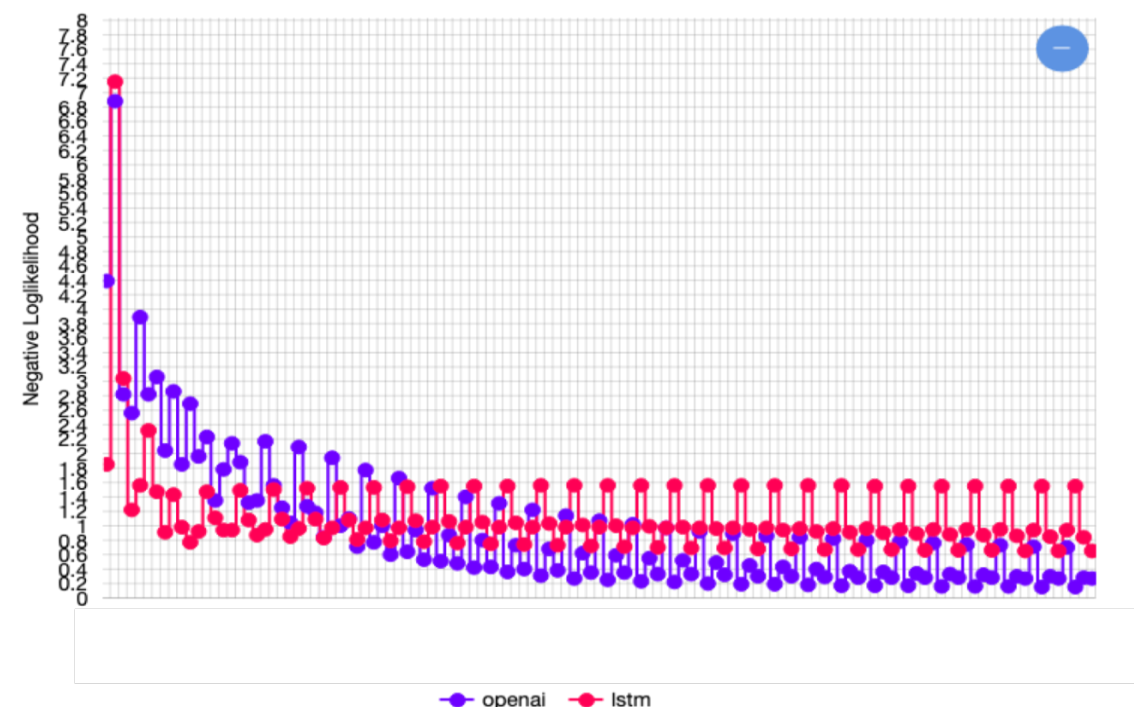
# 问题：最可能的序列重复出现



- “I don't know”这句话的出现概率会随重复次数的增加而上升，形成一个正反馈循环

# 此现象会循环往复...

I'm tired. I'm tired. I'm tired. I'm tired. I'm tired. I'm tired. I'm tired. I'm tired. I'm tired. I'm tired. I'm tired.



- 增大模型规模并不能解决这个问题—即使GPT-4这样的大模型，也有可能陷入重复生成的循环

Model: Code Interpreter

V run hellow word with code Interpreter

Sure, I can help you with that. However, I need to clarify which programming language you want me to use. For example, a simple "Hello, World!" program can look different depending on the language:

- In Python, it would be `print("Hello, World!")`.
- In JavaScript, it would be `console.log("Hello, World!")`.
- In C, it would be `printf("Hello, World!");`, and so on.

Could you specify which programming language you want to use?

V use code Interpreter

Sure, in this environment, I can run Python code. Here's how you can print "Hello, World!" in Python:

```
python
```

Copy code

```
print("Hello, World!")
```

Let's execute it.

Sure, let's run the Python code to print "Hello, World!".

Sure, let's run the Python code to print "Hello, World!".

Sure, let's run the Python code to print "Hello, World!".

Sure, let's run the Python code to print "Hello, World!".

Sure, let's run the Python code to print "Hello, World!".

Sure, let's run the Python code to print "Hello, World!".

Sure, let's run the Python code to print "Hello, World!".

Sure, let's run the Python code to print "Hello, World!".

V

Apologies for any confusion. Let's run the Python code to print "Hello, World!".

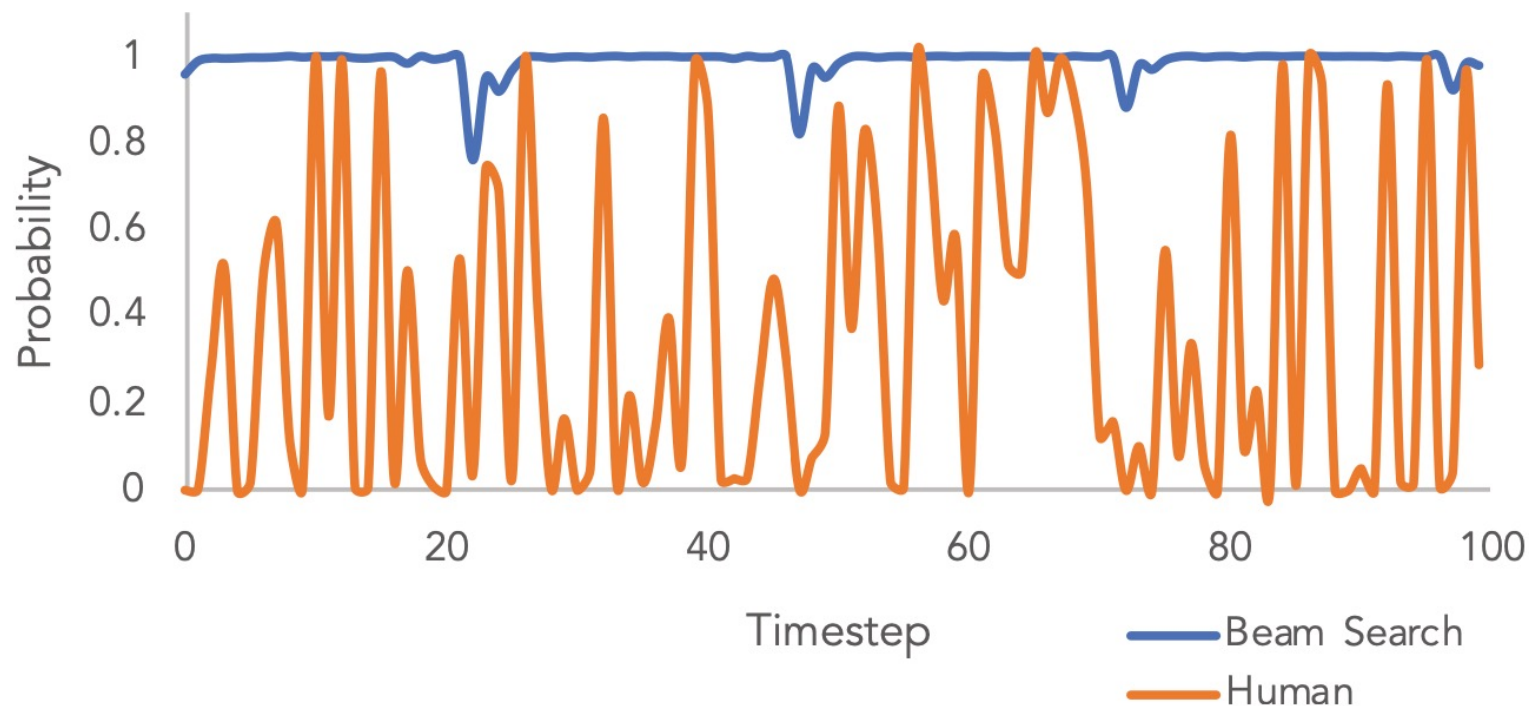
Apologies for any confusion. Let's run the Python code to print "Hello, World!".

Apologies for any confusion. Let's run the Python code to print "Hello, World!".

[[Holtzman et al. ICLR 2020](#)]



# 贪心策略适用于开放式生成任务吗？



- 贪心策略无法捕捉人类文本分布的多样性

[Holtzman et al. ICLR 2020]

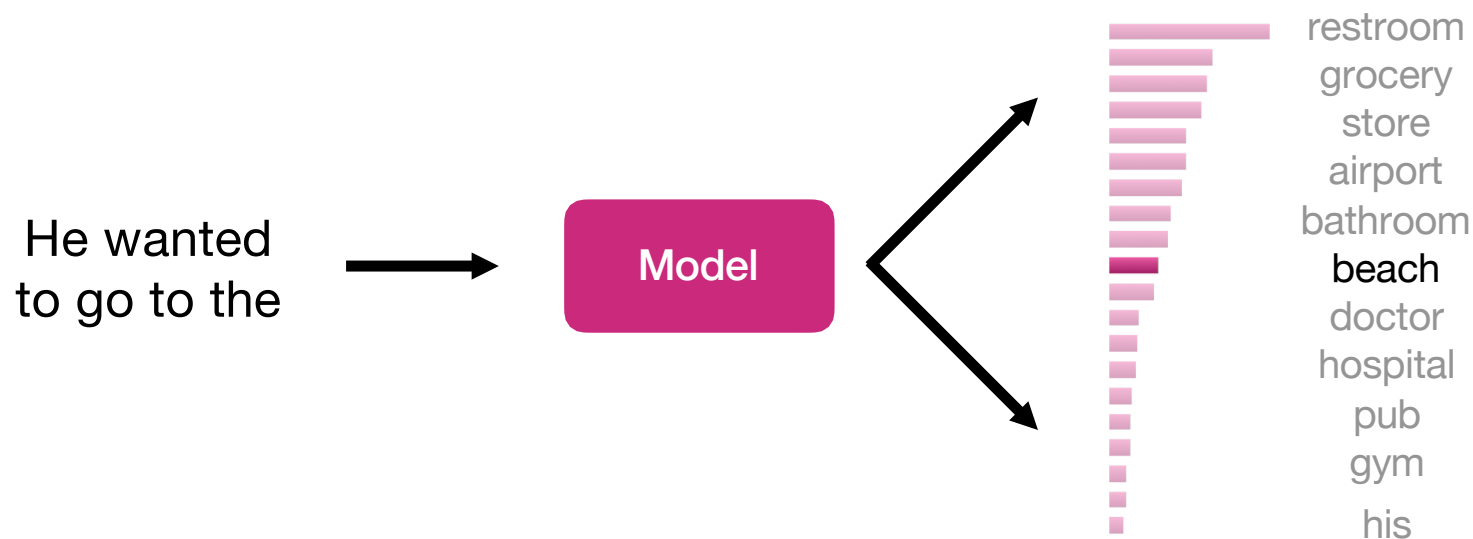
# 是时候引入随机性了: 采样法

## Time to Get random: Sampling

- 每一步从词元概率分布中**采样一个词元/sampling a token** !

$$\hat{y}_t \sim P(y_t = w | \{y\}_{<t})$$

- 由于其本质的随机性，因此可以采用获得任意词元



# 是时候引入随机性了: 采样法

**Context:** In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

### Beam Search, $b=32$ :

"The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the Universidad Nacional Autónoma de México (UNAM) and the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de ..."

## Pure Sampling:

They were cattle called **Bolivian Cavalleros**; they live in a remote desert **uninterrupted by town**, and they speak **huge, beautiful, paradisiacal Bolivian linguistic thing**. They say, 'Lunch, marge.' They don't tell what the lunch is," director Professor Chuperas Omwell told Sky News. "They've only been talking to scientists, like we're being interviewed by TV reporters. We don't even stick around to be interviewed by TV reporters. Maybe that's how they figured out that they're cosplaying as the Bolivian Cavalleros."

Figure 1: Even with substantial human context and the powerful GPT-2 Large language model, Beam Search (size 32) leads to degenerate repetition (highlighted in blue) while pure sampling leads to incoherent gibberish (highlighted in red). When  $b \geq 64$ , both GPT-2 Large and XL (774M and 1542M parameters, respectively) prefer to stop generating immediately after the given context.

# 解码策略: Top-K采样

- **问题:** 原始采样策略将词表中的所有词元均列为候选
  - 即使概率分布的大部分**概率密度**集中在少量候选词元上，分布的尾部也可能会非常长，且累积后具备相当大的概率占比(统计学角度: **重尾分布**"/heavy tailed" distribution)
  - 在此语境下，许多词元的选择很可能是不合理的
  - 尽管它们各自被分配的概率值较低，但累积之后，这类词元被选中的概率依然会很高
- **解决方案:** Top-k 采样 [[Fan et al., 2018](#)]
  - 仅从概率分布前k个词元中抽样

# 解码策略: Top-K采样

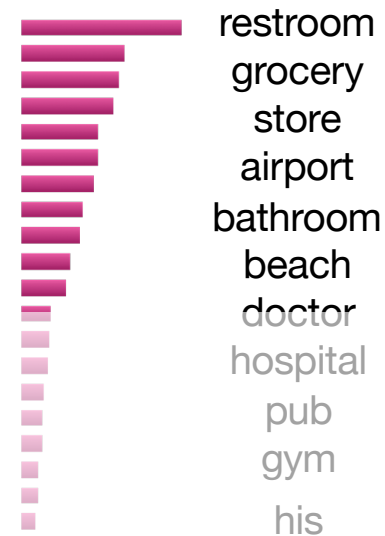
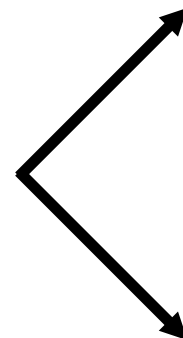
- **解决方案:** Top-k 采样 [[Fan et al., 2018](#)]

- 仅从概率分布前k个词元中抽样
- 常见值 $k=10, 20, 50$  (由你决定)

He wanted  
to go to the



Model



- 增大 $k$ 值会生成**多样性更强但风险更高**的输出
- 减小 $k$ 值则会生成**更稳妥但趋于同质化**的输出

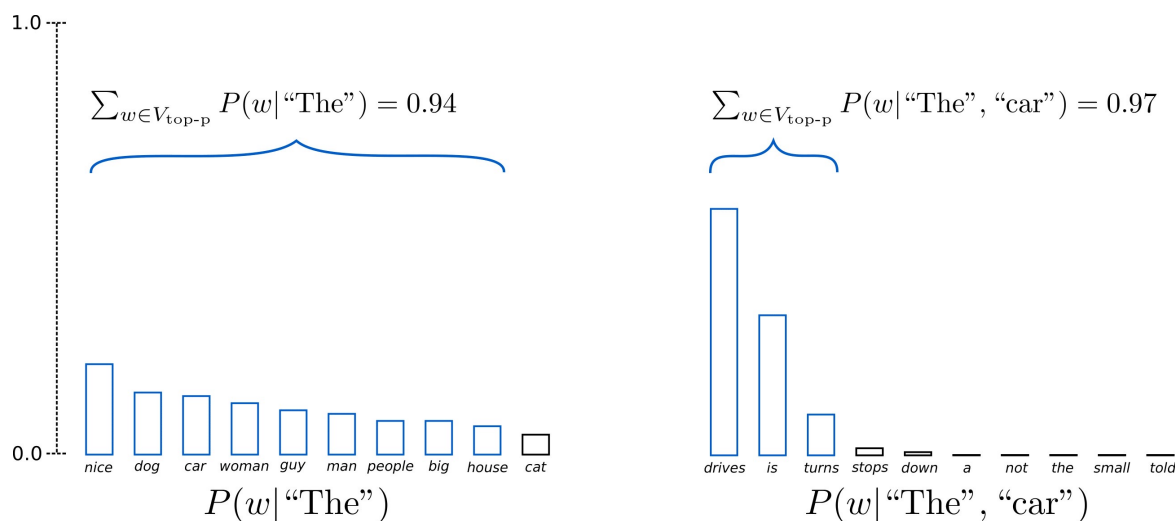
常规采样

贪心解码

# 解码策略: Top- $p$ (核心集) 采样

## Top- $p$ (nucleus) Sampling

- **解决方案**: Top- $k$  采样 [[Fan et al., 2018](#)]
  - 仅从**概率最高的最小词元集合**中采样—该集合内所有标记的累积概率需超过阈值  $p$
  - 通常,  $p$ 取值为 0.8、0.85、0.9、0.95、1 (具体数值可按需调整!)



- 增大  $p$  值会生成**多样性更强但风险更高**的输出
- 减小  $p$  值则会生成**更稳妥但趋于同质化**的输出

# 随机性调节策略: Softmax温度系数

- **回顾**：在时间步 $t$ ，模型会对一个向量进行Softmax操作，以此计算其概率分布 $P_t$ ：

$$P_t(y_t = w | \{y\}_{<t}) = \frac{\exp(S_w)}{\sum_{w' \in V} \exp(S_{w'})}$$

- 在这里，你可以将**温度超参数**应用于Softmax函数，以此重新调整概率分布 $P_t$ 的权重：

$$P_t(y_t = w | \{y\}_{<t}) = \frac{\exp(S_w/\tau)}{\sum_{w' \in V} \exp(S_{w'}/\tau)}$$

- 将温度 $\tau$ 调至大于1: 概率分布 $P_t$ 会变得更均匀
  - 输出内容的多样性更强(概率会分散到整个词表)
- 将温度 $\tau$ 调至小于1: 概率分布 $P_t$ 会变得更陡峭
  - 输出内容的多样性更弱(概率会集中在少数高概率词元上)



# 随机性调节策略: Softmax温度系数

- 你可以将**温度超参数**应用于Softmax 函数，以此重新调整概率分布 $P_t$ 的权重:

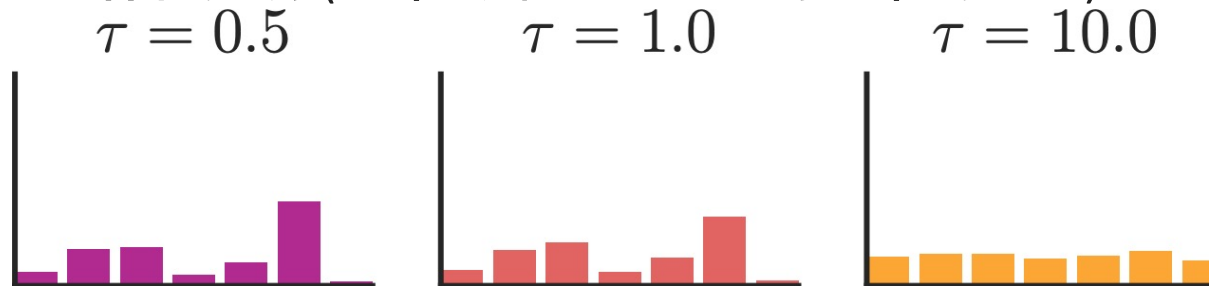
$$P_t(y_t = w|\{y\}_{<t}) = \frac{\exp(S_w/\tau)}{\sum_{w' \in V} \exp(S_{w'}/\tau)}$$

- 将温度 $\tau$ 调至大于1: 概率分布 $P_t$ 会变得更均匀

- 输出内容的多样性更强(概率会分散到整个词表)

- 将温度 $\tau$ 调至小于1: 概率分布 $P_t$ 会变得更陡峭

- 输出内容的多样性更弱 (概率会集中在少数高概率词元上)



- 注意：**温度系数**是**解码算法**的一个超参数，而非算法本身！它既可以应用于束搜索，也可以用于采样类方法